

Capitolul 1 Introducere

Domeniul algoritmilor paraleli și distribuiți preocupă de mai mult timp un număr mare de specialiști. Despre el, ca de altfel despre algoritmi în general, s-a scris mult și din diferite puncte de vedere. Un efort considerabil a fost depus pentru găsirea celor mai bune soluții paralele sau distribuite ale diverselor probleme, prin adaptarea unor soluții secvențiale cunoscute sau pornind de la premise conceptuale total distincte de acestea. A doua abordare ține cont de diferențele dintre calculul secvențial și calculul paralel, respectiv distribuit. Dacă în primul caz instrucțiunile programului sunt executate una după alta într-un același sistem, în calculul paralel mai multe părți ale unui program sunt executate simultan în procesoare diferite care comunică prin memorie comună, iar în calculul distribuit diferitele părți ale programului sunt executate în sisteme diferite care comunică doar prin transmitere de mesaje prin rețeaua care leagă între ele aceste sisteme. Algoritmii paraleli și distribuiți au la bază modele diferite de calculul secvențial (cel mai folosit este modelul (*Communicating Sequential Processes* dezvoltat de Hoare) care includ reprezentări ale concurenței, atomicizării și sincronizării diferitelor părți ale unui program. În plus, ei folosesc modele de programare care reprezintă comunicarea între procese prin date partajate sau prin mesaje. Având în vedere aceste aspecte, este ușor de înțeles că algoritmii paraleli și distribuiți **NU sunt** simple extensii sau versiuni ale celor secvențiali, pentru dezvoltarea lor fiind folosite alte **abordări**.

Lucrarea de față vizează studierea sistematică a algoritmilor paraleli și distribuiți pentru câteva din cele mai reprezentative clase de aplicații. Aceasta presupune adoptarea unui limbaj de descriere adecvat și utilizarea lui pentru prezentarea și analiza celor mai cunoscute soluții ale claselor de probleme selectate. Limbajul pseudocod pe care îl folosim, inspirat de propunerea lui G.R.Andrews [???], este **simplu**, ascunde detaliile de implementare pe care le găsim inevitabil în limbajele de programare sau bibliotecile specializate, dar încorporează conceptele și modelele de programare de care avem nevoie.

Descrierile algoritmilor sunt acompaniate de discutarea unor aspecte relative la metodele de dezvoltare a algoritmilor paraleli și distribuiți, precum și la analiza complexității lor. Demersul nostru ajută la deprinderea metodelor de proiectare (ingineria algoritmilor paraleli și distribuiți) și stimulează inițiativa în găsirea unor soluții noi.

Dacă în urmă cu câțiva ani studierea algoritmilor paraleli și distribuiți era apanajul

unui grup relativ restrâns de specialiști, situația actuală este cu totul diferită. Modelele și tiparele de rezolvare paralelă și distribuită sunt folosite curent, unele pe scara largă. Un exemplu este modelul Map-Reduce de la Google. În plus, calculatoarele moderne au mai multe procesoare sau procesoare “multi-core”. În fine, compilatoarele pentru limbajele concurente (Java) și bibliotecile de comunicare a mesajelor (MPI) sunt furnizate de toate firmele mari de software.

Pentru a încadra mai bine subiectul, vom încerca să răspundem succint la următoarele întrebări:

- care sunt principalele categorii de aplicații paralele și distribuite?
- care sunt principalele clase conceptuale?
- care sunt principalele metode de programare?
- care sunt principalele clase de sisteme folosite?

1.1. Categoriile de aplicații

Există mai multe motive pentru a utiliza calculul paralel în realizarea aplicațiilor: scurtarea timpului de execuție, realizarea unei fiabilități crescute, specializarea funcțională și natura inerent paralelă a aplicațiilor.

Una din căile de **creștere a performanțelor** unei aplicații este execuția simultană a diferitelor părți ale programului (părți pe care le numim procese) pe procesoare distincte. De cele mai multe ori, procesele nu sunt complet independente, ci comunică între ele. În funcție de frecvența comunicărilor, paralelismul are o **granularitate** mai mare (comunicări rare) sau mai fină (comunicări frecvente).

O mare parte dintre algoritmi paraleli prezentați în literatură se referă la această categorie de aplicații. Problemele care trebuie rezolvate aici sunt legate atât de găsirea unei împărțiri adecvate în procese a prelucrărilor ce trebuie realizate, cât și de găsirea celor mai potrivite mecanisme de comunicare sau sincronizare între procese.

Pentru aplicații la care **siguranța în funcționare** este un factor critic (de exemplu, comanda unui avion), replicarea unor părți ale lor (procese sau date) poate constitui o soluție de toleranță la defecte. Dacă anumite procesoare se defectează, replicile lor le pot suplini. Ceea ce interesează aici sunt tehnicile de realizare corectă a comutării, cu alte cuvinte protocoalele ce asigură comportarea corectă în sisteme în care funcționalitatea componentelor este parțială.

Unele aplicații pot fi împărțite în faze cu **funcționalități distincte**: de calcul în virgulă mobilă, de prelucrări grafice, de prelucrare de fișiere, de imprimare etc. Aceasta permite exploatarea mai bună a caracteristicilor diferitelor componente ale sistemelor de calcul, pentru realizarea unor performanțe deosebite (folosirea unor procesoare în virgulă mobilă, a unor procesoare grafice, a unor servere de fișiere sau de imprimare). Problemele importante sunt aici cele legate de partajarea eficientă a resurselor sistemelor de calcul.

Unele aplicații sunt **inerent paralele**. Un exemplu îl constituie accesul simultan la o bază de date. Problemele importante aici sunt excluderea mutuală și protecția utilizatorilor.

Sunt patru categorii de *specialiști* interesați de calculul paralel și distribuit:

- specialiști în calculatoare (computer scientists) din domeniile arhitectura, limbaje, algoritmi (căutare și sortare, recunoașterea formelor, toleranța la defecte, algoritmi cu înalt grad de paralelism, algoritmi cu ascunderea latentelor), metodologii;
- specialiști în calcul numeric (computational scientists) interesați în pachete de programe pentru calcul matriceal, sisteme de ecuații lineare și nelineare, cu derivate parțiale etc.
- alte domenii științifice și ingineresti (tehnologii și materiale noi, biomedicina, fizica nucleară și altele)
- comerciali (data mining, analize de risc etc.)

Cerințele acestora și caracteristicile aplicațiilor lor sunt prezente succint în tabelul 1.1.

Tabel 1.1

Domeniu	Științific și ingineresc	Comercial
Caracteristici	virgulă mobilă cod de dimensiune redusă rata I/E redusă puține date puțini utilizatori	întregi cod de dimensiune mare rata I/E mare multe date mulți utilizatori
Cerințe	performanța scalabilitate	performanța siguranța în funcționare disponibilitate

1.2. Clase conceptuale

Desprinderea unor clase conceptuale în paralelism este importantă ca sprijin în prima fază de abordare a unei probleme. Încadrarea într-una din aceste clase poate dirija proiectantul pe parcursul conceperii soluției paralele a unei probleme. Clasificarea adoptată aici este una din mai multele prezentate în literatura de specialitate. Ea aparține lui Carriero și Gelernter și privește paralelismul în termenii:

- rezultatului unui program,
- agendei de activități a programului,
- unui ansamblu de specialiști care împreună constituie programul.

Pentru a înțelege mai bine fiecare concept, vom lua ca exemplu construcția unei case. Folosirea mai multor persoane la acest lucru conduce inerent spre paralelism. Dar, există mai multe căi de folosire a lucrătorilor.

Putem porni prin divizarea produsului final în mai multe componente: fundație, pereți, ferestre, uși, contor electric și conductori, contor de apă, țevi, chiuvete etc. Putem apoi trece la construirea simultană a componentelor și la asamblarea lor, pe măsură ce sunt terminate. Fiecărui lucrător i se repartizează o parte a produsului final, de a cărei construire se îngrijește. Toți pornesc lucrul simultan și lucrează până în momentul când continuarea depinde de o anumită restricție (de exemplu, de o altă componentă). Acesta este paralelismul **rezultat**, numit astfel deoarece rezolvarea unei probleme pleacă de la împărțirea rezultatului în mai multe fragmente, care sunt calculate în paralel, luând în considerație (bineînțelese) restricțiile impuse de problemă.

Un exemplu tipic este calculul sumei a doi vectori de **n** elemente, $S = A+B$. Elementele lui S sunt independente între ele și pot fi calculate în paralel, într-un singur pas, folosind **n** procesoare, după următoarea schemă:

`co [j = 1 to n] {S[i] = A[i] + B[i]}`

O altă posibilitate este de a alcătui o **agendă de activități** necesare pentru a construi casa. Unele dintre ele se vor executa în paralel, altele sunt condiționate de o anumită ordine. Oricum, lucrătorii vor fi repartizați activităților curente din agendă, în numărul necesar și fără a se face vreo diferențiere între ei. Fiecare lucrează cât timp restricțiile naturale ale problemei le permite acest lucru.

Cea mai cunoscută **paradigmă** de implementare a acestei clase de paralelism este “replicated workers” sau “master-worker”. Un proces **master** inițializează calculele și crează un număr de procese **worker** identice. În mod repetat, fiecare worker preia o sarcină și o execută. Programul se execută la fel, indiferent de numărul proceselor worker și se termină odată cu epuizarea sarcinilor.

Pentru exemplificare, să considerăm o problemă de minim. Dat fiind un ansamblu E de înregistrări, $E = \{r\}$ și o funcție de cost $f:E \rightarrow R$ care asociază fiecărei înregistrări r un cost $f(r)$, se cere **min** $f(r)$ cu r din E . Procesul master umple un sac cu înregistrările din E . Fiecare proces worker extrage o înregistrare r , calculează $f(r)$ și, dacă găsește o valoare inferioară minimului parțial curent, o transmite master-ului. Acesta actualizează minimul curent. Programul se termină la “golirea” sacului.

În fine, putem avea în vedere faptul că lucrătorii sunt specializați și că realizarea unei case reclamă diferite specialități. Toți lucrătorii pot începe simultan. Inițial unii vor aștepta, până când condițiile necesare efectuării operației pentru care sunt specializați sunt îndeplinite. Oricum, la un moment dat, vor lucra mai multe persoane. Aceasta este abordarea paralelismului ca **ansamblu de specialiști**.

Această abordare este specifică lucrului în “banda de asamblare” (pipeline) și se pretează în cazul prelucrării mai multor obiecte identice (de exemplu un grup de case). În cazul general, ea conduce la programe organizate ca o rețea logică, în care nodurile execută calcule relativ autonome, iar comunicarea între noduri se face după tipare predictibile. Un exemplu tipic este cel al simulării unor circuite, în care funcționarea fiecărei componente este reprezentată de descrierea unui proces.

Autorii acestei clasificări insistă asupra faptului că diversele clase nu sunt net diferențiate între ele. De asemenea, evidențiază că **paralelismul de date**, considerat de alți autori ca o clasă conceptuală importantă, nu este decât o formă particulară a **agendei de activități**. În paralelismul de date, fiecare operație este aplicată sincron tuturor componentelor unei structuri de date. Asupra acestui subiect vom reveni, arătând în ce mod alegerea unei clase conceptuale influențează rezultatul proiectării.

1.3. Metode de programare

Odată stabilită clasa conceptuală cea mai potrivită pentru aplicație, urmează proiectarea algoritmului folosind o metodă de programare adecvată.

În programarea folosind **comunicare de mesaje** (figura 1.1), fiecare proces încapsulează structurile de date folosite (locale procesului), comunicarea între procese făcându-se prin mesaje. Fiecare proces are acces doar la datele sale locale. Pentru comunicare, procesele includ acțiuni explicite de transmitere și recepție de mesaje.

Comunicarea de mesaje exprimă într-un mod natural paralelismul “specialist”. Fiecare proces este un nod al rețelei de specialiști, între care comunicarea se face prin mesaje.

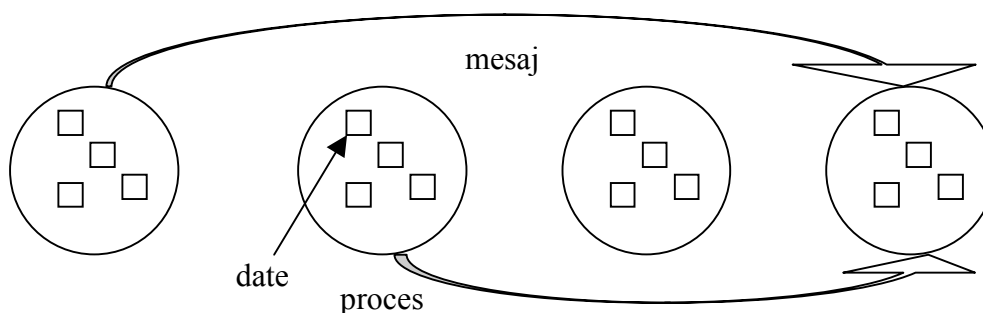


Figura 1.1

O abordare complet diferită (figura 1.2) încapsulează fiecare proces într-un obiect. Procesul este responsabil doar de calculul valorii corespunzătoare obiectului respectiv. Mediul exterior este interesat doar în cunoașterea valorii unui obiect, nu și în modul în care ea este calculată. Comunicarea nu se face prin acțiuni de transmitere și recepție. Pur și simplu, când un proces are nevoie de valoarea produsă de un alt proces, el o citește prin acces la obiectul respectiv. Aceste obiecte sunt așa numitele **structuri active (vii) de date**, sau obiecte active.

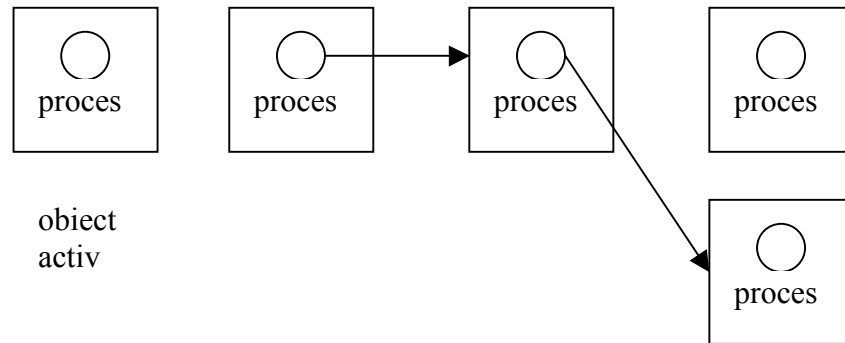


Figura 1.2

Această metodă exprimă bine paralelismul rezultat. Reluând exemplul sumei a doi vectori, fiecare obiect poate reprezenta un element $S[i]$ al vectorului *suma* și poate încapsula procesul ce calculează valoarea sa.

Între cele două abordări extreme se află cea la care procesele și datele sunt entități distincte. Procesele pot comunica prin citirea și scrierea **datelor partajate** (figura 1.3).

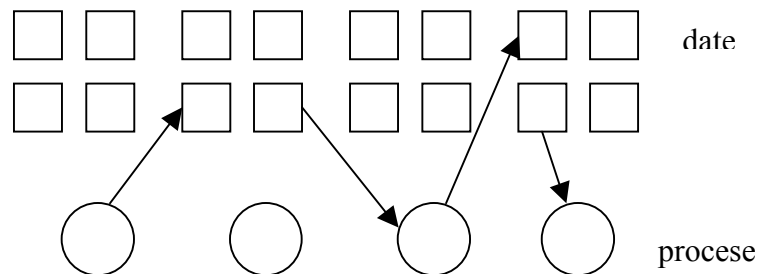


Figura 1.3

Această metodă se potrivește bine cu paralelismul “agenda”, datele partajate reprezentând un mijloc eficient de comunicare între procesele *master* și *worker*.

Tabelul 1.2 arată o corespondență posibilă între clasele conceptuale și metodele de programare paralelă. Ea nu trebuie interpretată rigid. Așa cum pentru o problemă de

rezolvat, programatorul are la dispoziție modele aparținând, eventual, unor clase conceptuale diferite, în același fel, pentru un model el are la dispoziție mai multe metode de programare.

Tabel 1.2

Clase conceptuale	Metode de programare
rezultat	obiecte active
agenda de activități	date partajate
ansamblu de specialiști	comunicare de mesaje

1.4. Clase de sisteme paralele și distribuite

În principiu, ar trebui ca efortul de găsim a unor algoritmi paraleli eficienți să nu țină seama de particularitățile unora sau altora din mașinile pe care se face implementarea acestora, ci să caute să exploateze cât mai bine paralelismul inherent problemei studiate. În practică, însă, trebuie avute în vedere detalii de implementare legate de mașina, care pot afecta (uneori într-o măsură importantă) timpul de execuție.

Una din clasificările clasice adoptate pentru calculatoarele paralele este cea a lui **Flynn** (1966). Ea consideră fluxurile de instrucțiuni și de date, pentru a distinge între următoarele categorii:

- SISD -Single Instruction stream Single Data stream (flux unic de instrucțiuni, flux unic de date);
- SIMD -Single Instruction stream Multiple Data stream;
- MISD -Multiple Instruction stream Single Data stream;
- MIMD -Multiple Instruction stream Multiple Data stream.

O altă clasificare, asemănătoare precedentei, consideră ca elemente de bază procesoarele și memoria, făcând distincție între următoarele categorii de sisteme:

- tablouri de procesoare (processor array)
- procesoare în linie de asamblare (pipeline processors)
- sisteme cu memorie partajată (multi-processors)
- sisteme cu transmitere de mesaje (distributed systems).

Deși există legături evidente între cele două clasificări, nu se poate face între ele o corespondență unu la unu. Astfel, deși modelul SIMD descrie tabloul de procesoare, el este folosit uneori și pentru procesoare în **linie de asamblare**. Pentru acestea din urmă, modelele SISD și MISD sunt, de asemenea, de multe ori aplicabile. A doua

clasificare prezentată este uneori preferată pentru corespondențele directe pe care le are în programare. Reprezentările grafice asociate modelelor lui Flynn sunt cele din figura 1.4.

SISD este modelul “von Newmann” din anii '40. El nu include, în esență, forme de paralelism, operațiile unui algoritm fiind executate secvențial, de un singur procesor.

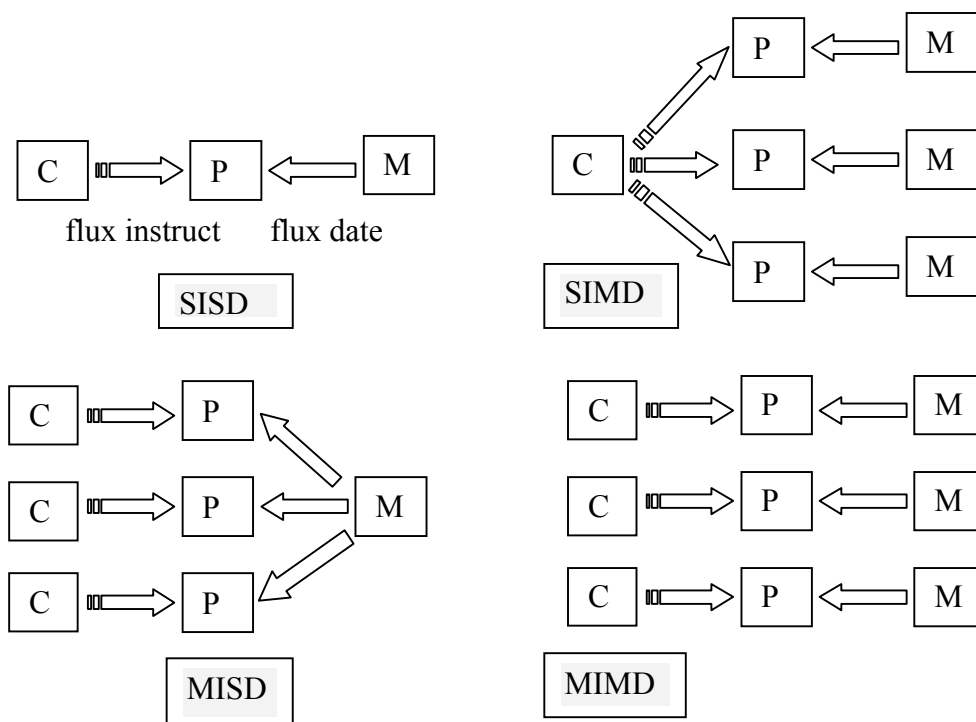


Figura 1.4
C - Control, P - Procesor, M - Memorie

În modelul MISD există un singur flux de date și mai multe fluxuri de control. În fiecare pas, procesoarele au acces la o aceeași dată din memorie, căreia îi pot aplica tratamente diferite, corespunzătoare unor fluxuri de instrucțiuni diferite. De exemplu, putem stabili dacă un număr natural este sau nu prim, într-un singur pas, dacă există un număr de procesoare egal cu numărul de divizori potențiali. Mai general, procesoarele pot face operații diferite asupra unei aceleiași valori: primul poate afla

rădăcina pătrată, al doilea calculează puterea a treia a aceluiași număr, iar un alt procesor află dacă numărul este sau nu prim.

Spre deosebire de modelele la care ne vom referi în continuare, MISD este mai rigid și are o utilitate practică foarte restransă.

Un **tablou de procesoare** (figura 1.5) constă dintr-un număr de procesoare identice, aflate sub controlul unei singure unități de comandă. Fiecare procesor are acces separat la datele sale locale, astfel încât, la un moment dat, o aceeași operație este executată simultan asupra mai multor unități de date. Acest mod de lucru sincron este tipic *tablourilor de procesoare*. În mod natural, operațiile caracteristice sunt cele matriceale. Așa cum s-a menționat, fiecare procesor are propria sa memorie locală. Pentru actualizarea ei este necesară comunicarea cu celelalte procesoare sau cu un calculator gazdă. Comunicarea poate fi realizată în două moduri, de unde rezultă două clase de sisteme SIMD: prin memorie partajată (SM - shared memory - SIMD) și prin rețea de interconectare.

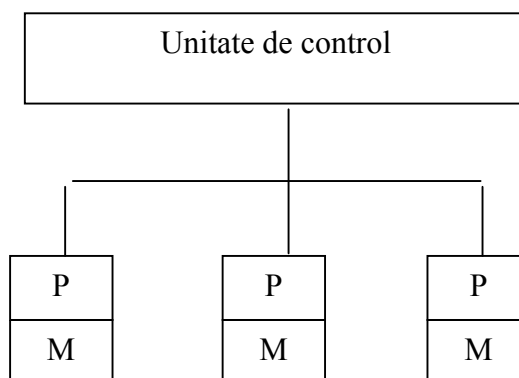


Figura 1.5

Primele sisteme, cunoscute și sub numele de **PRAM** (Parallel Random - Access Machine), au diferite variante, după cum accesul la aceeași locație de memorie pentru *citire* sau *scriere* sunt concurente sau exclusive. Evident că performanțele algoritmilor sunt influențate de acest aspect, așa cum rezultă și din exemplul următor, care se referă la o problemă de căutare.

Fie un fișier cu n intrări, unde n este foarte mare. Dându-se o valoare oarecare a , se cere să se precizeze dacă ea este în fișier. Considerăm că fișierul este împărțit în N

sub-fișiere, fiecare sub-fișier S_i , de dimensiune n/N , fiind accesibil unui procesor P_i distinct (sunt deci N procesoare $P_1, \dots, P_N$). Fiecare procesor P_i citește valoarea lui a , caută această valoare în sub-fișierul S_i și, în caz de succes, setează valoarea unei variabile rezultat R .

Problema poate fi rezolvată în n/N pași. Condiția este însă ca accesul la valoarea lui a pentru citire să fie concurrent. În plus, dacă fișierul *nu are înregistrări distincte*, este necesar ca și accesul la scriere pentru R să fie concurrent.

Dacă prima condiție nu este îndeplinită, valoarea lui a trebuie **difuzată** explicit tuturor proceselor. Deoarece comunicarea între procese se poate face doar prin memoria comună, difuzarea presupune folosirea unor variabile în această memorie.

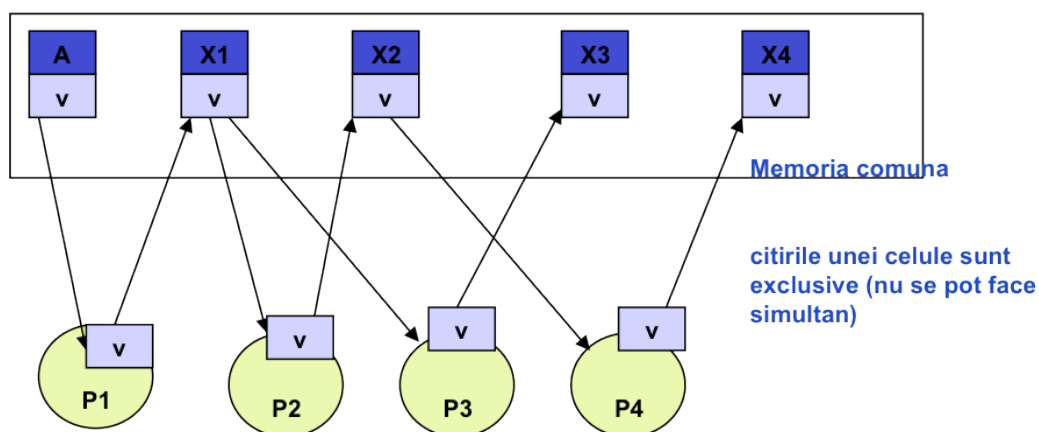


Figura 1.6

Difuzarea se poate face astfel (Figura 1.6):

- P_1 citește valoarea lui a într-o variabilă proprie și o memorează apoi în $x1$ din memoria comună;
- P_2 citește valoarea din $x1$ în variabila sa proprie și o memorează în $x2$ din memoria comună;
- P_3 și P_4 citesc valoarea din variabilele $x1$ respectiv $x2$, o memorează în variabilele lor proprii și apoi o memorează în $x3$ și $x4$ din memoria comună;
- P_5, P_6, P_7 și P_8 citesc valoarea din variabilele $x1, x2, x3$, și $x4$, o memorează în variabilele lor proprii și apoi o memorează în $x5, x6, x7$ și $x8$ din memoria comună ș.a.m.d.

Deoarece numărul de procese care cunosc valoarea lui **a** se dublează la fiecare pas, rezultă că difuzarea durează $\log_2 N$ pași. Întregul algoritm reclamă deci $\log N + n/N$ pași în cazul cel mai defavorabil.

Există și variante de terminare a algoritmului imediat ce un procesor a regăsit valoarea lui **a** în sub-fișierul corespunzător. În acest caz valoarea răspunsului **R** trebuie difuzată celorlalte procese. De data aceasta, valoarea rezultatului este inspectată de fiecare proces, înainte de a continua căutarea în propriul fișier, ceea ce conduce la un total de $\log N + (n/N) \cdot \log N$ pași în cazul cel mai defavorabil.

În privința sistemelor **SIMD interconectate** în rețea, topologiile uzuale sunt mai simple decât graful complet. Prezentăm câteva variante:

- a) tabloul uni-dimensional
- b) tabloul bi-dimensional
- c) arbore binar complet
- d) amestecare perfectă (perfect shuffle)
 - date fiind N procesoare $P_0, \dots, P_{N-1}$, cu N o putere a lui 2, P_i și P_j sunt legate printr-o linie uni-direcțională, unde

$$j = 2 \cdot i \quad \text{pentru } 0 \leq i \leq N/2 - 1$$

$$j = 2 \cdot i - N + 1 \quad \text{pentru } N/2 \leq i \leq N - 1.$$

Adesea, fiecare procesor cu număr par este legat prin conexiune bi-direcțională cu succesorul său. Pentru $N=8$, se obține configurația din figura 1.7.

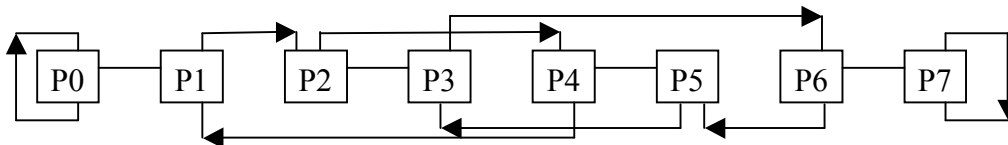


Figura 1.7

- e) cub
- f) hipercub.

Alegerea unei configurații depinde de aplicație, de performanțele dorite și de numărul procesoarelor disponibile. Astfel, pentru suma a n numere, o topologie avantajoasă este arborele cu $n/2$ frunze și $\log n$ nivele. Operația se realizează în $\log n$ pași (vezi figura 1.8).

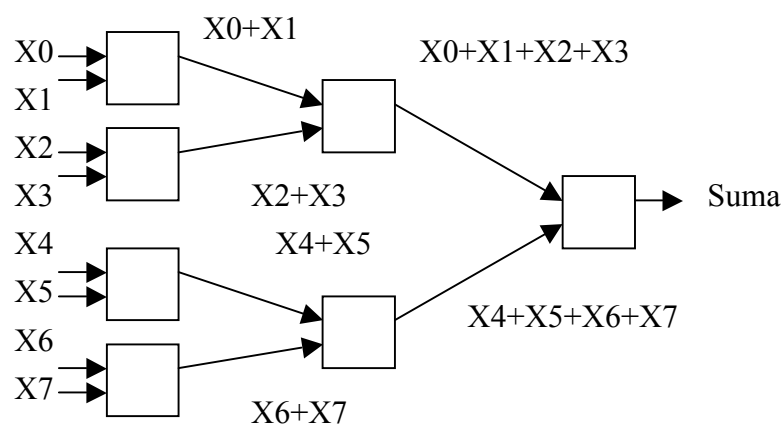


Figura 1.8

În cazul a m grupuri de câte n numere, sumele pot fi aflate în $\log n + m - 1$ pași, folosind tehnica liniei de asamblare (pipeline), la care ne referim ceva mai departe în această secțiune.

Exemplele de sisteme SIMD includ mai multe modele comerciale, dintre care cele mai cunoscute sunt **CM-2** produs de Thinking Machines și **MP-2** produs de Maspar. Din păcate, nici unul din aceste modele (care se bazează pe matrice de procesoare) nu a cunoscut succesul de piață. În schimb, mai larg acceptate au fost sistemele vectoriale, dintre care modelele **Cray-1**, **C90** și **T90** au dominat domeniul calculului științific. Mai recent, unitățile de procesare grafică **GPUs** (Graphics Processing Units) sunt implementate ca mașini SIMS, fiind adesea utilizate și în alte categorii de procesări decât cele grafice.

Exemple de limbaje de programare: DAP-FORTRAN, Actus (bazat pe Pascal), CMLISP (Connection Machine Lisp). Caracteristicile sistemelor se reflectă la nivelul limbajelor. Astfel, pentru FORTRAN, tablourile sunt tratate ca tipuri fundamentale, cu operatori specifici operațiilor matriceale.

Prelucrarea în **linie de asamblare** este un principiu utilizat în multe fabrici și este împrumutat în sistemele de calcul. Ideea este de a împărți o prelucrare în mai multe procese care se derulează unul după altul și se execută fiecare pe un procesor aparte. Fiecare proces depinde de rezultatul procesului precedent din secvență și furnizează rezultate procesului următor.

Dacă este necesară efectuarea mai multor prelucrări, atunci, derularea lor poate avea loc în paralel: în timp ce procesorul i execută un proces, procesorul $i-1$ execută procesul $i-1$ din prelucrarea ce urmează, iar procesorul $i+1$ execută procesul $i+1$ din prelucrarea precedentă.

Să considerăm calculul următor:

```

for i:=1 to 64 do
  begin
    E[i]:=A[i]+B[i];
    F[i]:=E[i]*C[i];
    G[i]:=F[i]-D[i]
  end

```

executat pe trei procesoare specializate pentru operațiile de adunare, înmulțire și respectiv scădere. Derularea în timp a operațiilor, în condițiile unei prelucrări în linie de asamblare la nivelul instrucțiunilor poate fi cea din figura 1.9.

E[I] :=	1	2	3	4			n		
F[I] :=		1	2	3	4		n-1	n	
G[I] :=			1	2	3		n-2	n-1	n
t	1	2	3	4	5		n	n+1	N+2

Figura 1.9

O clasificare a sistemelor cu prelucrare în linie de asamblare ține cont de nivelul de complexitate al proceselor, care poate fi:

- al operațiilor aritmetice (faze de instrucțiune)
- al instrucțiunilor
- al programelor.

Clasa cea mai generală este cea a sistemelor MIMD. Și aici întâlnim cele două variante de comunicare: prin memorie partajată (sisteme multiprocesoare) și prin transmitere de mesaje (sisteme multicalculatoare).

Un sistem cu **memorie partajată** (figura 1.10) constă din mai multe procesoare care au acces la module de memorie comună (sisteme multiprocesor). Procesoarele pot executa procese diferite, dar care trebuie corelate între ele (sincronizate) pentru a evita interferențele (și deci erorile de prelucrare). Numeroase modele au fost propuse pentru

programarea acestor sisteme. Dintre ele, amintim: primitivele P și V, regiunile critice, monitoarele.

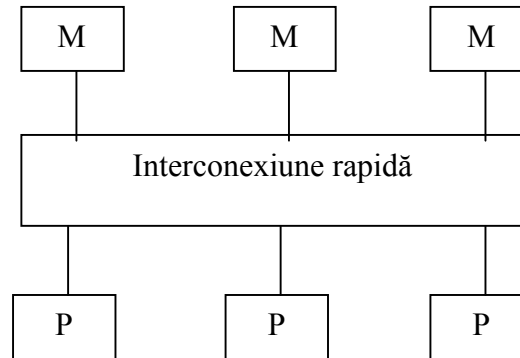


Figura 1.10

Există destul de multe sisteme experimentale care se înscriu în această categorie. Dintre cele comerciale mai recente, amintim **Sun Enterprise 10000**, care este un sistem din categoria UMA (Uniform Memory Access) pentru care procesoarele au același timp de acces la oricare modul de memorie. Configurația maximă a unui sistem include 64 de procesoare UltraSPARC distribuite în 16 grupuri a câte 4 procesoare. Fiecare grup este dispus pe o placă împreună cu 4 module de memorie RAM de câte 1 GB fiecare. Conexiunea între procesoare și blocurile de memorie este asigurată de un *switch crossbar* 16*16, numit *Gigaplane-XB*. Acesta are funcția de a transfera datele între memorie și cache-urile procesoarelor. Transferul se face pe 16 octeți, astfel că sunt necesare 4 cicluri pentru a transfera o linie de cache care are 64 octeți. În afara *switch*-ului, mai există patru magistrale de adrese care sunt folosite pentru *supraveghere* (*snooping*). Fiecare bus este folosit pentru o pătrime din spațiul de adrese fizice, astfel că pentru selecția unei magistrale din patru sunt folosiți doi biți de adresă. Când un procesor nu găsește datele în propria memorie cache el plasează adresa pe magistrala de adrese corespunzătoare pentru a vedea dacă datele sunt în memoria cache a altui procesor. Toate grupurile de procesoare inspectează toate cele patru magistrale. Cea care are datele cerute le furnizează imediat. Dacă nu există răspuns, datele nu există în cache și trebuie aduse din memorie.

Uneori, memoria partajată este **distribuită**, astfel încât există un modul de memorie mai apropiat de fiecare procesor. Accesul la acest modul este mai rapid pentru un procesor și mai lent pentru celelalte. Din acest motiv, modelul se mai numește **NUMA** (Non Uniform Memory Access). Pentru a scurta timpul de acces, unele

sisteme păstrează o parte a datelor în memoriile cache ale procesoarelor, care, evident trebuie păstrate coerente (**ccNUMA** - cache coherent NUMA). Aceste sisteme se mai numesc sisteme cu memorie distribuită, altele sisteme cu memorie locală (figura 1.11). Exemple de sisteme ccNUMA includ *SGI Origin 2000* și *Sequent NUMA-Q 2000*.

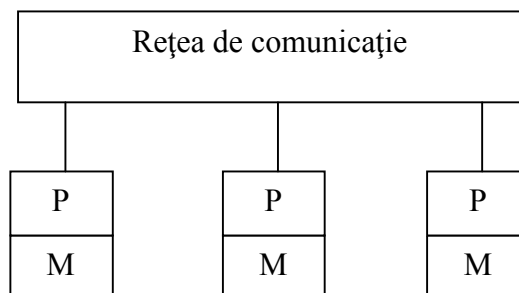


Figura 1.11

Sistemele cu **transmitere de mesaje** (se mai numesc **multicalculatoare**) constau din calculatoare interconectate. Procesoarele execută procese diferite, iar sincronizarea este realizată prin schimburi de mesaje. Diferențele între sisteme se manifestă prin:

- comunicare sincronă sau asincronă de mesaje
- schema de comunicare
 - directă de la proces la proces
 - prin cutii poștale globale
 - prin canale de legătură între procese

Există două categorii de multicalculatoare. Prima categorie, cunoscută sub denumirea de **MPP**, *Massively Parallel Processors*, include sisteme în care procesoarele (în număr ridicat) sunt interconectate prin rețele de mare viteză, construite special ca soluții “de firmă”, pentru a realiza performanțe foarte ridicate. Exemple de sisteme din această categorie sunt: *Cray T3E* și *IBM SP-2*.

A doua categorie include sisteme construite din PC-uri sau stații de lucru interconectate prin rețele comerciale. Astfel de sisteme se numesc **NOW** (*Networks Of Workstations*) sau **COW** (*Clusters Of Workstations*).

Un exemplu de aplicație pentru multi-calculatoare este jocul de șah, în care se generează mutările posibile, a căror mulțime poate fi organizată arborescent: rădăcina este configurația curentă; fiii săi corespund configurațiilor după prima mutare ș.a.m.d.

Găsirea mai rapidă a celei mai bune mutări se poate face distribuind unor procesoare distincte căutarea în sub-arbori diferiți.

Comunicarea între procesoare este necesară pentru transmiterea celei mai bune soluții găsite până la un anumit moment și pentru transmiterea unor sub-arbori în vederea continuării inspectării. La un moment dat, două procesoare distincte pot executa secvențe diferite de instrucțiuni, ceea ce este în concordanță cu caracteristicile sistemelor MIMD.

Capitolul 2. Limbajul de descriere a algoritmilor

În accepțiunea noastră un **program paralel / distribuit** este o colecție de procese secvențiale comunicante. Exisă mai multe categorii de programe concurente, delimitate în funcție de raportul dintre procese și procesoarele pe care acestea se execută. În **programele paralele**, procesele sunt executate de procesoare distincte care comunică prin **variabile partajate** aflate în memoria comună. Uneori, organizarea unui program ca o colecție de procese este utilă chiar în cazul în care acestea se execută pe un singur procesor, intercalat. Fiecare proces poate fi planificat și executat independent de celelalte, existând practic mai multe fire de execuție (threads) în același **program multithread**. Noțiunea de program multithread se extinde și la situația în care numărul de procese dintr-un program este mai mare ca numărul de procesoare pe care acestea se pot executa. În **programele distribuite**, procesele se execută pe mașini interconectate printr-o rețea de calculatoare și, ca urmare, comunică prin schimb de mesaje.

Deși există multe posibilități de a structura colecția de procese dintr-un program concurent, s-au impus câteva **tipare**: paralelismul iterativ, paralelismul recursiv, producători și consumatori, clienți și servere, procese egale (interacting peers). Ne vom referi la acestea pe parcursul prezentării limbajului de descriere a algoritmilor și a unor exemple semnificative.

În mod natural, descrierea adoptată pentru programele paralele se încadrează în modelul CSP (Communicating Sequential Processes) propus de Hoare. Ea se bazează pe construcțiile clasice ale unui limbaj de programare secvențială, extinse cu notațiile ce permit concurența, comunicarea și sincronizarea.

Prezentăm la început notațiile (pseudocod) corespunzătoare descrierii algoritmilor secvențiali. În secțiunile următoare vom introduce construcții specifice programării paralele (bazate pe variabile partajate și comunicare de mesaje).

2.1. Declarații

Tipuri de bază sunt considerate următoarele:

boolean -	bool
întreg -	int
real -	real / double
caracter -	char
șir -	string [lung-max]

Declarația variabilelor are forma generală

tip id = expresie;

care specifica tipul, numele și valoarea inițială a variabilei (inițializările sunt opționale). Exemplu: `int i, j=1;`

Definițiile de constante au forma generală

const tip id = expresie;

Exemplu: `const int c=1;`

Tablourile se declară prin specificarea tipului de bază, a numelui, a tipului indicilor (domeniul) și a valorii inițiale

tip id [domeniu] = expresie;

Exemple:

`int a[5];` sau `int a[1:5];`
`int b[1:10];`
`int c[1:5] = ([5] 0);`
`double c[15,15] = ([15] ([15] 1.0));`

Forma “`int a[1:5];`” este utilă pentru a specifica valorile limită ale domeniului, putându-se face ușor diferența între `a[1:5]` și `a[0:4]`, acolo unde este cazul.

Constructorul de tablou “`([5] 0)`” permite specificarea valorii inițiale a tabloului “`c[1:5]`” ale cărui elemente au toate valoarea inițială 0.

Tipuri definite de utilizator

Definirea tipul enumerat are forma generală

```
enum id {enumerare_simboluri};  
Exemplu: enum zile {l, ma, mi, j, v, s, d};
```

O **înregistrare** se definește ca o colecție de câmpuri cu tipuri potențial diferite

```
typedef struct {definitie_campuri} id;  
Exemplu:  
typedef struct {  
int front=1;  
int rear=1;  
int size=0;  
int contents[5];} queue;  
queue q1, q2[5];
```

În exemplu, este definit tipul “queue”, folosit apoi pentru declararea unei variabile simple și a unui tablou cu elemente de tip “queue”.

2.2. Instrucțiuni

Atribuirea **x=e** evaluează expresia **e** și asociază rezultatul variabilei **x**. Aceasta poate fi un scalar, un tablou, o înregistrare, un element de tablou, un câmp de înregistrare.

Interschimbarea **x1:=x2** interschimbă valorile celor două variabile.

skip este instrucțiune inoperantă. Execuția ei se termină imediat și nu are nici un efect asupra variabilelor programului.

Ieșirea dintr-o instrucțiune compusă se face prin **exit**;

Decizia are una din formele generale următoare:

```
if (conditie) instructiune;  
if (conditie) {bloc instructiuni};  
if (conditie) {bloc instructiuni}  
else if (conditie) {bloc instructiuni}  
else (conditie) {bloc instructiuni};
```

Exemplu:
if (x>y) m = x;

```
else if (y>x) m = y;  
else m = x+y;
```

Iterația (while) are forma generală
while (conditie) {bloc instructiuni};

Ciclul cu contor (fa) are forma generală
for [cuantificatori] {bloc instructiuni}

Fiecare cuantificator specifică un domeniu de valori pentru o variabilă contor (de tip implicit int):

variabila = val_init to val_finala st B

Corpul ciclului este executat pentru acele valori ale variabilei contor pentru care valoarea expresiei booleene B este *true* (such that B).

În cazul mai multor cuantificatori, aceștia se separă prin virgule. Corpul ciclului se execută pentru fiecare combinație de valori ale variabilelor contor, cu variabila cea mai din dreapta variind cel mai rapid.

Exemple:

1) transpunerea matricei m:

```
for [i=1 to n, j=i+1 to n] m[i,j]:=m[j,i];
```

2) ordonarea crescătoare a unui vector:

```
for [i=1 to n, j=i+1 to n st a[i]>a[j]] a[i]:=a[j];
```

2.3. Proceduri

O procedură definește un tipar parametrizat al unei operații. Forma sa generală este următoarea:

```
tip_rez p_id(tip_1 f1, ... ,tip_n fn){  
  declarații  
  instrucțiuni – pot contine return  
}
```

Definiția include:

tip_rez tipul rezultatului întors de procedură
p_id numele procedurii
fi sunt parametrii formali cu tipurile **tip_i**

O procedură fără **returns** este apelată explicit prin instrucțiunea **call**:

call p_id(e₁,...,e_n)

În timp ce o funcție (cu **returns**) apare ca operând într-o expresie:

x = p(e₁,...,e_n).

În ambele cazuri, e_i sunt expresii, avînd același tip cu parametrii formali corespunzători.

Exemplu: calculul factorialului.

```
int fact(int i){  
  if (i<0) return -1;  
  else if (i==0 OR i==1) return 1;  
  else {return i*fact(i-1);}  
}
```

2.4. Concurență și sincronizare

În accepțiunea noastră, un program paralel descrie două sau mai multe procese cooperante. Ca urmare, dacă la execuția unui program secvențial există un singur fir al controlului (thread of control), la cea a unui program paralel există mai multe fire, câte unul pentru fiecare proces.

Fie S_i un program secvențial (declarații + instrucțiuni). Construcția:

co S1 || S2 || ... || Sn oc

specifică execuția paralelă a programelor S1 ... Sn, efectul fiind o anumită întrețesere a acțiunilor lor atomice. Execuția lui **co** se termină atunci cînd se termină execuția fiecărui S_i.

Exemplu:

```
x=0; y=0;  
co x = x+1; || y=y+1; oc  
z=x+y;
```

Variabilele declarate înainte de **co** sunt comune proceselor din **co**. Procesele pot declara și variabile locale, domeniul unei astfel de variabile fiind procesul în care

apare declarația.

Adesea, un program paralel conține procese care execută același calcul asupra unor elemente de tablou diferite. Descrierea se poate face folosind cuantificatori la fel ca pentru instrucțiunea **for**.

co [cuantificator]{Sj}

Exemplu:

```
co [j=1 to n] {a[j]=0; b[j]=0;}
```

Fiecare proces are o copie proprie a lui j , care se comportă în cadrul procesului ca o constantă. Construcția precedentă descrie n procese distincte.

Exemplu: înmulțirea a două matrice:

```
real a[1:n,1:n], b[1:n,1:n], c[1:n,1:n];
co [i = 1 to n, j= 1 to n] {
    real sum = 0;
    for [k = 1 to n]
        sum=sum+a[i,k]*b[k,j];
    c[i,j] = sum;
}
```

Aici sunt n^2 procese, fiecare având constantele locale i și j , precum și variabilele locale sum și k . În cazul unor procese cu descrieri lungi, se folosește și următoarea formă, în care se asociază nume proceselor paralele conținute de un program.

Exemplu

```
real a[1:n,1:n], b[1:n,1:n], c[1:n,1:n];

process Prod (i = 1 to n, j= 1 to n) {
    real sum = 0;
    for [k = 1 to n]
        sum=sum+a[i,k]*b[k,j];
    c[i,j] = sum;
}
```

2.5. Atomicitatea și sincronizarea

Așa cum s-a precizat, execuția unui program paralel poate fi privită ca o întrețesere a acțiunilor atomice ale proceselor. Când procesele interacționează, nu toate

combinațiile sunt acceptabile. De exemplu, să considerăm următorul program și să luăm ca acțiuni atomice citirea și scrierea unei variabile:

```
int y= 0 ; z= 0 ;  
co x=y+z || y=1 ; z=2 oc
```

Dacă adunarea este realizată prin încărcarea valorii lui y într-un registru și adăugarea ulterioară a valorii lui z, valoarea lui x poate fi 0, 1, 2, sau 3, în funcție de vitezele celor două procese. La prima vedere, evaluarea unei expresii nu poate fi considerată acțiune atomică în context paralel, decât dacă variabilele din expresie nu sunt alterate de alte procese.

În realitate, atomicitatea este asigurată de o condiție mai slabă, numită proprietatea **cel-mult-o-dată** (at-most-once), care se formulează astfel:

O **expresie** *e* satisface proprietatea de *cel-mult-o-dată* dacă ea se referă cel mult o dată la cel mult o variabilă simplă (element de tablou sau câmp de înregistrare) care ar putea fi schimbată de un alt proces în timpul evaluării lui *e*. O **instrucțiune** de atribuire *x = e* satisface proprietatea de cel-mult-o-dată fie dacă *e* satisface această proprietate și *x* nu este citită de un alt proces (adică *x* este o variabilă locală), ori dacă *x* este o variabilă simplă și *e* nu se referă la nici o variabilă care ar putea fi schimbată de un alt proces.

De exemplu, nici o atribuire din construcția:

```
co x = y+1 || y = x+1 oc
```

nu satisface proprietatea menționată. De asemenea, atribuirea *x = y+z* din exemplul dat la începutul secțiunii nu satisface această proprietate, ea referind două variabile modificate de un alt proces.

În cazurile când proprietatea anterioară nu este satisfăcută, sau dacă dorim gruparea mai multor acțiuni într-o singură acțiune atomică de granularitate mai mare, este necesar un mecanism de sincronizare.

Pentru a specifica acțiuni atomice folosim paranteze unghiulare. De exemplu, *<e>* înseamnă că expresia *e* trebuie evaluată atomic. Pentru sincronizare, folosim construcția:

```
<await (B) S;>
```

unde *B* este o expresie booleană, iar *S* este o succesiune de instrucțiuni (secvențiale) care se termină cu siguranță.

Semnificația este următoarea: se așteaptă îndeplinirea condiției B, după care se execută S. Execuția este atomică, deci B este cu siguranță *true* la începerea execuției lui S și nici o stare internă a lui S nu este vizibilă din afară.

Exemplu:

`<await (s>0) s=s-1>`

întârzie procesul executat pînă cînd s devine pozitiv, apoi decrementează pe s.

Forma generală a instrucțiunii **await** specifică atît excluderea mutuală, cît și sincronizarea condiționată. Ea admite formele :

<code><s;></code>	pentru a exprima doar excluderea mutuală și
<code><await (B)></code>	pentru a exprima doar sincronizarea condiționată.

Exemplu: excludere mutuală la folosirea unei resurse comune.

Procesele P[i] al unei colecții de procese P[i=1 to n] folosesc o aceeași resursă comună (de exemplu, un același fișier sau un același dispozitiv de imprimare). Deoarece accesul simultan la resursa comună ar putea produce erori de funcționare, în orice moment resursa trebuie folosită de cel mult un proces. Operațiile de folosire a resursei comune constituie o **secțiune critică** a proceselor. Pentru simplitate considerăm că fiecare proces include o **secțiune critică** în care are acces exclusiv la resursa partajată, urmată de o **secțiune necritică** în care folosește doar resurse care îi sunt locale.

Problema este cum controlăm “intrarea” proceselor în **secțiunea critică** astfel încât să se asigure **excluderea mutuală** între procese, la utilizarea resursei comune. Pentru asta se folosește o variabilă **s** care are valoarea 1 dacă resursa comună este liberă și 0 dacă resursa este folosită de un proces. Descrierea soluției este următoarea:

```

int s = 1;                                /*s=1 – resursa liberă*/
                                           /*s=0 – resursa folosita*/

process P[i=1 to n]{
    while (true){
        <await (s>0) s = s-1;>;              /*acaparare resursa*/
        Sectiune critica;
        <s = s+1>;                          /*eliberare resursa*/
        Sectiune necritica
    }
}
```

Pentru intrarea în secțiunea critică, procesul $P[i]$ execută operația **atomică**

<await ($s > 0$) $s = s - 1$; >;

Atomicitatea asigură că operația este executată în întregime, de un singur proces:

- dacă $s == 0$ procesul este pus în așteptare;
- dacă $s > 0$, evaluarea condiției și modificarea lui s sunt executate ca o singură operație (execuția nu poate fi întreruptă de nici un alt proces);
- dacă mai multe procese încearcă să execute operația, doar unul primește permisiunea de execuție, celelalte rămânând în așteptare.

La ieșirea din secțiunea critică procesul $P[i]$ execută operația atomică

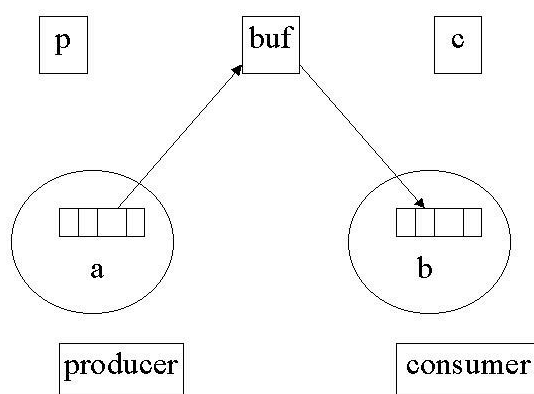
< $s = s + 1$; >;

prin care valoarea lui s devine 1; eventual, un alt proces va putea executa operația de acaparare a resursei comune.

Exemplu. Problema producător-consumator.

Se consideră un producător și un consumator care comunică printr-un singur tampon partajat. Producătorul generează și pune câte o valoare în tampon, iar consumatorul preia câte o valoare din tampon. Presupunem că tamponul poate păstra o singură valoare.

În soluția prezentată aici, considerăm că producătorul păstrează într-un tablou $a[1:n]$ valorile furnizate; consumatorul pune în tabloul $b[1:n]$ valorile preluate din tampon. Soluția trebuie să asigure că valorile ajung în tabloul b în același număr și în aceeași ordine ca în a .



Soluția are următoarea descriere:

```
int buf, p=0, c=0;
/*p = numarul de valori transmise in buf*/
/*c = numarul de valori preluate din buf*/

process producer{
    int a [1:n];
    while (p<n){
        <await (p==c);>
        buf = a[p+1];
        p = p+1
    }
}

process consumer{
    int b [1:n];
    while (c<n){
        <await (p>c);>
        b[c+1] = buf;
        c = c+1
    }
}
```

Observații: acțiunile atomice se rezumă la testarea condițiilor, fără a cuprinde și celelalte acțiuni asupra variabilelor comune. Semnificațiile celor două condiții sunt:

- $p=c$ atunci când tamponul **buf** este “gol”; condiția este îndeplinită la început ($p=c=0$) sau după ce consumatorul preia o valoare din tampon;
- $p>c$ după ce producătorul pune o valoare în **buf**.

Sugerăm cititorului să încerce găsirea unei justificări neformale a corectitudinii soluției.

Problemă. Elaborați o soluție pentru comunicare printr-un tampon limitat, $\text{buf}[1:k]$, cu $1 < k < n$.

Sincronizarea prin **await**, deși generală, este criticată din motive legate atât de claritatea descrierilor rezultate, cât și de eficiența posibilelor implementări:

- conduce uneori la soluții complicate, greu de înțeles și de demonstrat;
- nu face o separare clară între variabilele utilizate pentru sincronizare și cele folosite pentru rezultatele calculelor;

- tehnica de implementare a acestei sincronizări (numită *busy waiting* - așteptare activă) este inefficientă, fiecare proces testînd continuu condiția din **await** pînă cînd aceasta devine adevărată.

Nu este, deci, de mirare că s-au căutat mecanisme de sincronizare care să înlăture neajunsurile menționate (semafoare, regiuni critice, monitoare, bariere, comunicare de mesaje). Dintre acestea, ne referim în continuare la *bariere de sincronizare* și la *comunicarea prin mesaje*, des utilizate în exprimarea algoritmilor paraleli.

2.6. Bariere de sincronizare

Barierele reprezintă un mecanism de sincronizare asociat unei clase de paralelism numită **paralelism de date**. Corespondentul arhitectural al acesteia îl constituie modelul SIMD.

Un algoritm pentru paralelism de date este un algoritm **iterativ** care prelucrează în paralel și în mod repetat elementele unui tablou. Acest gen de algoritmi este în mod natural asociat modelului SIMD. El este însă util și în cazul multiprocesoarelor asincrone.

Un atribut important al algoritmilor iterativi paraleli este că fiecare iterație depinde (în general) de rezultatul iterației precedente. O variantă este să se organizeze fiecare iterație ca o colecție de procese paralele:

```
while (true){  
  co [k = 1 to n] {cod_proces-k;}  
}
```

Soluția este inefficientă deoarece creează și distruge procese la fiecare iterație. O a doua soluție, mai eficientă, creează procesele o singură dată, făcînd sincronizarea lor la trecerea de la o iterație la alta:

```
  co[k= 1 to n]{  
    while (true){  
      cod_proces_k;  
      barrier;  
    }  
}
```

Acest tip de sincronizare se numește **barieră**.

Pentru a da algoritmilor o formă generală, utilizăm o notație explicită pentru marcarea barierelor, și anume **barrier**. Menționăm însă că sistemele SIMD asigură prin construcție sincronizarea și elimină necesitatea programării explicite a barierelor. În cele ce urmează, prezentăm soluțiile unor probleme tipice din categoria paralelismului de date.

Exemplu: calcul grila

În multe aplicații, ca prelucrarea imaginilor sau rezolvarea ecuațiilor diferențiale, se utilizează operații asupra unor structuri matriceale sau de graf, numite calcule grilă sau calcule plasă. În grafică, ideea este de a utiliza o matrice care corespunde valorilor pixelilor. În rezolvarea unor ecuații diferențiale, elementele de la exteriorul matricei sunt inițializate cu valorile de graniță, scopul fiind găsirea valorilor elementelor din mijloc, ceea ce corespunde găsirii unei soluții stabile a ecuației.

În ambele cazuri, tiparul prelucrării este următorul:

```
inițializează matricea
while (neterminat){
    calculează o nouă valoare pentru fiecare punct
    evaluează terminarea
}
```

La fiecare iterație, noile valori ale punctelor sunt calculate în paralel.

Pentru exemplificare, considerăm o soluție a ecuației lui Laplace ($\Delta^2(\Phi) = 0$) în două dimensiuni. Fie grila[0:n+1, 0:n+1] o matrice de puncte. Putem utiliza în calcul o metodă cu diferențe finite (de exemplu, Jacobi). La fiecare iterație, calculăm o valoare a fiecărui punct interior ca media aritmetică a celor patru vecini cei mai apropiați. Metoda este staționară, astfel că putem termina iterațiile atunci când noile valori pentru punctele interioare diferă cu mai puțin de EPS de valorile anterioare.

```
real grila[0:n+1, 0:n+1], noua[0:n+1, 0:n+1];
bool converge = false;
process CalculGrila[i=1 to n, j=1 to n]{
    while (not converge){
        noua[i,j] = (grila[i-1,j]+grila[i+1,j]+
```

```

        grila[i,j-1]+ grila[i,j+1])/4;
    barrier;
    test_convergenta;
    barrier;
    grila[i,j] = noua[i,j];
    barrier;
}
}

```

Matricea **noua** este folosită pentru ca valorile din **grila** să depindă doar de vechile valori. Calculele se termină atunci când toate valorile din **noua** diferă cu mai puțin de EPS de valorile din **grila**. Aceste diferențe pot fi calculate în paralel, dar rezultatele trebuie combinate, ceea ce poate fi făcut utilizând un algoritm prefix paralel.

Problemă (Etichetarea regiunilor)

Data fiind o imagine bidimensională (o grilă de valori de pixeli) să se identifice și să se eticheteze în mod unic toate regiunile. O regiune este un set maximal de pixeli vecini, care au aceeași valoare.

2.7. Paralelismul de date

2.7.1. Operații cu vectori

Considerăm în continuare calculul paralel al sumelor tuturor prefixelor unui tablou linear. Mai precis, dat fiind tabloul $a[1:n]$, calculăm $s[1:n]$, unde $s[i]$ este suma primelor i elemente ale tabloului a .

Soluția secvențială uzuală este următoarea:

$s[1] = a[1];$

for $[i = 2 \text{ to } n]$ $s[i] = a[i] + s[i-1];$

Pentru a obține o paralelizare a acestei soluții, să considerăm mai întâi problema mai simplă a calculului sumei tuturor elementelor unui tablou. În acest scop, am putea însuma la început elementele tabloului două câte două. În etapa următoare, însumăm rezultatele obținute tot două câte două și așa mai departe. Un exemplu, pentru un tablou cu 8 elemente este dat în figura 2.1.

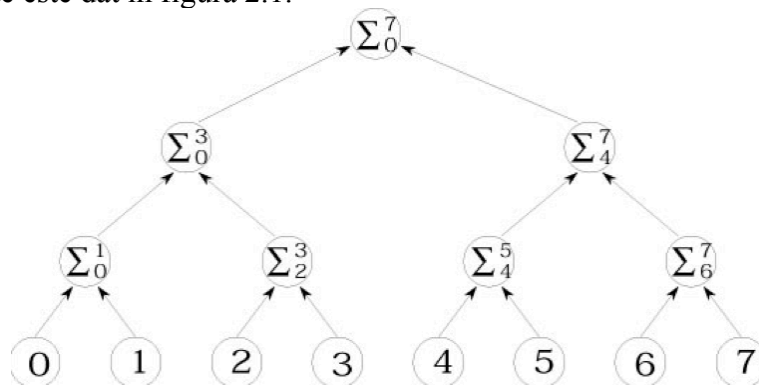


Figura 2.1

Nu este greu de văzut că putem calcula suma elementelor în $\log_2 n$ pași, folosind $n-1$ procesoare.

Pentru sumele prefixelor, adaptăm această soluție, bazată pe ideea dublării, la fiecare pas, a numărului elementelor ce au fost însumate. În acest scop, prezentăm mai întâi o schemă modificată a însumării tuturor elementelor, care permite deducerea mai simplă a algoritmului paralel.

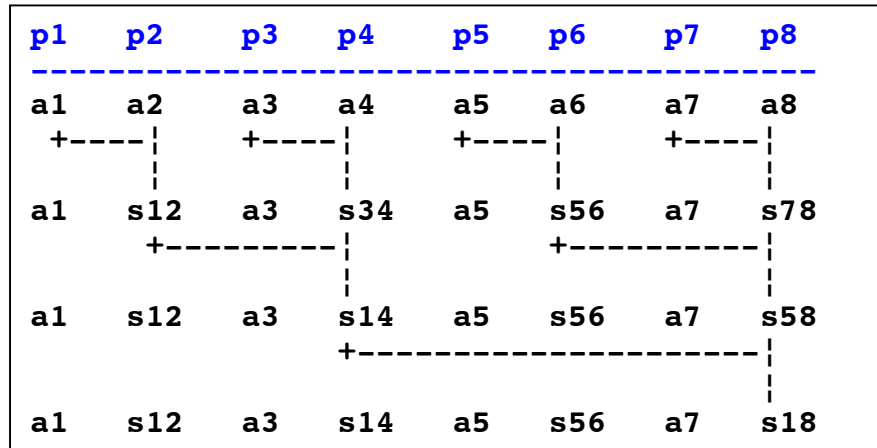


Figura 2.2

```

int a [n];
process suma [k=1 to n]{ // suma ~ procese pi din figura 2.2
    for [j = 1 to sup(log2 n)]{
        if (k mod 2j == 0) a[k] = a[k-2j-1] + a[k];
    }
    barrier;
}

```

Din păcate, această soluție este aplicabilă doar dacă numărul elementelor tabloului este o putere a lui 2. În caz contrar, putem adăuga un număr corespunzător de elemente nule. O altă soluție pleacă de la observația că în algoritmul de însumare, numărul de procesoare folosite efectiv se înjumătățește la fiecare pas, tot mai multe procesoare rămânând libere. Dacă folosim aceste procesoare putem ajunge la o soluție generală, valabilă pentru un număr oarecare de elemente. În acest scop, încercăm să utilizăm fiecare procesor la fel ca celelalte: la fiecare pas j , valorii parțiale $a[k]$ i se adaugă valoarea elementului din poziția $k-2^{j-1}$, dacă aceasta există. Pentru un tablou cu șase elemente, obținem schema din figura 2.3.

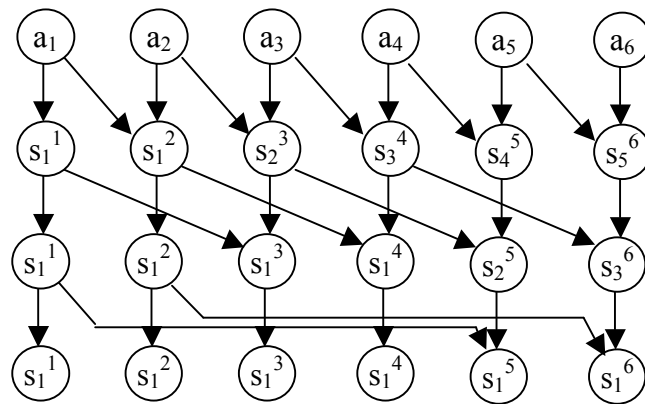


Figura 2.3

Soluția obținută ar avea următoarea descriere (varianta 1):

```

int a [n];
process suma [k=1 to n]{
    for [j = 1 to sup(log2 n)]{
        if (k-2j-1>=1) a[k] = a[k-2j-1] + a[k];
        barrier;
    }
}

```

Deși algoritmul pare satisfăcător, el ascunde o potențială eroare de sincronizare. Să observăm că, la fiecare ciclu, procesele actualizează valorile parțiale a[k]. Pentru un calcul corect, ar trebui ca toate procesele să execute simultan operațiile de adunare și apoi toate să facă atribuirile. Deoarece acest lucru impune introducerea unei noi bariere între adunare și atribuire, cele două acțiuni trebuie despărțite. Acest lucru este posibil prin introducerea unui tablou **temp**, folosit ca depozitar temporar al vechilor valori ale tabloului **a**. Soluția devine următoarea (varianta 2):

```

int a[1:n], temp[1:n];
process suma [k=1 to n]{
    for [j = 1 to sup(log2 n)]{
        temp[k] = a[k];
        barrier;
        if (k-2j-1>=1) a[k] = temp[k-2j-1] + a[k];
        barrier;
    }
}

```

În fine, putem evita operațiile de logaritmare și de ridicare la putere, prin utilizarea unei variabile **d** care păstrează distanța față de poziția elementului **k**, la care se află elementul cu care acesta se combină (varianta 3):

```
int a[1:n], temp[1:n];
process suma[k:1..n]{
    int d = 1;
    while (d<n){
        temp[k] = a[k];
        barrier;
        if (k-d>=1) a[k] = temp[k-d] + a[k];
        barrier;
        d = 2*d;
    }
}
```

Problemă. Ce modificări trebuie aduse algoritmului pentru a face **temp** locală fiecărui proces?

Putem face două **observații** importante:

- în forma finală, algoritmul calculează toate sumele prefix;
- algoritmul poate fi folosit pentru orice operație binară asociativă: înmulțirea, maxim, minim, sau logic, și logic, sau-exclusiv etc.

Descriere algoritmi SIMD

Având în vedere că în algoritmi SIMD toate acțiunile sunt sincronizate (deci orice acțiune ar trebui urmată de o barieră), în descrierea lor eliminăm sincronizările prin bariere. Calculul sumelor prefix poate fi reprezentat astfel (varianta SIMD):

```
int a[1:n];
process suma [k=1 to n]{
    int temp; /*temp este locala procesului k*/
    for [j = 1 to sup(log2 n)]{
        if (k-2j-1>=1){ temp = a[k-2j-1];
                        a[k] = temp + a[k];
        }
    }
}
```

2.7.2. Operații cu liste

Organizarea arborescentă a datelor este mult utilizată pentru motivul că ea conduce la algoritmi de inserare și căutare avînd complexitate $\sup(\log n)$. Folosirea paralelismului de date permite atingerea unei performanțe similare chiar și pentru structuri lineare cum sunt listele. Prezintă în continuare algoritmul de găsire a sfîrșitului unei liste simplu înlanțuite. Metoda poate fi aplicată și altor operații, cum ar fi: sumele parțiale ale valorilor elementelor din listă, inserarea unui element într-o listă de priorități, punerea în corespondență a două liste.

Considerăm o listă de pînă la n elemente, păstrate într-un tablou $\text{data}[1:n]$, legăturile între elemente fiind păstrate într-un tablou distinct, $\text{leg}[1:n]$. Capul listei este păstrat într-un element separat, cap . Dacă un element i face parte din listă, atunci fie $\text{cap} = i$, fie există un alt element j , cuprins între 1 și n , astfel încît $\text{leg}[j] = i$. Legătura ultimului element al listei este 0. De asemenea, $\text{leg}[k] = 0$ dacă elementul corespunzător nu face parte din listă.

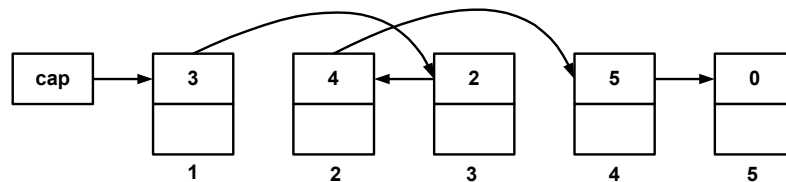


Figura 2.4

Algoritmul secvențial standard pornește din capul listei și parcurge legăturile între elemente, pînă atinge sfîrșitul listei. De aceea, timpul de execuție este proporțional cu numărul de elemente. În varianta paralelă, folosind tehnica "dublării" din algoritmul sumelor prefixelor, putem aduce timpul la $\sup(\log n)$. În acest scop, folosim un tablou $\text{end}[1:n]$, și dedicăm un proces separat, $\text{Afla}(i)$, pentru calculul fiecărui element $\text{end}[i]$, al acestui tablou. Presupunem că lista conține cel puțin două elemente.

Inițial, fiecare proces poziționează $\text{end}[i]$ la valoarea lui $\text{leg}[i]$. Valoarea $\text{end}[i]$ este actualizată în mod repetat. La fiecare ciclu, dacă $\text{end}[i]$ și $\text{end}[\text{end}[i]]$ sunt ambele diferite de 0, atunci $\text{Afla}(i)$ setează $\text{end}[i]$ la valoarea $\text{end}[\text{end}[i]]$. Astfel, după un ciclu, legăturile din end indică elemente aflate la două legături distanță față de elementele curente, după două cicluri pe cele aflate la patru legături distanță etc. După $\sup(\log n)$ pași, toate elementele vor indica sfîrșitul listei. Algoritmul este următorul:

```

int leg[1:n], end[1:n];
process Afla [i=1 to n){
    int nou, d=1;
    end[i] = leg[i];
    barrier;
    while (d<n){
        nou = 0;           //tampon pentru legatura urmatoare
        if (end[i]<>0 and end[end[i]]<>0) nou:=end[end[i]];
        barrier;
        if (nou<>0) end[i]:=nou;
        barrier;
        d = 2*d
    }
}

```

În varianta SIMD dispar barierele, iar ciclul și decizia se contopesc.

```

int leg[1:n], end[1:n];
process Afla [i=1 to n){
    end[i] = leg[i];
    while (end[i]<>0 and end[end[i]]<>0)
        end[i] = end[end[i]];
}

```

Observație. Algoritmul se poate aplica și pentru găsirea rădăcinilor unei păduri de arbori dirijați.

Variantă. Considerăm că pentru fiecare nod i , $\text{leg}[i]$ indică părintele nodului, într-un arbore al pădurii. De asemenea, dacă r este rădăcina atunci $\text{leg}[r] = r$. În aceste condiții, algoritmul devine:

```

int leg[1:n], end[1:n];
process Afla [i=1 to n){
    end[i] = leg[i];
    while (end[i]<> end[end[i]])
        end[i] = end[end[i]];
}

```

Combinație sume prefix - pointer jumping.

Considerăm că fiecărui nod i , i se asociază o pondere $w[i]$. Putem calcula, pentru fiecare i , suma ponderilor nodurilor aflate pe calea de la nodul i la rădăcina arborelui din care face parte. Se dă condiția $w[r] = 0$ pentru orice rădăcină r .

```
int leg[1:n], end[1:n], w[1:n];  
process Afla [ $i=1$  to  $n$ ){  
    end[i] = leg[i];  
    while (end[i] <> end[end[i]]){  
        w[i] = w[i] + w[end[i]];  
        end[i] = end[end[i]];  
    }  
}
```

Probleme

- Calculul sumelor parțiale ale elementelor unei liste înlănțuite.
- Punerea în corespondență a elementelor a două liste. Algoritmul asociază fiecărui element al unei liste un pointer către "prietenul" său din cealaltă listă. Complexitatea cerută este logaritmică.
- Algoritmul de calcul al sfârșitului unei liste simplu înlănțuite, în varianta în care lista poate avea mai puțin de două elemente.

Capitolul 3 - Complexitatea algoritmilor paraleli și distribuiți

Orice proces de proiectare a unui algoritm urmărește obținerea unui rezultat corect și performant. În cazul algoritmilor paraleli și distribuiți, performanța nu poate fi caracterizată printr-o singură cifră. Ea depinde de categoria de aplicații în care este folosit algoritmul, de multe ori performanța fiind o funcție de caracteristicile produsului program realizat (timp de execuție, memorie ocupată, număr de procesoare utilizate, scalabilitate, eficiență, fiabilitate, toleranță la defectări, portabilitate), dar și de costurile implicate de diferitele faze ale ciclului de viață (proiectare, implementare și întreținere). Ca în orice situație în care există mai multe alternative, și în cazul algoritmilor paraleli alegerea trebuie făcută pe baza înțelegerii costului diferitelor variante. Estimarea costurilor reclamă folosirea unor metrice și a unor modele de performanță. Ele permit evaluarea comparativă a soluțiilor, identificarea zonelor de gătuire, eliminarea ineficiențelor, încă înainte de a investi efort în implementarea lor. Deoarece și etapa de evaluare trebuie să fie eficientă, modelele alese pentru a fundamenta alegerea soluției trebuie să fie un compromis între efortul de modelare și importanța îmbunătățirilor realizate prin consumarea lui.

Recunoscând importanța tuturor factorilor de performanță, descrierea care urmează se limitează la o parte a lor, concentrându-se în special pe analiza timpului de execuție și a scalabilității. Alegerea este determinată de faptul că, de cele mai multe ori, acestea reprezintă aspectele cele mai problematice ale proiectării aplicațiilor paralele și distribuite. În analizele concrete pe care un proiectant le face, evaluarea acestor performanțe trebuie realizată în condițiile înțelegerii contextului dat de ceilalți factori menționați mai sus.

3.1. Sortarea pe un vector de procesoare

Pentru a discuta măsurile de performanță asociate algoritmilor paraleli, pornim de la un exemplu simplu: sortarea unui șir de numere folosind un sistem vectorial (un tablou unidimensional de procesoare), ca și cel prezentat în figura 3.1.

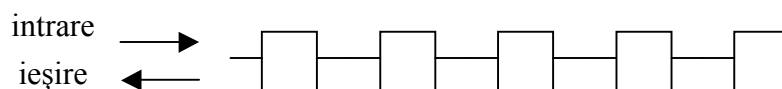


Figura 3.1

Fiecare procesor interior este conectat prin legături bidirecționale cu vecinii săi, din dreapta și din stânga. Procesoarele extreme pot avea doar câte o legătură și pot servi ca puncte de intrare/iesire pentru celelalte procesoare.

Fiecare procesor are o memorie locală și propriul său program. Sistemul are o funcționare sincronă: un ceas global comandă execuția simultană a operațiilor de către procesoare. La fiecare pas, fiecare procesor realizează operațiile următoare:

- recepția unor date de la vecini;
- inspecția memoriei locale;
- execuția calculelor specificate de program;
- modificarea memoriei locale;
- transmiterea unor date către vecini.

O astfel de funcționare se numește **sistolică**.

Pentru a sorta o listă de N numere, folosim un vector de N procesoare. Fiecare procesor execută un algoritm compus din doua faze. În fiecare pas al **primei faze**, fiecare procesor realizează următoarele operații:

- 1) acceptă o valoare de la vecinul din stânga;
- 2) compară valoarea cu cea memorată local;
- 3) transmite valoarea mai mare vecinului din dreapta;
- 4) memorează local valoarea mai mică.

Un exemplu este ilustrat în figura 3.2.

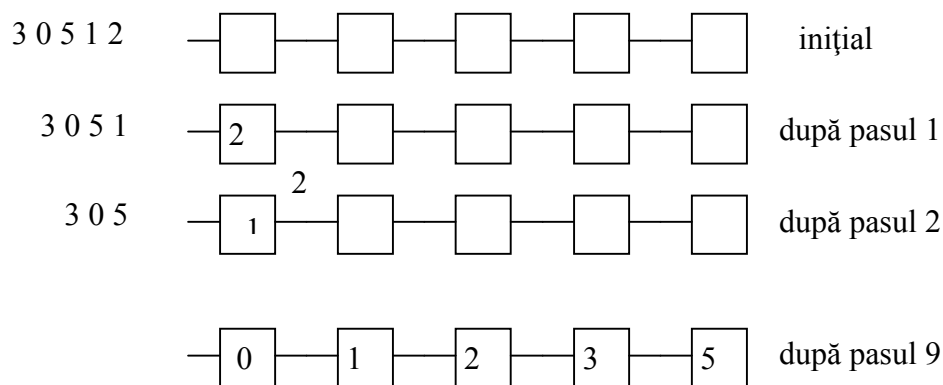


Figura 3.2

Nu este greu de arătat că după $2N-1$ pași, celulele conțin elementele șirului în ordine crescătoare. Astfel, prima celulă examinează N elemente din șir, reține valoarea cea

mai mică și transmite restul spre dreapta. A doua celulă, examinează restul de $N-1$ valori, o reține pe următoarea mai mică și transmite restul celorlalte celule. Prin inducție, rezultă că celula N primește valoarea maximă a șirului. Similar, deoarece o valoare ajunge în prima celulă după un pas, în celula a doua după trei pași ș.a.m.d., rezultă prin inducție că valoarea maximă ajunge în ultima celulă după $2N-1$ pași.

În **a doua fază** a algoritmului, valorile ordonate sunt scoase, prin celula din stânga. Există mai multe variante:

1. fiecare procesor începe să transmită spre stânga, imediat ce numerele sunt sortate;
2. cunoscând poziția sa din vector și numărând valorile inspectate, fiecare procesor calculează momentul când începe să transmită spre stânga;
3. procesorul din dreapta este special; el începe să transmită spre stânga imediat ce primește o valoare; toate celelalte încep să transmită spre stânga imediat ce primesc o valoare dinspre dreapta;
4. fiecare procesor începe să transmită spre stânga imediat ce nu mai primește o valoare dinspre stânga.

Din cele patru metode doar ultimele doua nu impun cunoașterea poziției procesoarelor în vector și contorizarea valorilor. Metoda 3 reclama $4N-3$ pași, în timp ce 4 cere $3N-1$ pași pentru ambele faze de sortare și de scoatere a rezultatelor.

Exercițiu. Alcătuiți descrieri pentru metodele 3 și 4, considerând pe rând modelele de calcul paralel cu memorie comună și cu transmitere de mesaje. În primul caz, comunicarea între procese se realizează prin variabile comune special dedicate.

3.2. Măsuri de performanță

Măsurile obișnuite de apreciere a algoritmilor paraleli sunt:

- **timpul total** T necesar execuției și
- **numarul de procesoare utilizate** P .

În algoritmul prezentat, $P = N$ și $T = O(N)$.

Observație. În secțiunea curentă timpii se referă la cazul cel mai defavorabil.

De multe ori, se compară performanțele atinse de un algoritm cu cele mai bune performanțe ale unui algoritm secvențial care rezolvă aceeași problemă. Astfel, dacă G este timpul de execuție al celui mai rapid algoritm secvențial, se definește **accelerarea** S (speedup) corespunzătoare unui algoritm paralel ca raportul dintre G și T , adică $S = G/T$. În cazul algoritmului precedent, $S = O(N \log N) / O(N) = O(\log N)$.

Ideal ar fi ca algoritmul paralel găsit să fie de P ori mai rapid decât cel mai bun algoritm secvențial, sau măcar $S = O(P)$. Un exemplu este căutarea unei valori într-un fișier distribuit, folosind mașini SIMD cu acces concurrent la citire și exclusiv la scriere. Fiecare procesor inspectează un sub-fișier de lungime n/P în timp $O(n/P)$. Timpul necesar unui algoritm secvențial pentru căutarea în întreg fișierul de lungime n , este $O(n)$. Spunem, în acest caz, că accelerația este lineară. Teoretic, nu se poate obține o accelerație mai bună. Astfel, dat fiind un algoritm paralel care se execută în T pași pe P procesoare, putem oricând folosi un singur procesor pe care procesele să se execute secvențial în TP pași. Ca urmare, $G \leq TP$. Semnul $<$ folosit aici subliniază faptul că, dacă serializarea unui algoritm paralel este totdeauna posibilă (simularea lui pe o mașină multiprogramată), în schimb, nu există o procedură efectivă de paralelizare a unui algoritm secvențial (există algoritmi secvențiali care, prin natura lor, nu pot fi paralelizați).

O accelerație liniară este realizabilă destul de rar, deoarece:

- împartirea problemei în P subprobleme, fiecare rezolvabilă în $1/P$ din timpul algoritmului secvențial corespunzător nu este mereu posibilă;

Să considerăm că într-un algoritm procentajul f din totalul calculelor trebuie să se desfășoare secvențial. Obținem:

$$T = f * G + (1-f) * G / P$$

sau

$$S = 1 / (f + (1-f) / P)$$

Relația arată că oricâte procesoare s-ar folosi, accelerația nu poate fi mai mare de $1/f$ (legea lui **Amdahl**). Pe de altă parte, dependența lui S de f este neliniară, o creștere mică a lui f ducând la o scădere importantă a accelerației S .

- topologia sistemului paralel folosit în rezolvarea problemei impune uneori restricții de timp.

Astfel, în adunarea a n numere folosind $n-1$ procesoare dispuse în topologie arborescentă, nu se poate obține un număr de pași sub $\log n$, câți sunt necesari pentru propagarea unei valori de la o frunză (intrarea) la rădăcina arborelui (ieșirea). Oricum, G/P este o margine inferioară pentru timpul de execuție al oricărui algoritm paralel care folosește P procesoare.

O alta măsură de performanță importantă este **costul** C algoritmului. Acesta este definit ca produsul dintre timpul de execuție și numărul de procesoare, adică $C = TP$.

În cazul nostru avem $C = O(N^2)$. Această măsură poate fi folosită în caracterizarea ineficienței datorate nefolosirii complete a procesoarelor, de către un algoritm paralel.

Mai precis, **eficiența** E a unui algoritm paralel este raportul $E = G/C$ dintre timpul de lucru al algoritmului secvențial optim și costul algoritmului paralel. Avem

$$E = G/C = G/(TP) = S/P$$

deci eficiența poate fi exprimată și ca raportul dintre accelerare și numărul de procesoare. În cazul exemplului dat avem $E = O((\log N)/N)$.

3.3. Cazul unui număr redus de procesoare

Majoritatea măsurilor de performanță sunt calculate pentru situația ideală în care numărul de procesoare utilizate egalează numărul de procese. În exemplul dat anterior, numărul de procesoare utilizate este N , programul de sortare cuprinzând N procese. Ce se întâmplă în cazul unui număr mai mic de procesoare?

Există o procedură generală foarte simplă de execuție a unui algoritm proiectat pentru o rețea de P_1 procesoare, pe o rețea de P_2 procesoare, unde $P_2 < P_1$. Metoda constă în simularea a $\sup(P_1/P_2)$ procesoare pe fiecare procesor real. Cea mai simplă variantă este de a plasa pe procesorul i procesele

$$(i-1)*\sup(P_1/P_2)+1, (i-1)*\sup(P_1/P_2)+2, \dots, i*\sup(P_1/P_2).$$

Orice algoritm care poate fi executat în T pași pe P_1 procesoare, poate fi executat în $T*\sup(P_1/P_2)$ pași în cazul a numai P_2 procesoare.

3.4. Calculul detaliat al complexității

De multe ori, paralelizarea calculului se face pentru atingerea unui timp de execuție cât mai redus. În legătură cu acest aspect, apare firească întrebarea dacă un anumit algoritm este optim sau nu, deci dacă există sau nu un algoritm cu un timp mai mic de execuție.

Răspunsul la această întrebare depinde, printre altele, de **modelul de evaluare** adoptat. În analiza făcută anterior pentru algoritmul de sortare, pașii algoritmului au fost numărați în termenii unor operații asupra cuvintelor. O altă posibilitate este de a considera **modelul bit**, mai detaliat. O astfel de analiză este justificată de existența unor mașini la care paralelismul se manifestă chiar la nivelul operațiilor asupra biților

de date, precum și de interesul teoretic pe care astfel de măsuri de complexitate îl prezintă.

Punctul cheie în detalierea operațiilor algoritmului de sortare la nivel de bit îl constituie comparațiile. O metodă simplă este compararea numerelor bit cu bit, secvențial, începând cu bitul cel mai semnificativ. A doua metodă folosește un arbore binar complet. Numerele sunt comparate bit cu bit, informațiile rezultate fiind apoi condensate prin împerecheri succesive, până la obținerea răspunsului final. Procedeu este ilustrat în figura 3.3.

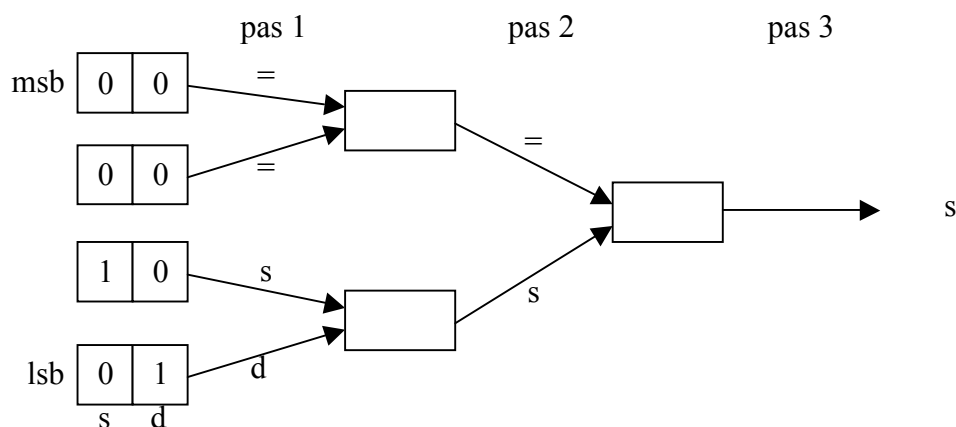


Figura 3.3

(msb – cel mai semnificativ bit; lsb – cel mai puțin semnificativ bit; s- valoarea din stânga; d - valoarea din dreapta)

Arborele binar este superior algoritmului secvențial, din două motive. Mai întâi, el utilizează $\log k + 1$ pași pentru a afla rezultatul, în timp ce primul cere k pași. Apoi, același arbore poate fi folosit pentru a specifica procesoarele care dintre cele două numere trebuie trecut mai departe și care trebuie păstrat (așa cum reiese din figura 3.4).

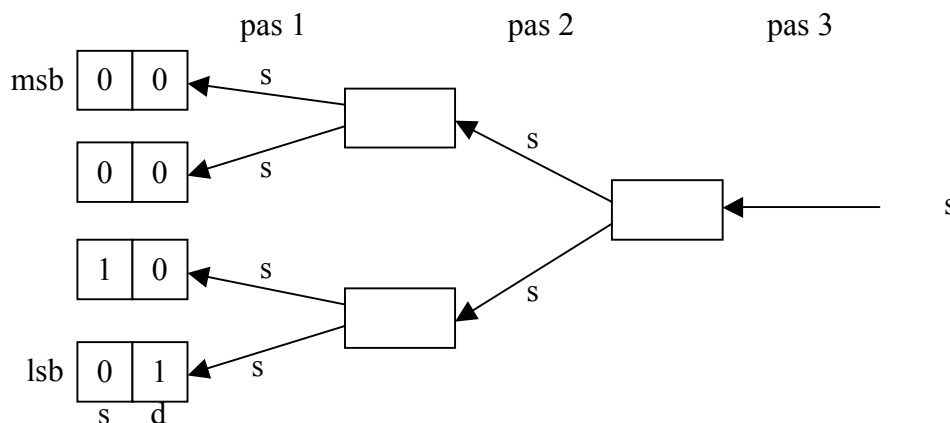


Figura 3.4

Cu aceasta, după $2 \cdot \log k$ pași, comparația celor două numere este terminată și cei k biți ai numărului mai mare pot fi mutați simultan spre următoarea celulă. Înlocuind fiecare procesor din vectorul folosit pentru sortare, cu un arbore complet de procesoare de un bit, putem construi o rețea cu $(2k-1)N$ procesoare de un bit, care face $(2N-1) \cdot 2 \cdot \log k$ pași pentru a executa prima fază a algoritmului.

Există însă un algoritm mai bun de comparare, care, în mod surprinzător, se bazează pe un tablou linear de procesoare și nu pe un arbore complet. Secretul constă în utilizarea acestuia în linie de asamblare (pipeline).

Ideea de bază este ilustrată în figura 3.5, în care se consideră un vector de trei procesoare pe bit, folosit pentru a găsi cea mai mică dintre valorile 2, 0, 5, 1, 2 (evident, în reprezentarea lor binară).

Vectorul de procesoare face, în mod repetat, compararea valorii memorate cu valoarea primită din stânga, o reține pe cea mai mică și o transmite în dreapta pe cealaltă.

Soluția are următoarele particularități:

- execuția în linie de asamblare face ca în timp ce un procesor compară o pereche de biți a două numere succesive, procesorul de sub el să lucreze cu biții mai puțin semnificativi ai perechii de numere anterioare, iar procesorul de deasupra să lucreze cu biții mai semnificativi ai perechii de numere succesoare;
- la fiecare pas, un procesor primește la intrare un bit, iar de la procesorul de deasupra o informație asupra comparației făcute de el la pasul anterior (aceasta

comparație se referă la biții mai semnificativi ai aceluiași numere) - **s** dacă numărul de la intrare este mai mare, **d** dacă numărul memorat este mai mare, = dacă cele două numere sunt egale (în rangurile mai mari decât rangul curent);

3 0 5 1 2 <- inițial			

0 0 1 0 0 []	<-msb	0 [0] -> 0 1 0	
/ / / / /		/ = /	
1 0 0 0 1 []		1 0 [0] -> 0 1	
/ / / / /		/ s /	
1 0 1 1 0 []	<-lsb	1 0 1 [1] -> 0	
inițial		după pasul 4	

0 0 1 0 [0]		[0] -> 0 0 1 0	
/ / / /		= / /	
1 0 0 0 1 []		1 [0] -> 0 0 1	
/ / / /		= / /	
1 0 1 1 0 []		1 0 [1] -> 1 0	
după pasul 1		după pasul 5	

0 0 1 [0] -> 0		[0] -> 0 0 1 0	
/ / / =		/ / /	
1 0 0 0 [1]		[0] -> 1 0 0 1	
/ / /		s / / /	
1 0 1 1 0 []		1 [0] -> 1 1 0	
după pasul 2		după pasul 6	

0 0 [0] -> 1 0		[0] -> 0 0 1 0	
/ / s		/ / / /	
1 0 0 [0] -> 1		[0] -> 1 0 0 1	
/ /		/ / / /	
1 0 1 1 [0]		[0] -> 1 1 1 0	
după pasul 3		după pasul 7	

rezultat ->		0 3 1 5 2	

Figura 3.5

- dacă primește **s**, procesorul transmite bitul de la intrare la ieșire și transmite **s** în jos;
- dacă primește **d**, procesorul transmite bitul memorat la ieșire, memorează bitul de la intrare și transmite **d** în jos;
- dacă primește **=**, transmite bitul mai mare la ieșire, memorează bitul mai mic și transmite în jos **s**, **d** sau **=** în mod corespunzător;
- când nu mai primește nimic la intrare, procesorul scoate în stanga valoarea memorată și apoi toate valorile primite din dreapta (faza a doua).

Se poate observa că acțiunea vectorului de procesoare pe un bit este similară celei a primului procesor de cuvânt, din algoritmul de ordonare. În ambele cazuri, cea mai mică valoare este pastrată, restul valorilor fiind transmis spre dreapta. Putem, deci, combina N vectori de câte k biți, formând astfel o matrice de procesoare, pentru a realiza sortarea celor N numere. Prima fază a sortării se poate executa în $(k-1)+(2N-1) = 2N+k-2$ pași, la nivel de bit. A doua fază a sortării se poate derula la fel ca în modelul "cuvânt", ceea ce conduce la sortarea completă a N cuvinte de câte k biți în $3N+k-2$ pași cu $N*k$ procesoare pe un bit.

3.5. Limite inferioare

Exemplul anterior este util sub două aspecte. Mai întâi, el arată rolul **topologiei** rețelelor de procesoare în atingerea unor performanțe bune în calculul paralel. Ne putem imagina cu ușurință că diferitele combinații de acțiuni exemplificate aici pentru operații foarte simple, pot apare în general, în calculul paralel, deci și pentru acțiuni de o complexitate mai mare.

În al doilea rând, el ne permite discutarea unor aspecte legate de **optimalitatea** algoritmilor paraleli. Dacă ne referim la algoritmul prezentat în termenii vitezei sau eficienței, putem spune că el nu este optimal, deoarece există algoritmi mai rapizi care utilizează chiar mai puține procesoare. Rețeaua necesară pentru acești algoritmi este însă mai complexă decât o matrice $k*N$. Dacă ne **limităm la matrice $k*N$** , atunci putem afirma că algoritmul este optim, deoarece orice algoritm de sortare executat pe o astfel de configurație reclamă un timp $O(k+N)$. Motivele sunt următoarele:

1) Dacă sunt folosite numai k procesoare pentru introducerea datelor, sunt necesari N pași numai pentru citirea tuturor celor N numere.

2) **Diametrul** unei rețele este distanța maximă între oricare două noduri ale sale. Distanța între două noduri este numărul minim de legături traversate pentru a ajunge de la un nod la celălalt. Pentru o rețea de $k*N$ noduri, diametrul este $N+k-2$. Diametrul este o indicație bună asupra complexității, deoarece comunicarea informației între nodurile aflate la o distanță d consumă d pași ai algoritmului.

De exemplu, presupunând că la sortare numerele $x_1 = x_110...01$, $x_2=...=x_N= 010...0$ sunt dispuse inițial în matricea de procesoare astfel că bitul i cel mai semnificativ al numărului j este dispus în matrice în poziția i,j , atunci timpul trebuie să fie cel puțin

$k+N-2$. Aceasta deoarece bitul cel mai puțin semnificativ al celui mai mare număr este 1 dacă și numai dacă $x_{11} = 1$, ceea ce înseamnă că informația din poziția 1,1 trebuie să ajungă în poziția k,N pentru ca sortarea să fie terminată.

3) **Lărgimea biseției** unei rețele este numărul minim de legături care trebuie îndepărtate pentru a împărți rețeaua în două subrețele separate, având același număr de noduri (sau diferența de un nod). Această caracteristică este importantă deoarece uneori valorile calculate de o jumătate a rețelei sunt necesare celeilalte jumătăți.

De exemplu, în sortare, este posibil ca numerele mai mari să fie plasate în jumătatea stângă a matricei de procesoare, iar numerele mai mici în dreapta. Ca urmare, $(k*N)/2$ biți trebuie să treacă din stânga în dreapta rețelei și invers. Cu o lărgime a biseției de k legături, sunt necesari cel puțin $N/2$ pași.

Caracteristicile prezentate trebuie folosite cu grijă în calculul complexității. Există exemple care arată că luarea lor în considerație fără discernământ poate conduce la aprecieri eronate. Astfel, în [Leighton92] se menționează cazul sortării a N numere de K biți folosind un arbore binar complet. O configurație posibilă este prezentată în figura 3.6.

Figura arată conținutul procesoarelor frunză, după sortare. Procesorul rădăcina are $\log N$ celule (unde N este numărul valorilor șirului de sortat). Procesoarele din nodurile intermediare ale arborelui sunt de o celulă. În fine, procesoarele frunză au câte K celule (unde K este numărul de biți din reprezentarea unei valori din șirul de sortat).

Diametrul rețelei este $2 \cdot \log N + 2 \cdot K - 2$, iar lărgimea benzii de intrare a datelor este $K \cdot N$ (presupunând că toate numerele sunt introduse deodată). În fine, lărgimea biseției este 1, dacă $\log N = 1$ și 2 în celelalte cazuri.

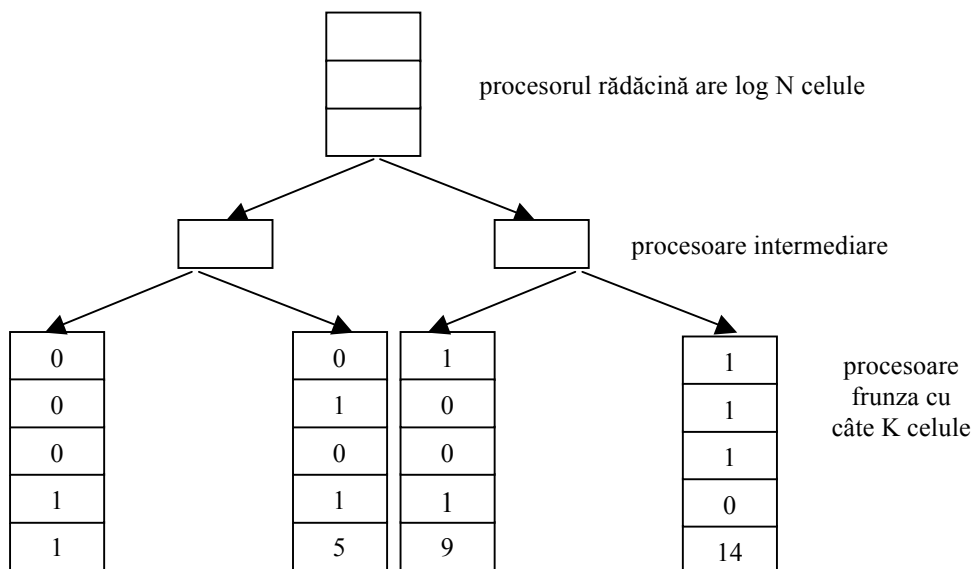


Figura 3.6

Deoarece, în cel mai rău caz, algoritmul trebuie să mute $\mathbf{KN}$ biți prin bisecție, ar trebui ca algoritmul de sortare să solicite $\mathbf{O(KN)}$ pași. Cu toate acestea, pentru situații în care $\mathbf{k}$ este mic, rezultatul este cu totul altul. În particular, pentru $\mathbf{k=1}$, există un algoritm $\mathbf{O(log\ N)}$ care realizează sortarea.

3.6. Proprietăți ale modelului de evaluare

Exemplele de evaluare prezentate au evidențiat modul de considerare a timpului de calcul și de comunicare în analiza algoritmilor paraleli. Sistematizăm în continuare proprietățile pe care se bazează aceste analize.

Proprietățile procesoarelor. Fiecare procesor are un control local. Acțiunile lui depind de conținutul memoriei locale și de intrările proprii. Complexitatea depinde de nivelul operațiilor considerate: pe bit sau pe cuvânt. Uneori **pachetul** este considerat ca unitate de date, operațiile uzuale fiind recepția, memorarea și transmisia unui pachet.

Proprietățile interconexiunilor. Presupunerile care se fac uzual despre rețeaua de procesoare sunt următoarele:

- topologia este fixă;

- gradul fiecărui nod este limitat;
- dimensiunea rețelei este o funcție polinomială (uzual lineară) de dimensiunea problemei.

Proprietățile protocolului de intrare/iesire. Constrângerea uzuală este ca o intrare este prevăzută o singură dată. Complexitatea include astfel și costul difuzării intrării, dacă este cazul. În plus, locul și momentul apariției fiecărei intrări și, corespunzător a fiecărei ieșiri, trebuie specificate în avans (adică înainte de începerea execuției algoritmului).

3.7. Modele generale

Cele prezentate anterior evidențiază faptul că analiza complexității algoritmilor paraleli presupune folosirea unui **model formal**, care să permită evaluarea parametrilor de performanță, cum sunt timpul de execuție, lucrul, accelerarea etc. Modelul folosit în exemplul dat este particular unei anumite categorii de mașini paralele, SIMD cu memorie locală. Din păcate, acesta nu este singurul model folosit în calculul paralel, deoarece organizarea mașinilor paralele diferă mult mai mult între ele decât cea a mașinilor secvențiale. Pe de altă parte, folosirea unui model particular, care să surprindă cât mai fidel caracteristicile fiecărei categorii de mașini este nepractică, datorită numărului mare de modele care ar fi necesare. O alternativă este folosirea unui număr mic de modele abstracte, care nu încearcă să reproducă fidel comportarea vreunei mașini paralele particulare, dar au avantajul că algoritmi dezvoltați pe baza lor pot fi traduși cu ușurință în programe care se dovedesc eficiente pe mașinile paralele reale.

Modelele paralele generalizează modelul secvențial RAM, prin introducerea mai multor procesoare. Ele se clasifică în trei categorii, în care este ușor să plasăm arhitecturile de mașini paralele definite în capitolul anterior:

- mașini cu memorie locală (din care face parte și modelul folosit anterior)
- mașini cu memorie modulară și
- mașini paralele cu acces aleator la memorie PRAM.

O mașină cu memorie locală cuprinde o mulțime de procesoare, fiecare cu propria sa memorie, atașate unei subrețele de interconectare. Fiecare procesor poate accesa direct propria memorie și prin intermediul rețelei de interconectare celelalte memorii. Accesul local se consideră că durează o unitate de timp, accesul distant, la memoria

unui alt procesor depinde de performanțele rețelei de interconectare și de nivelul traficului celorlalte procesoare.

O mașină cu memorie modulară constă din N procesoare și M module de memorie, atașate unei rețele comune. Un procesor accesează un modul de memorie prin transmiterea unei cereri prin rețea. Uzual, timpul de acces al oricărui procesor la oricare modul este uniform, dar depinde de performanțele rețelei și de "tiparul" operațiilor de acces la memorie.

O mașină PRAM constă din N procesoare conectate la o memorie comună, partajată. Se consideră că, într-un singur pas, procesele pot avea acces simultan, direct la memoria partajată.

Desigur, modelele sunt foarte generale și destul de îndepărtate de structura și funcționarea mașinilor reale. De exemplu, nici o mașină reală actuală nu se conformează în mod ideal modelului PRAM, realizând într-o unitate de timp, orice acces la memoria partajată. Cu toate acestea, modelele menționate au fost folosite în dezvoltarea algoritmilor paraleli și distribuiți, datorită simplității lor și a ușurinței de folosire în proiectarea algoritmilor.

3.7.1. Modelul grafurilor orientate aciclice (work-depth)

O alternativă naturală la utilizarea unor modele orientate pe categorii de mașini este folosirea unor modele orientate spre algoritmi. Fără a ține seama de particularitățile sistemelor de calcul paralele și distribuite pe care urmează a fi executați algoritmi dezvoltați, modelele facilitează exprimarea fără constrângeri artificiale a rezolvării problemelor (cum ar fi numărul de procesoare sau topologia sistemului distribuit), exploatând la maximum paralelismul intrinsec al problemelor. O clasă importantă a unor astfel de modele este cea denumită "lucru-adâncime", la care ne referim în continuare.

Multe calcule pot fi reprezentate în forma unor grafuri (circuite) orientate aciclice, după următoarele reguli:

- fiecare intrare este reprezentată ca un nod fără arce de intrare
- fiecare operație este reprezentată ca un nod ale cărui arce de intrare provin de la nodurile care reprezintă operanzii
- o ieșire este reprezentată ca un nod fără arce de ieșire.

Un exemplu este prezentat în figura 3.7, care descrie un algoritm de calcul al sumei elementelor unui tablou unidimensional.

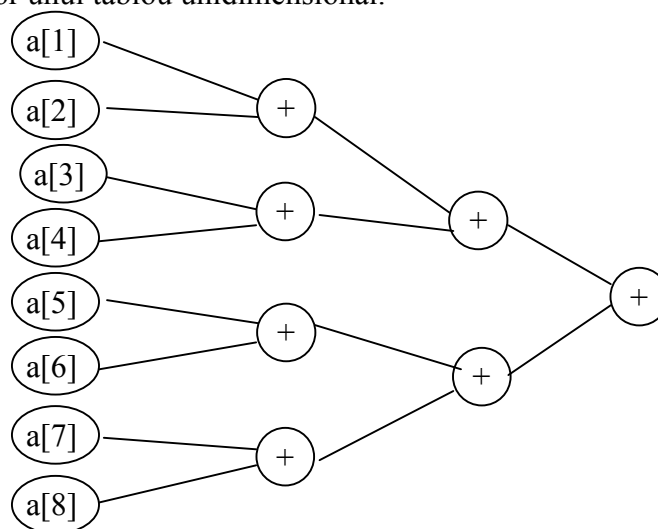


Figura 3.7

Graful descrie operațiile executate de algoritm și constrângerile de precedență asupra ordinii în care operațiile sunt executate. Modelul este independent de orice arhitectură și orice număr concret de procesoare. El pune în evidență:

- **lucrul** efectuat de algoritm, adică numărul total de operații
- **adâncimea**, adică lungimea celui mai lung lanț de dependențe secvențiale din algoritm.

Raportul lucru / adâncime se numește **paralelismul algoritmului**. În exemplul din figura 3.7, lucrul cerut de calculul sumei este de 7 operații. Adâncimea este de trei operații și poate fi calculată ușor pornind de la orice frunză și numărând operațiile pe calea până la rădăcina arborelui.

În acest caz, generalizarea este foarte simplă: pentru însumarea a n numere sunt necesare $n-1$ operații, adâncimea fiind de $\log n$.

3.7.2. Folosirea modelului pentru calculul complexității

Modelul de graf poate fi folosit pentru aprecierea complexității algoritmilor paraleli pornind de la forma acestora într-un limbaj de descriere. Să reluăm problema anterioară, a cărei descriere este următoarea:

```
real a[1:n], b[1:n];           /* presupunem  $n = 2^k$  */  
real s;  
process suma [i = 1 to n]{  
    for [i = 1 to n] b[i] = a[i];  
    for [h = 1 to log n]  
        if (i <= n/2h)  
            b[i] = b[2*i-1] + b[2*i];  
    if (i==1) s = b[1];  
}
```

Descrierea nu conține nici o mențiune despre numărul de procesoare, sau despre felul în care operațiile vor fi alocate procesoarelor. În schimb, el poate fi analizat în număr de unități de timp necesare, în fiecare unitate de timp executându-se în paralel orice număr de operații.

Astfel, numărul total de unități de timp este $\log n + 2$, deci $T(n) = O(\log n)$. În prima unitate de timp sunt executate în paralel n operații. În iterația h se fac $n/2^h$ operații, unde $1 \leq h \leq \log n$. În ultima unitate de timp are loc o singură operație. Rezultă că lucrul făcut de algoritm este $W(n) = n + \sum_{h=1, \log n} n/2^h + 1 = O(n)$.

3.7.3. Principiul de planificare pentru PRAM

Deși numărul modelelor ce ar putea fi utilizate în dezvoltarea algoritmilor paraleli este relativ mare, există, din fericire, deseori tehnici automate de traducere a algoritmilor de la un model la altul. Aceste traduceri conservă lucrul realizat de algoritm, în limitele unui factor constant. Teorema următoare, datorată lui **Brent**, descrie modul de traducere de la modelul **work-depth** la un algoritm PRAM.

Dacă se ia în considerație numărul de procesoare p al unei anumite configurații de sistem de calcul, atunci cel mult p operații pot fi executate în paralel în fiecare unitate de timp. Principiul enunțat în continuare arată că, pentru un algoritm care rulează în $T(n)$ unități de timp, executând $W(n)$ operații, se poate obține o adaptare a algoritmului care să ruleze pe p procesoare PRAM în cel mult $\inf(W(n)/p) + T(n)$ unități de timp (**teorema lui Brent**).

Justificare

Fie $W_i(n)$ numărul de operații executate în unitatea i de timp, unde $1 \leq i \leq T(n)$. Pentru fiecare i , $1 \leq i \leq T(n)$, fiecare set de $W_i(n)$ operații se execută în $W_i(n)/p$ pași paraleli, pe cele p procesoare. În cazul în care execuția este reușită, algoritmul PRAM corespunzător pentru p procesoare ia mai puțin de

$\sum_i \sup(W_i(n)/p) \leq \sum_i (\inf(W_i(n)/p) + 1) \leq \inf(W(n)/p) + T(n)$
pași paraleli.

Exemplificare

Pentru exemplificare, să considerăm că executăm algoritmul de însumare a elementelor unui tablou pe un sistem PRAM cu p procesoare, unde $p = 2^q \leq n = 2^k$. Împărțim tabloul A în p subtablouri, astfel încât procesorul s să fie responsabil de subtabloul s , adică de elementele $A[l*(s-1)+1]$ până la $A[l*s]$. Inițial, procesorul s copiază aceste elemente în $B[l*(s-1)+1]$ până la $B[l*s]$. La fiecare nivel al arborelui binar, elementele B se înjumătățesc față de nivelul anterior, dar se reîmpart în mod egal între cele p procesoare. Numărul de operații posibil concurente la nivelul h este $n/2^h$. Dacă acest număr este mai mare decât numărul de procesoare $p = 2^q$, adică

$$n/2^h = 2^{k-h} \geq p = 2^q$$

ceea ce echivalează cu

$$k-h \geq q$$

atunci operațiile se împart egal procesoarelor. Altfel, operațiile sunt executate de cele 2^{k-h} procesoare de indici inferiori.

```
real A[1:n], B[1:n];          /* n = 2^k si p = 2^q */
real S;
int l = n/p;
process suma [s = 1 to p]{
  for [j = l*(s-1)+1 to l*s] B[j] = A[j];
  for [h = 1 to log n]{
    if (k-h >= q)                /*nr op >= nr proc */
      for [j = 2^{k-h-q}*(s-1) + 1 to 2^{k-h-q}*s]
        B[j] = B[2*j-1] + B[2*j];
    else
      if (s <= 2^{k-h}) B[s] = B[2*s-1] + B[2*s];
  }
  if (s = 1) S = B[1];
}
```

Notă. Algoritmul merge pentru $p \geq n/2$.

Pentru a estima timpul de execuție al algoritmului, observăm că primul pas ia $O(n/p)$ unități de timp, deoarece fiecare procesor execută n/p atribuiri. Iterația h din ciclu ia $O(n/(2^h p))$, deoarece un procesor are de executat cel mult $\sup(n/(2^h p))$ operații. Ultimul pas ia o unitate de timp. Ca urmare, timpul de execuție pentru p procesoare, $T_p(n)$ este inferior lui

$$\begin{aligned} n/p + \sum_{h=1, \log n} \sup(n/(2^h p)) &\leq n/p + \sum_{h=1, \log n} (\inf(n/(2^h p)) + 1) \leq \\ &\leq n/p + \sum_{h=1, \log n} \inf(n/(2^h p)) + \log n \leq 2n/p + \log n \end{aligned}$$

și deci $T_p(n) = O(n/p + \log n)$, ceea ce corespunde principiului planificării.

3.7.4. Lucru și cost

Cele doua noțiuni sunt corelate între ele. Dat fiind un algoritm de lucru $W(n)$ și de timp $T(n)$, algoritmul poate fi simulat pe p procesoare PRAM într-un timp dat de formula:

$$T_p(n) = O(W(n)/p + T(n))$$

Costul corespunzător (produsul timp*numer_procesoare) este atunci egal cu

$$C_p(n) = T_p(n) p = O(W(n) + p T(n))$$

În general, costul este mai mare decât lucrul unui algoritm. Aceasta se datorează utilizării parțiale a procesoarelor pe parcursul execuției unui algoritm paralel. Astfel, pentru algoritmul precedent, considerând că se utilizează n procesoare, cel mult jumătate pot fi active la prima execuție a ciclului **fa** exterior, cel mult un sfert rămân active la a doua execuție a ciclului ș.a.m.d. Abordarea prezentată presupune utilizarea procesoarelor libere pentru alte sarcini.

Există, bineînțeles, anumite **cerințe** pentru realizarea unor algoritmi PRAM eficienți. Astfel, numărul de procesoare p trebuie să fie mai mic decât n (dimensiunea problemei) și adaptiv (dependent de dimensiunea problemei). Uzual, se consideră că $p(n)$ este satisfăcător dacă este o funcție subliniara de n , de exemplu rădăcina pătrată din n . În orice caz, algoritmul să nu depindă de numărul de procesoare.

Timpul de execuție $T_p(n)$, dependent de dimensiunea problemei, să fie mic, semnificativ mai mic decât cel al variantei secvențiale. De asemenea, el să fie adaptiv (față de dimensiunea problemei) și invers proporțional cu numărul de procesoare.

În fine, costul să fie minim. De multe ori se compară costul algoritmului paralel cu costul celui mai bun algoritm secvențial care rezolvă aceeași problemă. Dezvoltarea unor algoritmi cost-optimali va fi prezentată într-o altă secțiune.

3.7.5. Implementare pentru rețele de procesoare

În cazul implementării algoritmilor paraleli pe rețele de procesoare, există mai multe posibilități de alegere a topologiei: liniară, inel, arbore, matrice, cub, hipercub. De regulă, folosirea unor topologii diferite conduce la rezolvări diferite ale aceleiași probleme. Transpunerea soluției pentru un hipercub este deosebit de simplă, datorită capacității acestuia de adaptare la diferite topologii, printre care și aceea de arbore.

Un hipercub constă din $p = 2^d$ procesoare, indexate de la 0 la $p-1$, interconectate într-un cub boolean d -dimensional, definit după cum urmează. Fie reprezentarea binară a lui i , $0 \leq i \leq p-1$, de forma $i_{d-1}i_{d-2} \dots i_0$. Două procesoare sunt conectate dacă indicii lor diferă doar într-o singură poziție a reprezentărilor lor binare. De exemplu, într-un cub 4-dimensional, nodul 0 (0000) este conectat cu nodurile 1 (0001), 2 (0100), 4 (0100) și 8 (1000).

Hipercubul are o structură recursivă. Se poate extinde un cub d -dimensional la unul $(d+1)$ -dimensional, prin dublarea primului și prin realizarea legăturilor între nodurile corespunzătoare din cele două cuburi d -dimensionale. Indexul nodurilor din primul cub se extinde prin adăugarea cifrei semnificative 0, iar indexul nodurilor din cel de al doilea prin adăugarea bitului semnificativ 1.

Hipercubul este popular datorită regularității structurii, diametrului redus (egal cu $\log p$) și datorită abilității de a manevra simplu și rapid multe calcule.

Pentru exemplul pe care îl rezolvăm aici, fiecare element $A[i]$ al tabloului cu n elemente este memorat într-un nod $P[i]$ al hipercubului cu n noduri ($n=2^d$). Algoritmul de calcul al sumei consta din d iterații și memorează rezultatul în $P[0]$. Primul pas constă în însumarea elementelor care se află în noduri ai căror indecși diferă prin cifra cea mai semnificativă. Valorile sunt memorate în cubul $(d-1)$ -dimensional cu nodurile având indecși cu cifra cea mai semnificativă egală cu zero. În cazul în care $n=8$, se realizează următoarele calcule:

$$A[0] = A[0] + A[4]$$

$$A[1] = A[1] + A[5]$$

$$A[2] = A[2] + A[6]$$

$$A[3] = A[3] + A[7]$$

Celelalte iterații continuă după reguli similare. Pentru exemplul ales avem:

$$A[0] = A[0] + A[4] + A[2] + A[6]$$

$$A[1] = A[1] + A[5] + A[3] + A[7]$$

iar în ultimul pas, în $A[0]$ obținem întreaga sumă.

În algoritmul următor, $i(l)$ denotă indexul obținut din i prin complementarea bitului l . Operația $A[i] := A[i] + A[i(l)]$ se face în doi pași: în primul pas, procesorul $P[i]$ copiază valoarea $A[i(l)]$ preluată de la procesorul $P[i(l)]$ pe legătura dintre cele două; în al doilea pas, $P[i]$ face adunarea și memorează suma în $A[i]$.

```

real A [0:n-1];                      /* n = 2d */
process suma [i = 0 to n-1]{
    for [l = d-1 to 0]                 /*ciclul se repeta de d=log n ori*/
        if (i <= 2l - 1)               /*i <= 2l cand l = 2k*/
            A[i] = A[i] + A[i(l)];
}

```

Algoritmul se termină în $O(\log n)$ unități de timp, la fel ca și implementarea PRAM.

Problemă. Descrieți algoritmul de însumare pentru cazul în care numărul de procesoare p este mai mic decât numărul de elemente n și fiecare nod memorează n/p elemente ale tabloului A .

Capitolul 4. Dezvoltarea algoritmilor folosind variabile partajate

Una din problemele centrale ale dezvoltării algoritmilor folosind variabile partajate este **sincronizarea**, în cele două forme cunoscute: excluderea mutuală și sincronizarea condiționată. Prima formă înseamnă gruparea mai multor acțiuni ale unui proces într-o singură acțiune atomică, ascunzând astfel stările intermediare. Cea de a doua formă presupune întârzierea execuției unui proces până la îndeplinirea unei anumite condiții.

Nu este, deci, de mirare că în istoria algoritmilor cu variabile partajate s-a consumat un efort considerabil pentru găsirea unor mecanisme de rezolvare sistematică a problemei sincronizării, câteva rezultate notabile fiind reprezentate de:

- semafoare,
- regiuni critice,
- monitoare.

Câteva dintre problemele "clasice" rezolvate prin utilizarea acestora sunt:

- producători-consumatori,
- cititori-scriitori,
- problema filozofilor,
- problema bărbierului.

Problemele menționate își au izvorul în domeniul sistemelor de operare dar sunt utilizate astăzi în multe alte domenii care pun accent pe paralelizarea codului. Fiecare din soluțiile găsite își are importanța sa deosebită, toate la un loc alcătuind o "bibliotecă" de modele de rezolvare aflate la dispoziția proiectanților. Înglobarea lor în soluțiile diferitelor probleme nu este întotdeauna o sarcină ușoară, dacă se urmărește obținerea unor variante eficiente.

În acest capitol prezentăm soluțiile câtorva probleme clasice utilizând mecanismul **semafoarelor**. Acest mecanism folosește o variabilă cu un set limitat de valori întregi. Valoarea unui semafor poate fi modificată doar prin intermediul a două operații **primitive**, notate P și V, motiv pentru care variabilei i se asociază un tip special, notat aici **sem**. Primitivele P și V pot fi definite folosind notațiile de sincronizare și atomicitate prezentate într-un capitol anterior:

```
sem bin = 1 ;  
P(s) :: <await (bin>0) bin = bin-1;>  
V(s):: <bin = bin+1;>
```

Semaforul **bin** poate avea doar valorile 0 sau 1, motiv pentru care este numit **semafor binar**. Operatia P(bin) se poate executa doar daca bin=1; prin executia sa, valoarea lui **bin** devine 0. Operatia V nu include conditionari, rolul ei fiind cresterea valorii lui **bin** cu o unitate. Asa cum se va vedea in exemplele urmatoare, programele sunt structurate astfel incat oricare proces sa poata executa o primitiva V doar **dupa** executia unei primitive pereche P (cu alte cuvinte, doar cand valoarea semaforului este 0).

Valoarea initiala a semaforului poate fi mai mare ca 1, caz in care avem de a face cu un **semafor general**.

```
sem gen = 10 ;           /* valoarea initiala a lui gen este 10 */  
P(s) : <await (gen >0) gen = gen -1;>  
V(s):  < gen = gen +1;>
```

Teoretic, un semafor general poate avea orice valoare intreaga ne-negativa. Practic, valoarea semaforului **gen** nu coboara sub 0 si (prin programarea corespunzatoare a primitivelor P si V) nu depaseste valoarea initiala (care este, deci, valoarea maxima a semaforului).

Alte detalii despre semafoare si utilizarea lor sunt prezentate in exemplele ce urmeaza.

4.1 Sectiuni critice

Fiecare proces Proc[i] al unei colectii de procese Proc[i=1 to n] executa ciclic operatii intr-o **sectiune critica** in care are acces exclusiv la anumite **resurse partajate** de toate procesele, urmata de o **sectiune necritica** in care foloseste doar **resurse locale**. **Problema** ce trebuie rezolvata este asigurarea ca, in orice moment, cel mult un proces sa se afle in sectiunea critica. Cu alte cuvinte, trebuie controlata „intrarea” in si „iesirea” din sectiunea critica astfel incat sa se asigure excluderea mutuala a proceselor la executia sectiunilor critice.

Primitivele P si V asigura, intr-o forma simpla, accesul exclusiv la resursele partajate. Pentru asta, putem folosi un semafor **mutex** a carui valoare sa specifice daca resursele sunt libere (mutex=1, nici un proces nu este in sectiunea critica) sau ocupate (mutex=0, un proces este in sectiunea critica). Astfel, intrarea unui proces in sectiunea critica trebuie conditionata de executia cu succes a unei primitive P(mutex). La iesirea din sectiunea critica, procesul trebuie sa execute o primitiva V(mutex).

```

sem mutex = 1;
    /*denumirea mutex provine de la mutual exclusion */
process P[i=1 to n]{
    while (true){
        P(mutex);                /*acaparare sectiune critica*/
        Sectiune critica;
        V(mutex);                /*eliberare sectiune critica*/
        Sectiune necritica;
    }
}

```

Pentru intelegerea functionarii, este important sa ne amintim ca, prin definitie, operatiile P si V sunt atomice (doar una din ele se poate executa la un moment dat). Astfel, daca mutex=1 si mai multe procese vor sa intre in sectiunea critica, primul care executa primitiva P gaseste **mutex** la valoarea 1, o modifica la 0 si continua cu executia sectiunii critice. Executia celorlalte procese este suspendata pana cand procesul aflat in sectiunea critica termina si pune **mutex** din nou pe 1.

4.2 Producator si consumator

Problema: se considera un proces Producator si un proces Consumator care comunica printr-un singur tampon partajat. Un producator pune o valoare in tampon, iar un consumator ia o valoare din tampon. Trebuie asigurat ca valorile nu sunt suprascrise si nu sunt citite din tampon de mai multe ori.

Si in acest caz avem de a face cu procese care folosesc o resursa comuna, tamponul prin care se comunica valorile intre ele. Evident ca accesul trebuie sa fie exclusiv (la un moment dat, un singur proces are acces la tampon). Doar ca procesul Producator trebuie sa aiba acces cand tamponul este gol, in timp ce procesul consumator trebuie sa aiba acces cand tamponul este plin. Un singur semafor, care ar permite sau nu accesul, nu este suficient. In schimb, folosirea a doua semafoare, **gol** pentru accesul la tampon gol si **plin** pentru accesul la tampon plin, ar rezolva problema. In aceasta varianta, procesul Producator ar intra in sectiunea critica daca primitiva P(gol) este executata cu succes, iar procesul Consumator intra in sectiunea critica daca P(plin) este executata cu succes. La iesirea din sectiunea critica, procesul Producator trebuie sa anunte, prin V(plin), ca tamponul a fost umplut, iar procesul Consumator trebuie sa anunte, prin V(gol), ca tamponul a fost golit.

```

typeT buf;

```

```

sem gol = 1, plin = 0;          /* sem. binare */

process Producator (i= 1 to M){
    typeT v;
    while (true){
        v = produce ();
        P(gol);
        buf = v;
        V(plin)
    }
}

process Consumator [i= 1 to N){
    typeT w;
    while (true){
        P(plin);
        w := buf;
        V(gol);
        consuma (w);
    }
}

```

Semafoarele **gol** si **plin** sunt **binare** si asigura excluderea mutuala a proceselor, in sensul ca, la un moment dat, un singur proces (fie Producator, fie Consumator) depune sau extrage o valoare din **buf**. Valorile initiale ale semafoarelor marcheaza absenta unei valori din **buf** (**gol**=1, **plin**=0). Ca urmare, un proces Producator va putea pune o valoare in **buf**, modificand semafoarele pentru a marca prezenta unei valori in **buf** (**gol**=0, **plin**=1). Ca urmare, un proces Consumator va putea prelua valoarea, golind astfel tamponul **buf** si modificand semafoarele pentru a marca golirea tamponului (**gol**=1, **plin**=0).

Din aceasta descriere, trebuie sa remarcam ca cele doua semafoare **gol** si **plin** nu sunt independente unul de altul, in orice moment cel mult unul din ele putand avea valoarea 1. In plus, atunci cand unul din procese este in sectiunea critica ambele semafoare au valoarea 0. Un astfel de grup in care suma tuturor semafoarelor nu depaseste niciodata valoarea 1 se numeste **semafor binar splitat** (split binary semaphore). Regulile de baza ale unui semafor binar splitat sunt urmatoarele:

- initial, doar unul din semafoare are valoarea 1, restul zero;
- oricare proces care utilizeaza grupul de semafoare executa mai intai primitiva P pe un semafor din grup, doar dupa aceea executa primitiva V pe oricare semafor din grup;

- codul dintre primitivele P si V se va executa in excludere mutuala, toate semafoarele avand valoarea 0.

4.3 Comunicarea producator si consumator prin tampon limitat

Aceasta varianta foloseste drept tampon un tablou, care permite pastrarea mai multor valori la un moment dat. Cerintele sunt pastrate, adica trebuie asigurat ca valorile nu sunt suprascrise si nu sunt citite din tampon de mai multe ori. Se adauga conditia, care la varianta precedenta era indeplinita implicit, ca ordinea in care valorile sunt consumate sa fie cea in care valorile sunt produse.

De data aceasta, semafoarele **gol** si **plin** trebuie sa fie generale. Semaforul **gol** numara locurile goale din **buf**, iar **plin** numara valorile existente in **buf**. In plus, pentru a nu limita numarul valorilor transmise, alegem un tampon circular de dimensiune k.



Valorile sunt plasate in tampon in ordinea elementelor tabloului, de la 1 la k circular (dupa elementul k urmeaza 1). Primul element din tampon va fi indicat de o variabila **prim**, iar primul loc gol prin variabila **ultim**. Initial, ambele variabile au valoarea 1.

In functionare, procesul **Producator** acapareaza un loc gol (daca nu exista atunci se blocheaza pana se elibereaza unul) si depune o valoare. Pozitia curenta pentru **Producator** este desemnata de variabila **ultim**.

Procesul **Consumator** acapareaza o valoare depusa (daca nu exista atunci se blocheaza pana apare una) si preia valoarea din **buf**. Pozitia curenta pentru **Consumator** este desemnata de variabila **prim**. Operatiile de depunere si de extragere nu se exclud mutual. Procesul **Producator** poate depune o valoare in pozitia **ultim** din **buf** in timp ce **Consumator** preia una din pozitia **prim** din **buf**.

```
typeT buf [1:k];  
sem gol= k; plin= 0;    /*sem. generalizate*/  
  
process Producator {
```

```

    typeT v;
    int ultim = 1;    /*pozitia goala in tampon*/
    while (true){
        v = produce ();
        P(gol);        /*exista locuri goale?*/
        buf[ultim] = v; ultim = ultim mod k + 1;
        V(plin);
    }
}

process Consumator {
    typeT w;
    int prim = 1;    /*primul element din tampon*/
    while (true){
        P(plin);        /*exista valori in buf?*/
        w = buf[prim]; prim = prim mod k + 1;
        V(gol);
        consuma (w);
    }
}

```

Solutia tamponului circular este utila in situatii in care producerea, respectiv consumul valorilor se face in rafale: producatorul poate furniza mai multe valori fara a astepta preluarea imediata a fiecarei valori de catre consumator; la randul sau, consumatorul poate prelua valorile depuse in tampon in timp ce producatorul executa alte operatii de calcul.

4.4 Mai multi producatori si mai multi consumatori

Aceasta varianta este o extindere a solutiei anterioare, ea incluzand mai multi producatori si mai multi consumatori care partajeaza tamponul circular. Trebuie asigurat ca valorile nu sunt suprascrise si nu sunt citite din tampon de mai multe ori. De asemenea, ordinea in care valorile sunt consumate trebuie sa fie cea in care valorile sunt produse.

Sa nu uitam ca fiecare producator vrea sa plaseze o valoare produsa, in primul loc liber al tamponului. Asta cere ca variabila care indica acest loc (**ultim**) sa fie partajata intre producatori, iar accesul producatorilor sa fie exclusiv. Pentru a gestiona excluderea intre producatori vom folosi un semafor **mutexP**.

Aceeași situație o găsim la consumatori, care vor avea acces la variabila comună care indică prima valoare din tampon (**prim**), excluderea mutuală fiind gestionată prin semaforul **mutexC**.

```

typeT buf[1:k];
sem gol = k, plin = 0;           //sem. generalizate
int prim = 1, ultim = 1;
sem mutexP = 1, mutexC = 1;

process Producator[i = 1 to M]{
    typeT v;
    while (true){
        v = produce ();
        P(gol);
        P(mutexP);
        buf[ultim]=v; ultim = ultim mod k + 1;
        V(mutexP);
        V(plin);
    }
}

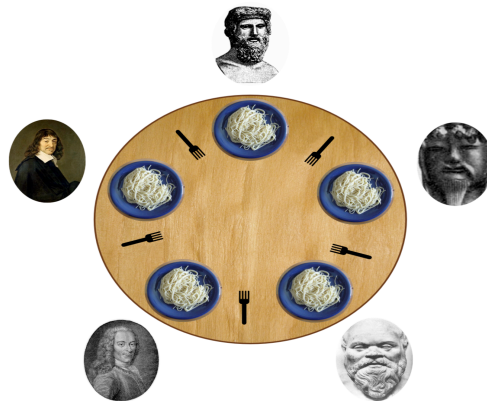
process Consumator[i = 1 to N]{
    typeT w;
    while (true){
        P(plin);
        P(mutexC);
        w := buf[prim]; prim := prim mod k + 1;
        V(mutexC);
        V(gol);
        consuma (w);
    }
}

```

4.5 Problema filozofilor

Cinci filozofi stau în jurul unei mese rotunde și își petrec timpul gândind și mâncând. În mijlocul mesei este o farfurie cu spaghetti. Pentru a putea mânca, un filozof are nevoie de două furculițe. Pe masă sunt cinci furculițe, fiecare situată între doi filozofi vecini. Regula este ca fiecare filozof poate folosi furculițele din imediată sau apropiere. Problema este să se scrie un program care simulează comportarea

filozofilor. In particular, trebuie evitata situatia in care nici un filozof nu poate acapara ambele furculite (de exemplu, ia furculita din stanga sa si nu ii mai da drumul).



Daca toti filozofii ar lua mai intai furculita din stanga si apoi pe cea din dreapta se poate ajunge la blocare definitiva prin asteptare circulara: fiecare asteapta o resursa pe care procesul care a acaparat-o nu o elibereaza. Solutia ca sa se evite asteptarea circulara este ca unul din filozofi sa procedeze altfel si anume sa ia mai intai furculita din dreapta si apoi pe cea din stanga.

```
sem f[1:5] = ([5] 1);
process Filozof [i=1 to 4]{
while (true){
    P(f[i]); P(f[i+1]);
    mananca;
    V(f[i]); V(f[i+1]);
    gandeste;}
}
process Filozof[5]{
while (true){
    P(f[1]); P(f[5]);
    mananca;
    V(f[1]); V(f[5]);
    gandeste;}
}
```


4.6 Problema cititorilor si scriitorilor

Problema: exista doua tipuri de procese, cititori si scriitori care impart o resursa. Mai multi cititori pot avea acces simultan la resursa comuna daca nici un scriitor nu o modifica in acelasi timp. Un scriitor poate modifica resursa daca ea nu este accesata de nici un alt proces, fie el scriitor sau cititor.

Algoritmul foloseste un semafor **rw** pentru excluderea mutuala intre procese. El asigura ca in orice moment, avem una din urmatoarele situatii:

- resursa nu este utilizata;
- resursa este utilizata de un singur scriitor;
- resursa este utilizata de unul sau mai multi cititori.

Semaforul este folosit in modul clasic de scriitori. Un scriitor poate intra in sectiunea critica daca operatia P(rw) se executa cu succes. La iesirea din sectiunea critica procesul scriitor executa o operatie V(rw) si elibereaza resursa.

Semaforul **rw** este gestionat putin mai complicat de cititori, deoarece resursa poate fi folosita simultan de mai multi cititori. Daca resursa este libera, un cititor o poate acapara daca operatia P(rw) reuseste. Alte procese cititor se pot alatura si pot intra in sectiunea critica fara a mai modifica semaforul **rw**. De asemenea, procesele cititor pot parasi sectiunea critica fara a modifica semaforul. Doar ultimul cititor care paraseste sectiunea critica trebuie sa elibereze resursa executand un V(rw). Pentru a determina care procese cititor executa operatiile P si V, algoritmul tine evidenta numarului de cititori, **nr** care folosesc resursa. Deoarece **nr** este partajata de cititori si operatiile asupra lui trebuie executate in regim de exclusivitate (un cititor la un moment dat), accesul la aceasta variabila este controlat de un semafor de excludere mutuala **mutexR**.

```
sem rw = 1, mutexR = 1;
int nr = 0;

process Cititor [i = 1 to m){
while (true){
    P(mutexR);
    nr = nr+1;
    if (nr = 1) P(rw); /*daca primul cititor*/
    V(mutexR);
    citeste din resursa comuna;
    P(mutexR)
    nr = nr-1;
```

```

        if (nr = 0) V(rw);    /*daca ultim cititor*/
    V(mutexR);}
}

process Scriitor (j = 1 to n){
while (true){
    P(rw);
    scrie in resursa comuna;
    V(rw);}
}

```

Solutia acorda preferinta cititorilor: daca un cititor acapareaza resursa, alti cititori pot sa i se alature si sa citeasca. Doar cand ultimul cititor a terminat, resursa este eliberata (V(rw)). Teoretic, procesele scriitor pot fi intarziate oricat de mult (termenul folosit este **starvation** – infometarea scriitorilor), in detrimentul actualizarii prompte a valorii resursei partajate.

Solutia pe care o prezentam in continuare asigura ca:

- (1) noile cereri de la cititori sunt intarziate (cititorul este pus in asteptare) daca un scriitor este in sectiunea critica sau asteapta sa intre in sectiunea critica;
- (2) un cititor este trezit din asteptare doar daca nu exista un scriitor in asteptare sau in sectiunea critica.

Pentru a aplica aceasta politica, avem nevoie de variabile de contorizare a cititorilor si scriitorilor care folosesc resursa sau sunt in asteptarea accesului la resursa:

nw - numarul de scriitori care folosesc resursa,
nr - numarul de cititori care folosesc resursa,
dw - numarul de scriitori care asteapta sa foloseasca resursa,
dr - numarul de cititori care asteapta sa foloseasca resursa.

Cu aceste notatii, conditiile anterioare pot fi specificate astfel:

- (1) $nw > 0$ or $dw > 0$
- (2) $nw = 0$ and $dw = 0$.

Similar, un scriitor este intarziat daca un alt scriitor sau un cititor se afla deja in sectiunea critica, adica:

$nr > 0$ or $nw > 0$.

Pentru controlul accesului la sectiunile critice conform acestor conditionari, definim urmatorul grup de semafoare:

e – semafor de intrare – controleaza intrarea intr-o actiune atomica si faciliteaza astfel executia mai multor operatii ca o singura actiune atomica;
 r – semafor pentru intarzierea cititorilor (asteapta ca nw=0);
 w – semafor pentru intarzierea scriitorilor (asteapta ca nw=0 si nr=0).
 In plus este nevoie de executia mai multor operatii ca o singura actiune atomica folosind in acest scop un semafor care cotroleaza intrarea intr-o astfel de actiune:

Cele trei semafoare alcatuiesc impreuna un **semafor binar splitat**. Tehnica de executie a primitivelor, numita "pasarea stafetei", se bazeaza pe urmatoarele reguli:

- initial un semafor este 1 si un proces poate prelua stafeta printr-o primitiva P asupra acestuia;
- cand un proces detine stafeta (se afla intr-o sectiune critica si toate semafoarele sunt 0), el poate executa o operatie V asupra unuia din cele trei semafoare, pasand stafeta unui alt proces, care "asteapta" la acel semafor (intr-o operatie P).

```
int nr = 0;      /*nr cititori care folosesc resursa */
int nw = 0;      /*nr scriitori care folosesc resursa */
int dr = 0;      /*nr cititori intarziati */
int dw = 0;      /*nr scriitori intarziati */
sem e = 1;       /*intrare sectiune atomica */
sem r = 0;       /*intarzie cititori daca nw>0 sau dw>0*/
sem w = 0;       /*asteapta ca nw=0 si nr=0*/
```

```
process Cititor [i = 1 to m]{
    while (true){
        P(e);
        if (nw>0 or dw>0){
            dr = dr+1; V(e); P(r);
        }
        nr = nr+1;
        if (dr>0) {dr = dr-1; V(r);}
        else if (dr=0) V(e);
    }
    citeste din resursa comuna;
    P(e);
    nr = nr-1;
    if (nr = 0 and dw>0){dw = dw-1; V(w);}
    else if (nr>0 or dw=0) V(e);
}
}
```

```

process Scriitor [j = 1 ton){
    while (true){
        P(e);
        if (nr>0 or nw>0){dw = dw+1; V(e); P(w);}
        nw = nw+1;
        V(e);
        scrie in resursa comuna;
        P(e);
        nw = nw-1;
        if (dr>0 and dw=0){dr = dr-1; V(r);}
        else if (dw>0){dw = dw-1; V(w);}
        else if (dr=0 and dw=0) V(e);
    }
}

```

Capitolul 5. Dezvoltarea aplicațiilor pentru calculatoare SM SIMD

O clasă importantă de sisteme paralele este cea care corespunde modelului SIMD cu memorie partajată (Shared Memory SIMD). Ele sunt cunoscute și sub denumirea de PRAM (Parallel Random Access Machines). În funcție de modul de acces la memorie pentru operațiile de citire și scriere, sistemele SM SIMD se subdivid în:

- EREW (Exclusive Read Exclusive Write);
- CREW (Concurrent Read Exclusive Write);
- ERCW (Exclusive Read Concurrent Write);
- CRCW (Concurrent Read Concurrent Write).

Descrierea urmărește prezentarea unor metode generale de dezvoltare a algoritmilor paraleli (partiționarea, divide et impera) și a unei abordări posibile pentru obținerea algoritmilor cost-optimali.

Câteva **caracteristici** importante ale algoritmilor SM SIMD sunt următoarele:

- algoritmi indică acțiunile executate de procesoare (în general, fiecare procesor este dedicat execuției acțiunilor unui proces)
- algoritmi explicitează separat acțiunile de citire și cele de scriere (pentru a putea aprecia costul operațiilor în raport cu categoriile de sisteme PRAM menționate)
- lansarea și terminarea proceselor au "overhead" redus (instrucțiuni **co ... oc** pot fi incluse în cicluri)
- algoritmi se încadrează, de obicei, în clasa "fine grain" (granularitate fină).

Aceste caracteristici sunt subliniate prin notația specifică PRAM, care a fost prezentată în capitolul 2 al lucrării.

5.1. Proprietățile (dorite ale) algoritmilor paraleli

Număr de procesoare

În cele ce urmează, **n** reprezintă dimensiunea problemei, iar **p(n)** este numărul de procesoare utilizate la execuția algoritmului. Numărul de procesoare trebuie să îndeplinească următoarele cerințe:

- **p(n)** trebuie să fie mai mic decât **n**

Este nerealist să presupunem că avem **n** procesoare. Uzual se ia **p(n)** ca funcție sublineară de **n** (de exemplu, **log n** sau **sqrt(n)**)

- **p(n)** trebuie să fie adaptiv

Algoritmul nu trebuie să depindă de un număr fix de procesoare.

Timp de execuție

În cele ce urmează, $t(n)$ este timpul de execuție în funcție de dimensiunea problemei. Condițiile sunt următoarele:

- $t(n)$ să fie mic,
semnificativ mai mic decât timpul de execuție al variantei secvențiale
- $t(n)$ să fie adaptiv,
și să fie invers proportional cu $p(n)$

Cost

În cele ce urmează, $c(n)$ este costul algoritmului $c(n) = t(n) * p(n)$. Condițiile sunt următoarele:

- $c(n)$ să fie minim
deci **algoritmul să fie cost-optimal**.

Exemplu

Algoritmul de **selecție** a unei valori din secvența $S = \{s_1, \dots, s_n\}$, prezentat mai departe în acest capitol, se execută pe o mașină EREW SM SIMD cu N procesoare, unde $N < n$. Algoritmul are proprietățile formulate mai înainte.

(i) Algoritmul folosește $p(n) = n^{1-x}$ procesoare, cu $0 < x < 1$, unde valoarea lui x se obține din formula $N = n^{1-x}$. Deci, $p(n)$ este subliniar și adaptiv.

(ii) Algoritmul se execută în $t(n) = O(n^x)$, cu x obținut ca mai sus. Deci, timpul de execuție este mai scurt decât cel al unui algoritm secvențial. Totodată, algoritmul este adaptiv: cu cât este mai mare $p(n)$, cu atât este mai mic $t(n)$.

(iii) Costul este $c(n) = n^{1-x} * O(n^x) = O(n)$, ceea ce reprezintă un optim.

Observații

i) Satisfacerea cerințelor menționate este dificilă, uneori imposibilă.

ii) În practică, pentru valorile $c(n)$, $p(n)$, $t(n)$ se consideră valorile rotunjite obținute printr-o estimare pesimistă.

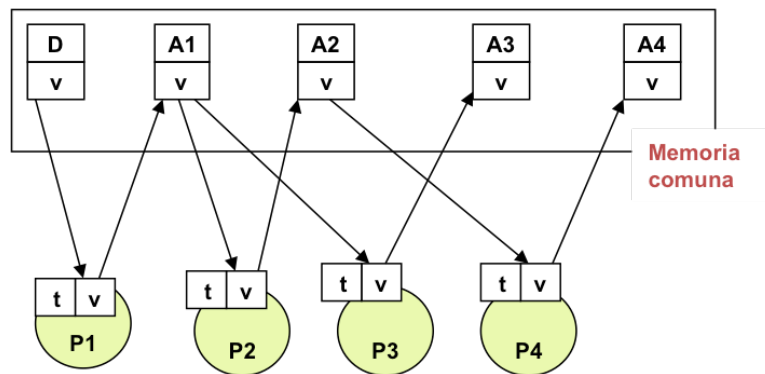
5.2. Două proceduri utile

Două operații tipice apar în execuția algoritmilor paraleli în sisteme EREW SIMD:

- fiecare procesor trebuie să citească conținutul unei celule din memoria comună (difuzarea unei valori);
- fiecare procesor trebuie să calculeze valoarea unei funcții dependente de datele deținute de celelalte procesoare (calculare prefix).

Difuzarea unei valori

Fie D celula din memoria comună ce trebuie difuzată celor N procesoare ale unui sistem EREW SIMD. Algoritmul BROADCAST presupune folosirea unui tablou $A[1:N]$. Acțiunile de citire și de scriere sunt explicitate distinct și explicit în descrierea algoritmului. Acest tratament este specific sistemelor PRAM.



```

int D, A[1:N];
Pas 1: Process P[1]{
    int t;
    1.1. citește valoarea din D;
    1.2. o memorează în propria memorie t;
    1.3. o scrie în A[1];
}

Pas 2: Process P[j=2 to N]{
    int t;
    for [i = 0 to (log N -1)]{
        if (j >= 2i+1 and j <= 2i+1){
            2.1. citește valoarea din A[j-2i];
            2.2. o memorează în propria memorie t;
        }
    }
}

```

```

                2.3. o scrie în A[j];
            }
        }
    }

```

Acțiunile de citire și de scriere sunt specificate distinct și explicit în descrierea algoritmului, iar pașii executați sincron de procesoare sunt foarte bine precizați.

Observații

- Deoarece numărul de procesoare care dețin valoarea din D se dublează la fiecare iterație, timpul de execuție este $O(\log N)$.
- Dimensiunea tabloului A poate fi redusă la $N/2$, valorile memorate în ultimul pas fiind inutile.
- Utilizare tipică: difuzarea lui n , în cazul algoritmilor adaptivi.

Calculul sumelor prefix

Fiind dat vectorul $A[1:N]$, fiecare procesor P_i calculează suma

$A[1] + \dots + A[i]$.

Algoritmul este următorul:

```

int a[1:n];
process suma [k=1 to n]{
    int temp;           /*temp este locala procesului k*/
    for [j = 1 to sup(log2 n)]
        if (k-2j-1>=1){
            temp = a[k-2j-1];
            a[k] = temp + a[k];
        }
    }
}

```

Observație

- Deoarece numărul procesoarelor care termină de calculat suma parțială se dublează la fiecare pas, calculul se face în $O(\log N)$ pași folosind $O(N)$ procesoare.

5.3. Căutarea paralelă

În cele ce urmează, prezentăm soluția unei probleme de căutare a unei valori x într-o secvență ordonată S de n valori, pentru o mașină SM SIMD. Pentru simplitate, considerăm că elementele secvenței sunt toate distincte între ele, adică:

$$s[1] < s[2] < \dots < s[n].$$

Metoda secvențială de căutare binară constă în compararea lui x cu elementul din mijlocul secvenței S . În funcție de rezultatul acestei comparații, căutarea se termină sau continuă cu una din cele două jumătăți separate de elementul din mijloc. Algoritmul are complexitatea $O(\log n)$.

Să considerăm mai întâi cazul unui sistem EREW format din N procesoare, $1 \leq N \leq n$. Pentru a face cunoscută valoarea x tuturor procesoarelor, se poate folosi un algoritm de difuzare (broadcast) care are complexitatea $O(\log N)$. Secvența S este divizată în N subsecvențe de lungime n/N , fiecare subsecvență fiind atribuită unui procesor. Folosind o procedură de căutare binară pe secvența atribuită, un procesor găsește rezultatul în $O(\log(n/N))$ în cazul cel mai defavorabil. Timpul total cerut este deci de ordinul $O(\log N) + O(\log(n/N))$ adică $O(\log n)$, același cu timpul necesar căutării binare pe un procesor secvențial.

În cazul unui sistem CREW, difuzarea valorii lui x se poate face într-un singur pas. Mai mult, se poate adopta o politică de căutare diferită, obținându-se o performanță mai bună pentru timpul de execuție.

Algoritmul se bazează pe împărțirea, în fiecare etapă, a secvenței în care se face căutarea în $N+1$ subsecvențe de aceeași lungime. Fiecare din cele N procesoare inspectează o valoare s aflată în poziția dintre două secvențe adiacente. Dacă $x < s$ atunci toate elementele aflate la dreapta lui s sunt ignorate în pasul următor. Similar, dacă $x > s$ sunt ignorate elementele mai mici ca s . Următorul subinterval de căutare se obține făcând intersecția subintervalelor păstrate de diferitele procesoare. Deoarece fiecare etapă se aplică unei secvențe de dimensiune $1/(N+1)$ din secvența precedentă, sunt necesare $O(\log_{N+1}(n+1))$ etape.

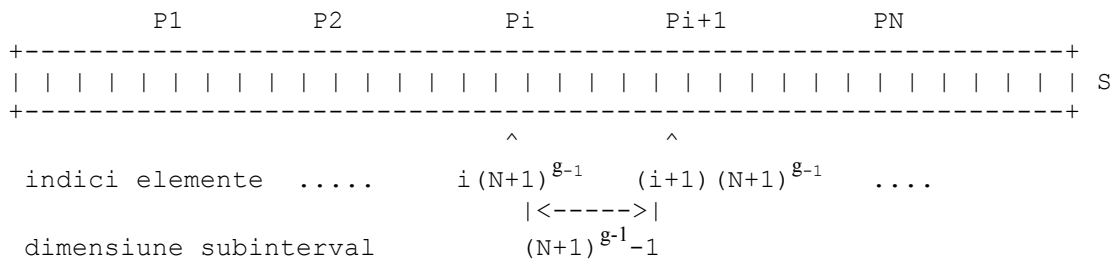


Figura 5.1

Fie relația adevărată pentru $(N+1)^{g-1}$. Pentru a inspecta o secvență de lungime $(N+1)^g-1$, procesorul P_i compară x cu $s[j]$, unde $j = i(N+1)^{g-1}$, așa cum se arată în figura 1. În urma comparației, se reține o subsecvență de dimensiune $(N+1)^{g-1}$, care (conform ipotezei de inducție) se poate inspecta în $g-1$ etape.

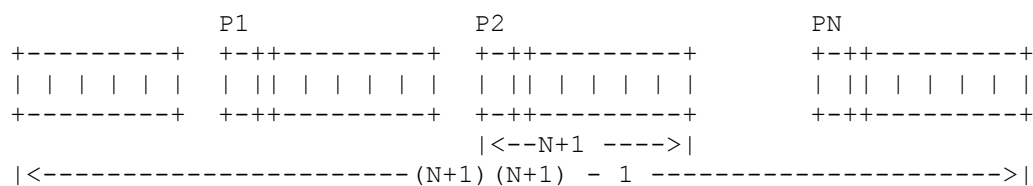
$$\begin{array}{ccccccc} \text{P1} & \text{P2} & & & & & \text{PN} \\ + & - & - & - & - & - & + \\ | & | & | & | & | & | & \\ + & - & - & - & - & - & + \end{array} \quad n_1 = N$$


Figura 5.2

În general, un sistem cu N procesoare poate "acoperi" în g pași o secvență de lungime cel mult $n_g = (N+1)^g - 1$. În primul pas, această secvență este împărțită în subsecvențe de $n_{g-1} = (N+1)^{g-1} - 1$ elemente, procesorul P_i inspectând elementul din poziția $i * (N+1)^{g-1}$ (considerând că primul element are poziția 1). Dacă primul element are indicele q , atunci poziția care corespunde lui P_i este $(q-1) + i * (N+1)^{g-1}$.

Ca urmare, dată fiind lungimea n a unei secvențe de numere, numărul de pași g ai procedurii de căutare rezultă din relația $n \leq (N+1)^g - 1$, adică este $g = \sup(\log(n+1)/\log(N+1))$.

Exemplul 1

Pentru exemplificarea execuției algoritmului, fie secvența de numere dată în figura .3a și $x=45$, $N=3$. Deoarece $n=15$, g este cel mai mic număr care satisface relația

$(N+1)^g \geq n+1$, adică $4^g \geq 16$. Rezultă $g=2$. Lungimea unei subsecvențe în primul pas este egală cu 3. Rezolvarea este schițată în figura 5.3.

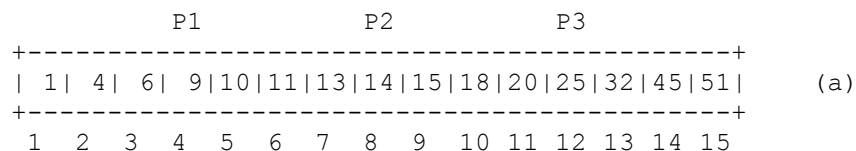


Figura 5.3a

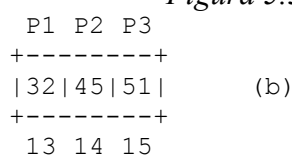


Figura 5.3b

Exemplul 2

Să considerăm o a doua execuție, cu $N=2$, $x=21$. Deci, din relația $(N+1)^g \geq n+1$ rezultă $3^g \geq 16$, deci $g=3$. Lungimea unei subsecvențe în primul pas este egală cu 8. Rezolvarea este schițată în figura 5.4.

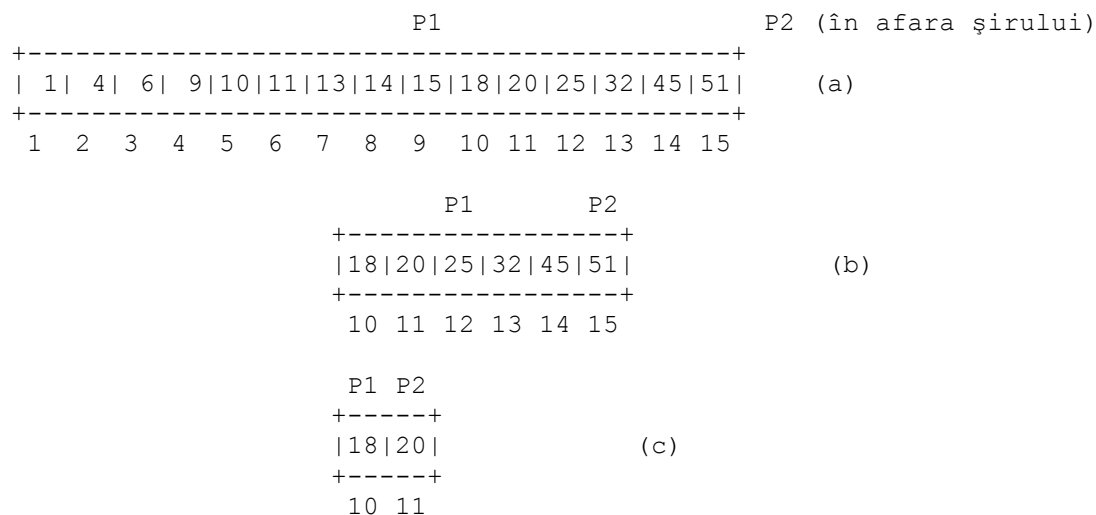


Figura 5.4

Pentru a determina subintervalul care rămâne în fiecare etapă, fiecare proces P_i , $1 \leq i \leq N$ utilizează o variabilă $c[i]$ care ia una din valorile **stg** sau **dr**, după cum trebuie reținută partea din stanga, respectiv cea din dreapta. În plus, se folosesc $c[0]=dr$ și $c[N+1]=stg$. Inițial limitele subsecvenței de căutare sunt $q=1$ și $r=n$. De asemenea, g are inițial valoarea numărului maxim de etape

$$g = \lceil \log(n+1)/\log(N+1) \rceil$$

și scade cu o unitate la fiecare iterație.

Algoritmul calculează valorile $c[i]$ și găsește i pentru care $c[i-1] \neq c[i]$. Pe baza lui se determină următorul subinterval $[q,r]$ de căutare. Mai precis,

$$q := q + (i-1)(N+1)^{g-1} + 1, \text{ iar}$$

$$r := q + i(N+1)^{g-1} - 1.$$

Un singur procesor calculează noile valori pentru q și r , celelalte având acces la aceste valori în timp constant.

În descrierea ce urmează sunt marcate cele trei etape ale algoritmului, precum și pașii care-i compun.

```

/* Initializarea indicilor secvenței de cautat */
int q = 1; /* limita stg interval crt */
int r = n; /* limita dr interval crt */
/* Initializarea rezultatului k si a numarului de pasi g */
int x, k = 0; /* x = numar cautat k = pozitia lui in sir */
int g = sup(log(n+1)/log(N+1))
enum sens {stg,dr};
sens c[0:N+1]; /* c[i] - x este la stg / dr pozitiei procesului i */
c[0] = dr; c[N+1] = stg;
int j[1:N]; /* j[i] = pozitia din sir inspectata de proces i */
process P[i = 1 to N]{
/* gaseste pozitia k unde se afla valoarea x */
    while (q<=r and k=0){ /* subsir exista si valoare negasita */
        j[i] = (q-1) + i*(N+1)g-1;
/* P[i] compara x cu s[j[i]] si determina partea de secv acceptata */
        if (j[i] <= r) /* pozitie in interior sir */
            if (s[j[i]] == x) k = j[i]; /* pozitie gasita */
            else if (s[j[i]] > x) c[i] = stg;
            else c[i] = dr;
        else {j[i] = r+1; /* j[i] indica o pozitie afara din sir */
            c[i] = stg;}
/* calculeaza indicii subsecvenței urmatoare */
/* doar un proces poate face modificarea intervalului de cautare */

```

```

        if (c[i] <> c[i-1]){
            r = j[i] - 1;
            if (i>1) q = j[i-1] + 1;}
/* doar procesul N executa operatia urmatoare */
        if (i==N and c[i]<>c[i+1]) q = j[i] +1;
        g := g-1;
    }      /*end while*/
}          /* end process */

```

Observație

Operațiile paralele din algoritm se execută în același timp în toate procesoarele. Operațiile secvențiale sunt executate de un singur procesor.

Pentru **exemplificarea** execuției algoritmului, fie secvența de numere dată în figura .5a și $x=45$, $N=3$. Inițial, $q=1$, $r=15$, $k=0$ și $g=2$. Rezultă $j_1=4$, $j_2=8$ și $j_3=12$. Ca urmare, în prima etapă:

P1 compară x cu $s[4]$ și deoarece $45>9$ alege $c[1]=dr$;

P2 compară x cu $s[8]$ și deoarece $45>14$ alege $c[2]=dr$;

P3 compară x cu $s[12]$ și deoarece $45>25$ alege $c[3]=dr$.

Rezultă $c[3]<c[4]$. Ca urmare, se pune $q=13$, iar r rămâne 15. Secvența se restrânge la cele trei elemente din figura .5b, iar g devine 1.

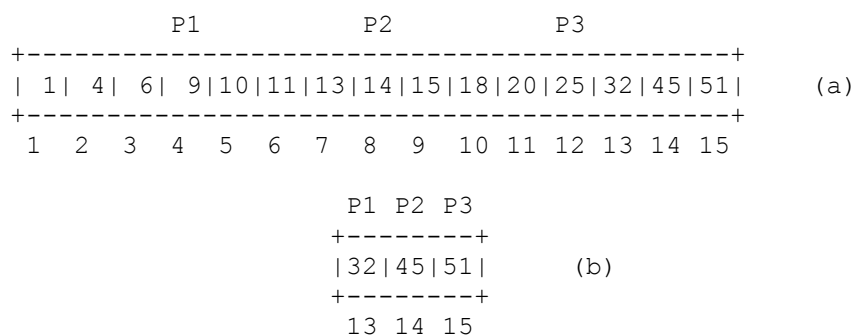


Figura 5.5

În a doua etapă, se obțin valorile $j_1=13$, $j_2=14$ și $j_3=15$. Ca urmare:

P1 compară x cu $s[13]$ și deoarece $45>32$ alege $c[1]=dr$;

P2 compară x cu $s[14]$ și deoarece $45=45$ pune $k=14$ și lasă $c[2]$ neschimbat;

P3 compară x cu $s[15]$ și deoarece $45<51$ alege $c[3]=stg$.

Noile valori pentru q și r sunt 15, respectiv 14. Oricum, iterațiile se opresc datorită condiției $k < 0$.

Să considerăm o a doua execuție, cu $N=2$, $x=21$. Inițial $g=3$. În prima iterație avem $j_1=9$ și $j_2=18 > 15$, deci în afara secvenței. Ca urmare:

P1 compară x cu $s[9]$ și deoarece $21 > 15$ alege $c[1]=dr$;

P2 pune j_2 la 16 (deoarece valoarea calculată este în afara secvenței) și actualizează $c[2]=stg$.

Deoarece $c[1] < c[2]$, se stabilesc noile valori $q=10$ și $r=15$, obținându-se subsecvența din figura 5.6.b și $g=2$.

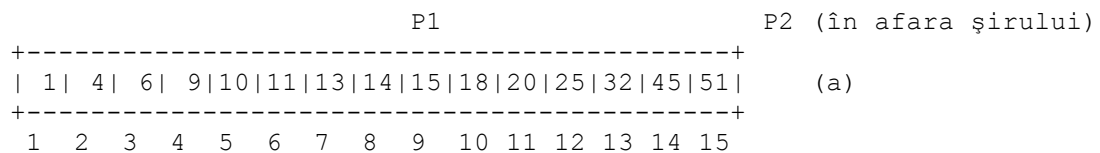


Figura 5.6a

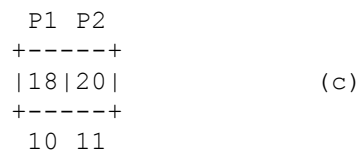
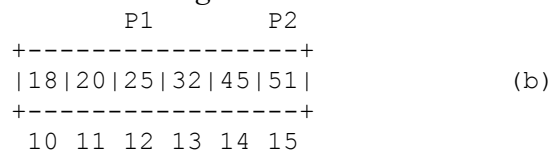


Figura 5.6b,c

În a doua iterație, P1 calculează $j_1=9+3$ și compară x cu $s[12]$. Deoarece $21 < 25$ se obține $c[1]=stg$. Similar, $j_2=15$ și, deoarece $21 < 51$, avem $c[2]=stg$. Ca urmare, $c[0] < c[1]$ și intervalul se restrânge pentru a treia iterație la $r=11$ și q nemodificat (figura 5.6c).

În a treia iterație, j_1 este 10, P1 calculând $c[1] = dr$, deoarece $21 > 18$. Similar, P2 calculează $c[2] = dr$, deoarece $21 > 20$. Ca urmare q capătă valoarea 12, mai mare decât valoarea lui r provocând astfel ieșirea din ciclu cu $k=0$.

Observații

Deoarece pasul 3 al algoritmului precedent se execută de cel mult g ori, rezultă că algoritmul are complexitatea $t(n) = O(\log_{N+1}(n+1))$. Lucrul efectuat de algoritm este $O(N \log_{N+1}(n+1))$, deci algoritmul nu este cost-optimal. Cu toate acestea, se poate arăta că el este optim din punct de vedere al timpului de calcul. Pentru a arăta acest lucru, să observăm că, folosind N procesoare, algoritmul poate compara x cu cel mult N valori $s[i]$, la un moment dat. După aceste comparații și după eliminarea din secvență a elementelor despre care se știe că nu sunt egale cu x , lungimea subsecvenței rămase este de cel puțin:

$$\sup((n-N)/(N+1)) \geq (n-N)/(N+1) = [(n+1)/(N+1)]-1.$$

După g repetiții, secvența rămasă este de lungime

$$[(n+1)/(N+1)^g]-1.$$

Deci, numărul de iterații cerute de orice algoritm paralel trebuie să fie mai mare de cel mai mic g pentru care avem:

$$[(n+1)/(N+1)^g]-1 \leq 0, \text{ adică}$$

$$g = \sup(\log(n+1)/\log(N+1)).$$

În cazul în care $N=n$ problema este rezolvată în timp constant.

În privința cerinței ca toate elementele secvenței S să fie **distincte**, ea poate fi înlăturată dacă actualizarea lui k la găsirea unei coincidențe este realizată printr-o procedură specială care evită conflictele de acces simultan (modelele folosite până acum sunt cu scriere exclusivă). Această procedură ia un timp $O(\log N)$, mărind astfel timpul de execuție al algoritmului la:

$$t(n) = O(\log(n+1)/\log(N+1)) + O(\log N).$$

Putem aprecia importanța acestui timp suplimentar dacă luăm cazul $N=n$, în care nu se realizează nici o îmbunătățire față de algoritmul secvențial.

Menținerea eficienței algoritmului în cazul în care în șir pot exista **valori egale** se poate realiza prin folosirea unui model CRCW. În acest caz, este posibil ca la un conflict (mai multe elemente egale cu x se află în secvența S), doar cea mai mică valoare a indicilor elementelor respective să fie dată lui k .

6. Comunicarea prin mesaje

În cele prezentate pînă aici, comunicarea între procese este realizată prin date comune (globale), iar sincronizarea prin mecanismul furnizat de instrucțiunea **await**. Transmiterea de mesaje, la rîndul său, îndeplinește cele două funcții: **comunicarea** este realizată deoarece datele ajung de la transmițător la receptor, iar **sincronizarea** este realizată deoarece un mesaj nu poate fi recepționat înainte de a fi transmis.

La utilizarea comunicării prin mesaje, canalele sunt obiectele puse în comun de procese. Absența unor variabile comune reclamă tehnici speciale, diferite de cele folosite în cazul memoriei partajate. Ele țin cont de faptul că procesele comunicante prin mesaje pot fi distribuite unor procesoare care nu au memorie comună. Din acest motiv, **programele paralele** care utilizează comunicare de mesaje se mai numesc **programe distribuite**. Asta nu înseamnă că astfel de programe nu pot fi executate de sisteme cu memorie comună. În acest caz, canalele sunt implementate folosind memoria partajată și nu o rețea de comunicație.

6.1. Comunicarea asincronă prin mesaje

În cazul comunicării prin mesaje asincrone, **canalele** sunt cozi nelimitate de mesaje. Un proces adaugă un mesaj unei cozi prin execuția unei operații **send**. Deoarece cozile sunt teoretic infinite, operația nu este blocantă. Un proces primește un mesaj dintr-o coadă prin execuția unei operații **receive**. Cînd coada este vidă, operația întîrzie procesul, pînă la primirea unui mesaj. Canalele păstrează ordinea mesajelor, deci sunt cozi FIFO.

Declarația unui canal are forma următoare:

chan nume_canal (tip₁ id₁,..., tip_n id_n)

în care se specifică numele canalului, precum și numele și tipurile cîmpurilor care compun un mesaj transmis prin acel canal.

Se pot declara și **tablouri (grupuri indexate) de canale** ca în exemplul următor:

chan rezultat [1:n](int)

în care se declară n canale avînd mesaje cu un singur cîmp de tip întreg. Numele cîmpurilor sunt opționale, fapt rezultat și din exemplul specificat.

Un proces transmite un mesaj unui canal executînd operația

send nume_canal (expresie₁,..., expresie_n)

unde expresiile trebuie să aibă aceleași tipuri cu cele ale câmpurilor mesajelor, specificate în declarația canalului. Efectul execuției unui `send` este calculul valorilor expresiilor, alcătuirea unui mesaj cu aceste valori și adăugarea lui la sfârșitul cozii asociate canalului.

Un proces recepționează un mesaj prin execuția unei operații

receive nume_canal (variabila₁,..., variabila_n)

unde variabilele trebuie să aibă aceleași tipuri cu câmpurile din declarația canalului. Efectul este întârzierea receptorului pînă cînd în coadă există cel puțin un mesaj, urmată de preluarea primului mesaj din coadă și atribuirea valorilor câmpurilor variabilelor din listă.

Deoarece canalele sunt (declerate) globale, orice proces poate transmite și recepționa pe un canal. Cînd canalele sunt folosite astfel, ele mai sunt numite **cutii poștale**. Cînd receptorul este unic, avem de-a face cu un **port**. În fine, atunci cînd și transmițătorul este unic, avem de-a face cu o **legătură**.

O altă operație utilă este testul existenței mesajelor pe un canal, avînd forma

empty (nume_canal)

și întorcînd valoarea adevărat sau fals, după cum canalul nu are nici un mesaj, sau are cel puțin unul. Utilizarea acestui test trebuie făcută cu grijă, avînd în vedere că:

- dacă testul este adevărat, nu înseamnă că la reluarea execuției procesului canalul este în continuare gol;
- dacă testul este fals, nu înseamnă că următorul **receive** va produce recepția imediată a unui mesaj (decît dacă există un singur receptor).

Exemplu. (Filtru) Prezentăm codul unui proces care transformă un flux de caractere într-o succesiune de linii de lungime maximă cunoscută.

```
chan input (char), output (char [1:Maxl]);  
process car_linii {  
    char line [1:Maxl]; int i =1;  
    while true {  
        receive input (line[i]);  
        while (line[i] <> CR and i < Maxl) {  
            /*line[1:i] = ultimele i caractere introduse*/
```

```

        i = i+1; receive input (line[i]);
    }
    send output (line); i = 1}
}

```

Alt exemplu. (Replicated workers)

În problema cuadraturii, trebuie calculată aria mărginită de graficul unei funcții continue ne-negative $f(x)$, axa x și dreptele $x=l$ și $x=r$. Soluția clasică impune împărțirea intervalului $[l,r]$ într-un număr de subintervale și folosirea unui trapez pentru a aproxima aria corespunzătoare fiecărui subinterval. Dublăm apoi numărul de subintervale din $[l,r]$ și comparăm rezultatul obținut cu cel anterior. Dacă diferența este acceptabilă calculul se termină, altfel se recurge la o nouă dublare a numărului de subintervale.

O soluție mai bună este cea adaptivă, în care se pornește procesul cu aria obținută prin considerarea unui singur subinterval $[l,r]$ și apoi a două subintervale $[l,m]$ și $[m,r]$. Dacă rezultatele sînt suficient de apropiate, aria trapezului mare este considerată o aproximație bună a ariei de sub curba $f(x)$. Dacă nu, problema se împarte în două subprobleme care se tratează separat: calculul ariei din subintervalul $[l,m]$ și calculul ariei din subintervalul $[m,r]$. Se repetă acest proces pînă cînd se obține o soluție acceptabilă pentru fiecare subinterval. Spre deosebire de prima soluție, în a doua se generează noi subprobleme doar pentru o parte din subintervale. În ambele cazuri, subproblemele sînt independente, ceea ce permite paralelizarea ușoară a algoritmului.

În acest scop, se utilizează un proces **administrator**, care generează prima problemă și așteaptă rezultatul, precum și mai multe procese de **lucru**, care rezolvă subproblemele, generând eventual alte subprobleme. Procesele de lucru împart un același canal sac, ce conține problemele de rezolvat. O problemă este caracterizată prin cinci valori: limitele subintervalului a și b , valorile $f(a)$ și $f(b)$, precum și aria corespunzătoare (pentru a evita recalcularea sa). Cînd un proces de lucru găsește răspunsul la o subproblemă, el trimite administratorului, printr-un canal separat, rezultatul. Programul se termină cînd administratorul detectează calculul întregii arii.

```

chan sac (real a, b, fa, fb, aria);
chan rezultat (real a, b, aria);
process Administrator {
    real l, r, fl, fr, a, b, aria, total;

```

```

    alte variabile marcare intervale terminate;
    fl = f(l); fr = f(r);
    aria = (fl+fr)*(r-l)/2;
    send sac(l, r, fl, fr, aria);
    while (nu s-a calculat toata aria) {
        receive rezultat (a, b, aria);
        total = total + aria;
        marcheaza intervalul [a,b] terminat;
    }
}

process Lucru [i = 1 to n] {
    real a, b, m, fa, fb, fm;
    real arias, ariad, ariat, dif;
    while (true) {
        receive sac (a, b, fa, fb, ariat);
        m := (a+b)/2; fm := f(m);
        calculeaza arias si ariad;
        dif := ariat - arias - ariad;
        if (dif mic) send rezultat (a, b, ariat);
        else {send sac (a, m, fa, fm, arias);
              send sac (m, b, fm, fb, ariad);}
    }
}

```

Exerciții

- 1) In programul precedent, toate procesele de lucru receptioneaza din același canal (sac). Modificați programul astfel încat să se folosească doar canale cu un singur receptor și mai mulți transmițători.
- 2) Scrieți un program de sortare rapidă folosind procese replicate. Inițial, tabloul $a[1:n]$ este local administratorului. La terminarea programului, șirul sortat trebuie înapoiat administratorului care-l va memora în $a[1:n]$.
- 3) Folosiți metoda proceselor replicate pentru a găsi toate soluțiile în problema celor 8 regine.

6.2. Comunicarea sincronă prin mesaje

Comunicarea asincronă de mesaje are unele neajunsuri privind confirmarea recepției și eficiența:

- pentru ca transmițătorul să fie informat de recepția mesajului comunicat, este nevoie

de un mesaj de răspuns;

- dacă răspunsul nu sosește, transmițătorul nu poate ști dacă mesajul a fost sau nu primit (receptorul a căzut înainte de , sau după primirea mesajului);
- spațiul ocupat de mesajele transmise dar nerecepționate poate fi mare și deci inefficient utilizat.

Comunicarea **sincronă** elimină aceste neajunsuri. Aici, ambele primitive, **send** și **receive** sunt blocante. Astfel, la execuția unei transmisii, procesul așteaptă pînă cînd un altul efectuează o recepție din același canal. Transmițătorul poate continua dacă mesajul a fost efectiv preluat. Mesajele nu trebuie să fie memorate temporar în cozi.

Pe de altă parte, comunicarea sincronă este uneori neconvenabilă. După cum se vede, ea este o legătură unu la unu între procese, astfel încît anumite operații, cum ar fi difuzarea de mesaje, sunt greu de realizat.

Un proces transmite **sincron** un mesaj unui canal executînd operația

synch_send nume_canal (expresie₁,..., expresie_n)

Exemplu – comunicare producator / consumator

```
chan prodcons (real);  
process prod {  
    real date[n];  
    for [i = 1 to n] {  
        calculeaza date[i];  
        synch_send prodcons (date[i]);  
        /* intarzie transmitatorul */  
    }    /* daca receptorul este mai lent */  
}  
process cons {  
    real buf[n];  
    for [i = 1 to n] {  
        receive prodcons (buf[i]);  
        prelucreaza datele primite;  
    }  
}
```

6.3. Modelul CSP pentru comunicarea sincronă

Pentru reprezentarea primitivelor de comunicare sincronă, putem folosi și notația din CSP (Hoare) relativă la comunicarea cu gardă. Aceasta poate fi folosită în egală măsură în comunicația asincronă sau în tehnica **rendezvous**. (Spre deosebire de comunicarea sincronă, numită și rendezvous simplu, în tehnica rendezvous extins un proces acceptă un apel executat de un alt proces, prin intermediul unei instrucțiuni specifice **input** sau **accept**, realizează diverse prelucrări și apoi întoarce rezultatele obținute.)

Modelul CSP folosește construcții diferite de notațiile prezentate de noi anterior, nu doar pentru comunicarea sincronă ci și pentru decizie (selecție), iterație și altele. Prezentăm pe scurt aceste notații.

Declarația variabilelor are forma generală

var $id_1 : tip_1 := val_1; \dots ; id_n : tip_n := val_n;$
unde inițializările sunt opționale.

Comanda cu gardă are forma generală

$B \rightarrow S$

unde B este o expresie booleană

S este o instrucțiune (simplă sau compusă). S nu poate fi executată dacă B nu este *true*.

Selecția (if) are forma generală

if $B_1 \rightarrow S_1$

$[] B_2 \rightarrow S_2$

...

$[] B_n \rightarrow S_n$

fi

Gărzile sunt evaluate într-o ordine arbitrară. Dacă B_i este *true* atunci se execută S_i . Cu alte cuvinte, dacă mai multe gărzi sunt *true*, alegerea instrucțiunii executate este nedeterministă.

Iterația (do) are forma generală

```

do B1 -> S1
[] B2 -> S2
...
[] Bn -> Sn
od

```

Comunicarea sincronă între procese are loc prin primitive de intrare/ieșire, de forma:

Destinație ! port (e₁,...,e_n)
 Sursă ? port (v₁,...,v_n)

unde **Destinație** și **Sursă** sunt procese, iar **port** identifică unul din canalele de comunicație ale acestora. Pentru prima dintre primitive, de ieșire, valorile expresiilor e_i sunt transmise portului specificat al procesului **Destinație**. Portul poate lipsi, dacă nu există posibilitatea de confuzie, iar dacă mesajul are un singur câmp, se poate renunța și la paranteze.

Pentru a doua primitivă, mesajul primit la portul specificat, de la procesul **Sursă**, este asociat variabilelor v_i.

De menționat că primitivele se execută simultan în tandem, dacă:

- instrucțiunea de ieșire apare în procesul specificat de instrucțiunea de intrare;
- instrucțiunea de intrare apare în procesul specificat de instrucțiunea de ieșire;
- identificatorii porturilor coincid;
- toate atribuirile de forma v_i := e_i ar fi valide.

Fie instrucțiunile:

- (1) X ? (a,b) și
- (2) DIV ! (3*x+y, 12).

Conform primeia, se preiau de la procesul X două valori și se atribuie variabilelor **a** și **b**. Prin cea de a doua, se evaluează cele două expresii și se transmit procesului DIV rezultatele. Dacă procesul DIV execută comanda (1), iar procesul X comanda (2), atunci cele două se derulează simultan și au același efect ca atribuirea (a,b) := (3*x+y, 12).

Exemple.

1) **Filtru**. Procesul **Copy** face copierea unor valori de la procesul **vest** la procesul **est**:

```

process Copy { char c;
    do true -> vest ? c; est ! c od
}

```

2) **Client - Server**. Calculul cmmdc printr-un proces server, CMMDC:

```

process CMMDC { int id, x, y;
    do true -> CLIENT[*] ? arg (id,x,y);
        do x > y -> x := x-y;
        [] x < y -> y := y-x
    od
    CLIENT[id] ! result (x)
od
}

```

CLIENT-ul trebuie să conțină instrucțiunile

```

...CMMDC ! arg (i,v1, v2); CMMDC ? result (r);...

```

În acest exemplu, procesul CMMDC acceptă argumente de intrare de la orice proces client (CLIENT[*]) și trimite răspunsul către clientul al cărui nume este primul argument (**id**) inclus în mesajul primit pe portul **est**.

Primitivele de intrare/ieșire pot fi extinse la instrucțiuni de **comunicare cu gardă**, care au forma:

B; C -> S;

unde B este o expresie booleană opțională

C este o primitivă de comunicare opțională, iar

S este o listă de instrucțiuni.

Dacă B este omisă, ea are valoarea implicită *true*. Dacă C este omisă, expresia se reduce la o comandă cu gardă, uzuală.

Împreună B și C alcătuiesc **garda**. Garda este rezolvată cu succes dacă B este *true* și execuția lui C se poate face imediat (este în așteptare o instrucțiune de comunicare pereche). În acest caz, se execută mai întâi comanda de comunicare C și apoi secvența S. Pe de altă parte, dacă B este *false* garda nu este reușită. În fine, garda se blochează dacă B este *true*, dar C nu poate fi executată imediat.

Comunicarea cu gardă este utilizată în instrucțiunile **if** și **do**. Astfel, la un **if**, dacă cel puțin o gardă se rezolvă cu succes, una din secvențele asociate este luată la îndeplinire și executată. Dacă toate gărzile sunt nereușite atunci **if** se termină. Dacă nici o gardă

nu este reușită, dar anumite gărzi sunt blocate, atunci execuția se întârzie pînă cînd una reușește. Similar se execută și **do**.

În gărzi pot apare atît instrucțiuni de intrare, cît și instrucțiuni de ieșire.

Exemple.

1) **Filtru**. Transmitere cu înlocuire. Transmite către procesul **est** caracterele produse de procesul **vest**. Înlocuiește fiecare pereche de asteriscuri ****** cu caracterul săgeată **^**. Prin convenție, ultimul caracter transmis de **vest** nu este un asterisc.

```
process INLOCUIRE {  
    do char c; vest?c ->  
        if c<>asterisc -> est!c  
        [] c=asterisc -> vest?c;  
        if c<>asterisc -> est!asterisc; est!c  
        [] c=asterisc -> est!sageata  
        fi  
    fi  
od
```

2) **Comunicare printr-un tampon limitat**.

```
process Copy { char buffer [1:10];  
    int front = 1, rear = 1, count = 0;  
    do count<10; vest ? buffer[rear] ->  
        count = count+1; rear = (rear mod 10) +1  
    [] count > 0; est ! buffer [front] ->  
        count = count -1; front = (front mod 10) +1  
    od
```

Procesul are două gărzi. Prima este reușită dacă există loc în tampon și **vest** este gata să trimită un caracter. A doua reușește cînd există caractere în tampon și **est** este pregătit să accepte un caracter. De remarcat că instrucțiunea **do** nu se termină niciodată, deoarece cel puțin una din condițiile booleene din gărzi este îndeplinită.

7. Execuția programelor distribuite

În cazul programelor distribuite, **timpul total de execuție** se definește ca timpul scurs

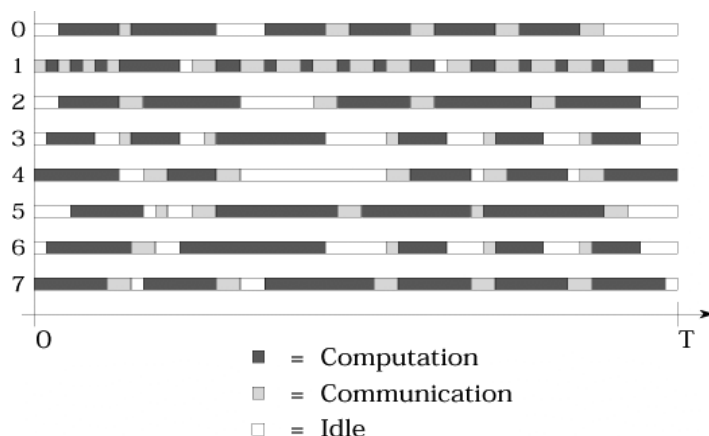


Figura 3.8 (Foster 1995)

din momentul când primul procesor începe execuția sa (mai precis a unuia din procesele programului distribuit) până în momentul în care ultimul procesor termină execuția. În evoluția unui proces se disting mai multe perioade: de calcule, de comunicare și de inactivitate, astfel că timpul total de execuție se exprimă ca suma timpilor de calcul (T_{comp}), de comunicare (T_{commun}) și de inactivitate (T_{idle}) raportată la numărul de procesoare (P):

$$T = (1/P) (T_{\text{comp}} + T_{\text{commun}} + T_{\text{idle}})$$

sau

$$T = (1/P) (\sum_{i=0, P-1} T_{\text{comp}}^i + \sum_{i=0, P-1} T_{\text{commun}}^i + \sum_{i=0, P-1} T_{\text{idle}}^i)$$

unde:

T_{comp}^i = timpul total de calcul pe procesorul i

T_{commun}^i = timpul total de comunicare al procesorului i

T_{idle}^i = timpul total de inactivitate al procesorului i .

În calculul analitic al timpului total de execuție, se neglijează, de regulă, **timpul de inactivitate**.

Timpul de calcul depinde de dimensiunea problemei, N , numărul de taskuri sau de procesoare și de caracteristicile procesoarelor și memoriei.

Timpul de comunicare este timpul consumat pentru transmiterea și pentru recepția mesajelor. Aceștia diferă de la un tip de comunicație la altul; astfel, dacă două procese sunt executate de procesoare diferite, comunicarea între ele are o durată mai mare (comunicarea între procesoare consumă mai mult timp); comunicarea între două procese executate de același procesor este mai eficientă (consumă mai puțin timp). Pentru simplificare, se consideră că cele două tipuri de comunicări au durate comparabile și se utilizează o formulă comună de calcul. Timpul de comunicare a unui mesaj (transmitere sau recepție) are două componente:

t_s - corespunde timpului consumat de operațiile de pregătire a comunicării și este independent de lungimea mesajului;

t_w - corespunde timpului de comunicare a unui cuvânt din mesaj.

Cu acestea, timpul de comunicare a unui mesaj de lungime L devine:

$$T_{\text{msg}} = t_s + t_w L$$

Formula este confirmată experimental (Foster 1995) prin măsurarea duratei transmisiei dus-întors a unui mesaj (round-trip time).

Într-un model de cost mai detaliat, se poate lua în considerare competiția între procese pentru comunicarea pe un același canal. Evident această competiție lungește timpul de comunicare. Dacă S procese vor să utilizeze în același timp un canal, durata comunicării efective se mărește de S ori (evident, timpul de pregătire nu este afectat):

$$T_{\text{msg-b}} = t_s + t_w S L$$

Valoarea lui S diferă de 1 doar dacă cele S procese transmit în același sens pe un canal, așa cum se arată în Figura 3.9: în cazul (b), P_0 și P_1 partajează canalul (P_1, P_2), în timp ce în cazul (c), P_0 și P_3 transmit în sensuri diferite pe acest canal și se consideră că transmisiile nu interferă.

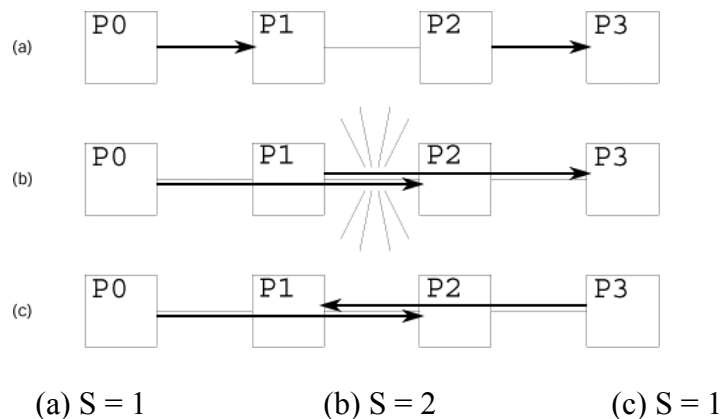


Figura 3.9 (Foster 1995)

Ca aplicație, considerăm **algoritmul lui Floyd** pentru calculul drumurilor celor mai scurte într-un graf reprezentat prin matricea de adiacențe I , a cărei variantă secvențială este următoarea:

```

for [k = 0 to N-1]
  for [i = 0 to N-1]
    for [j = 0 to N-1]
       $I[i,j]_{k+1} = \min(I[i,j]_k, I[i,k]_k + I[k,j]_k);$ 

```

Dacă t_c este durata unei iterații în algoritmul precedent, **timpul total de execuție** al algoritmului secvențial este:

$$T_{\text{secv}} = t_c N^3$$

Vom analiza două variante distribuite, ambele folosind P procesoare. În **prima variantă**, se face o descompunere uni-dimensională a matricei I în P grupuri de linii (fiecărui procesor i se asociază N/P linii). Algoritmul poate folosi cel mult N procesoare, deci întotdeauna numărul de procesoare este mai mic decât dimensiunea matricei, $P < N$. Fiecare procesor păstrează local una sau mai multe linii din I și este responsabil de calculul valorilor elementelor din liniile respective. Dacă procesorul p păstrează liniile de la p_i_start la p_i_end atunci codul său se poate descrie în felul următor:

```

for [k = 0 to N-1]

```

```

for [i = local_i_start to local_i_end]
  for [j = 0 to N-1]
     $I[i,j]_{k+1} = \min(I[i,j]_k, I[i,k]_k + I[k,j]_k);$ 

```

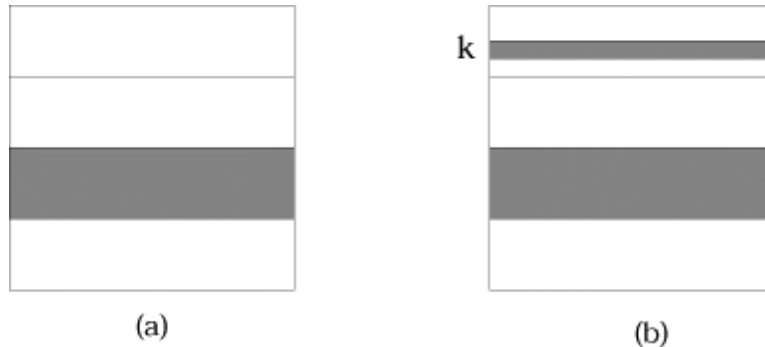


Figura 3.10 (Foster 1995)

În pasul k , procesoarele au nevoie de linia k a matricei I_k (Figura 3.10). Procesorul care păstrează această linie o comunică în $\log P$ pași folosind algoritmul de difuzare, înainte ca fiecare din celelalte procesoare să înceapă calculul liniilor matricei I_{k+1} de care răspund. Deoarece fiecare mesaj are N cuvinte, timpul de difuzare a unei linii este:

$$\log P (t_s + t_w N).$$

Timpul total de execuție al acestei variante distribuite este:

$$T_{\text{dist-1}} = t_c N^3/P + N \log P (t_s + t_w N)$$

El este compus din timpul de calcul $t_c N^3/P$, care este timpul algoritmului secvențial împărțit la numărul de procesoare plus timpul de comunicare $N \log P (t_s + t_w N)$, care este de N ori timpul de difuzare a unei linii.

În **a doua variantă**, facem o descompunere bi-dimensională, pe linii și coloane: fiecare procesor păstrează local elementele aflate pe una sau mai multe linii și simultan pe una sau mai multe coloane din I și este responsabil de calculul valorilor elementelor din liniile și coloanele respective. Evident, numărul de procesoare P nu poate fi mai mare de N^2 . Cele P procesoare se dispun într-o matrice de $\sqrt{P}$ linii pe $\sqrt{P}$ coloane. Dacă procesorul p păstrează elementele aflate pe liniile de la p_i_start la p_i_end și simultan pe coloanele de la p_j_start la p_j_end atunci codul său se poate descrie în felul următor:

```

for [k = 0 to N-1]
  for [i = local_i_start to local_i_end]
    for [j = local_j_start to local_j_end]
       $I[i,j]_{k+1} = \min(I[i,j]_k, I[i,k]_k + I[k,j]_k);$ 

```

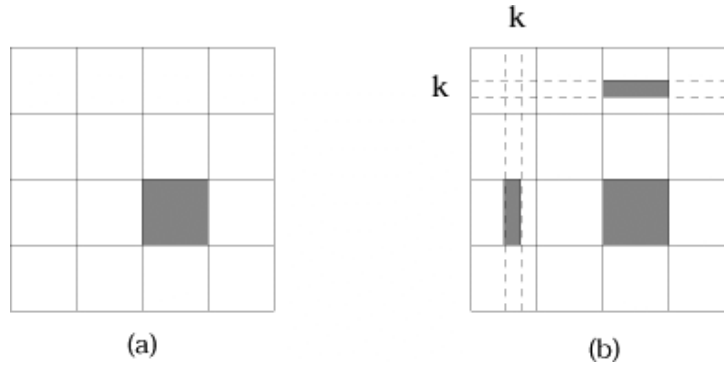


Figura 3.11 (Foster 1995)

În fiecare pas k , fiecare proces are nevoie de datele sale locale și de câte $N/\sqrt{P}$ valori de la alte două procese care păstrează:

- elementele din linia k și coloanele de la p_{j_start} la p_{j_end}
- elementele din coloana k și liniile de la p_{i_start} la p_{i_end} .

Fiecare din cele două mesaje are $N/\sqrt{P}$ elemente, astfel încât difuzarea unuia durează

$$\log \sqrt{P} (t_s + t_w N/\sqrt{P})$$

deoarece difuzarea se face unui număr de $\sqrt{P}$ procese.

În cei N pași, operația se repetă de $2N$ ori. Ca urmare, timpul total de execuție:

$$\begin{aligned}
T_{\text{dist-2}} &= t_c N^3/P + 2N \log \sqrt{P} (t_s + t_w N/\sqrt{P}) \\
&= t_c N^3/P + N \log P (t_s + t_w N/\sqrt{P}).
\end{aligned}$$

3.9. Modelul LogP

Un model mai adecvat studiului performanțelor pe mașini reale este **LogP** (Figura 3.12).

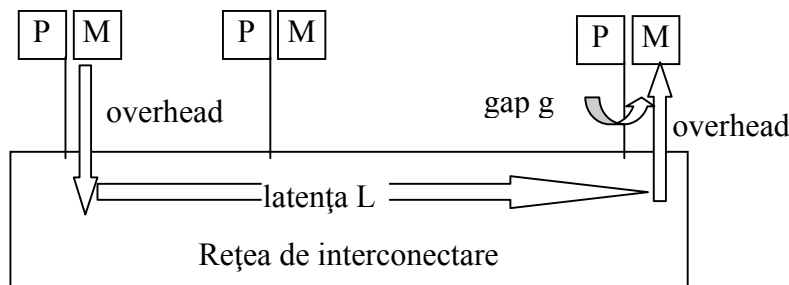


Figura 3.12 Modelul LogP

Modelul consideră P procesoare, fiecare cu memoria sa locală, legate printr-o rețea de interconectare. Parametrii modelului sunt:

- L – latența, sau întârzierea de transmitere a unui mesaj conținând un număr mic de cuvinte, de la procesorul / memoria sursă la destinatar
- o – overhead, durata pentru care procesorul este angajat în transmiterea sau recepția fiecărui mesaj; în timpul acesta procesorul nu poate face alte operații
- g – gap, intervalul minim de timp între două transmișii succesive sau două recepții succesive la același nod
- P – numărul de module procesor / memorie.

Rețeaua are o capacitate limitată, astfel că cel mult $\sup(L/g)$ mesaje pot fi în tranzit la un moment dat, de la orice procesor sau la oricare procesor. Cea mai simplă operație de comunicare, transmiterea unui singur pachet de la o mașină la alta, cere un timp de $L+2o$. O operație cerere/răspuns, precum o citire sau o scriere blocantă, ia un timp de $2L+4o$. Fiecare procesor este implicat un timp de $2o$, restul timpului putând fi utilizat pentru calcul, sau pentru alte operații de intrare/ieșire.

Ca aplicație considerăm **difuzarea unei valori**. Conform modelului PRAM, transmiterea are loc conform tiparului următor:

$P_0 \rightarrow P_1$	de la P_0 la P_1
$P_0, P_1 \rightarrow P_2, P_3$	de la P_0 și P_1 la P_2 și P_3
$P_0, P_1, P_2, P_3 \rightarrow P_4, P_5, P_6, P_7$	...

Presupunând $g \geq 0$ și luând $P=8$, $o=2$, $g=4$, $L=6$, obținem din calcul un timp total de difuzare egal cu 30 de unități (Figura 3.13).

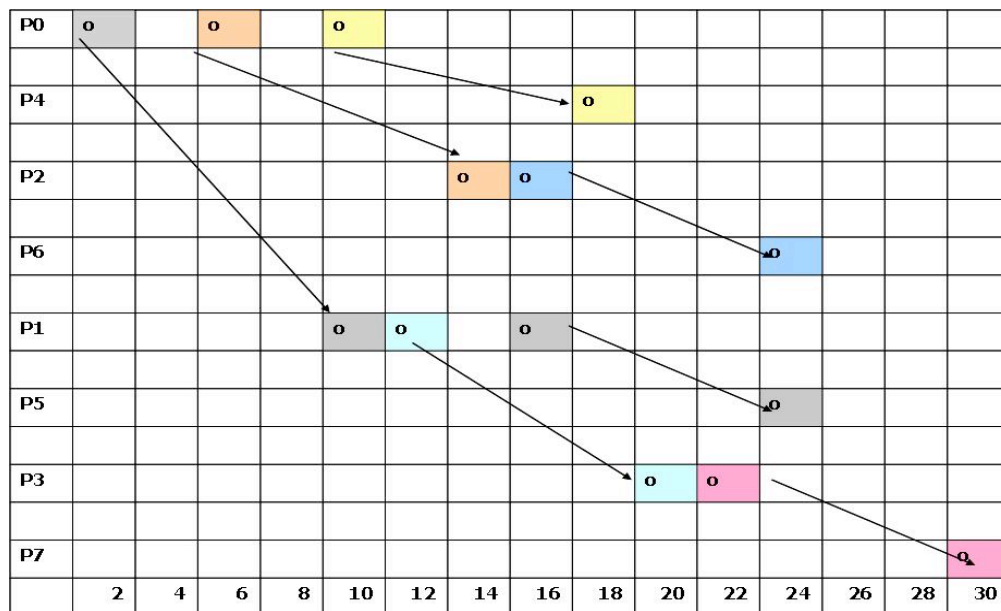


Figura 3.13. Schema de difuzare pentru tiparul PRAM

Pentru modelul LogP, adoptăm convenția că **orice procesor care primește o valoare, o transmite imediat altor procesoare**, cu condiția ca nici un procesor să nu primească mai mult de o valoare. Graful corespunzător variantei optime este cel din figura 3.14. Timpul corespunzător este de 24 de unități (Figura 3.15).

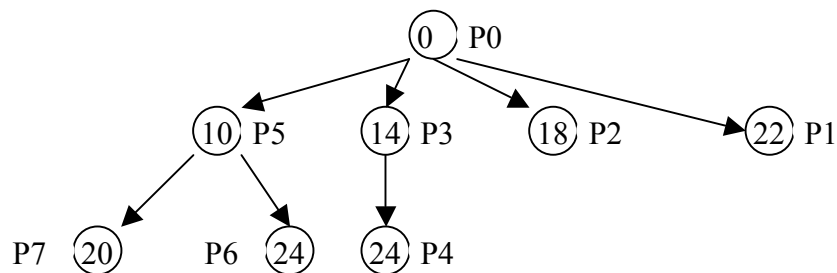


Figura 3.14. Graful variantei optime

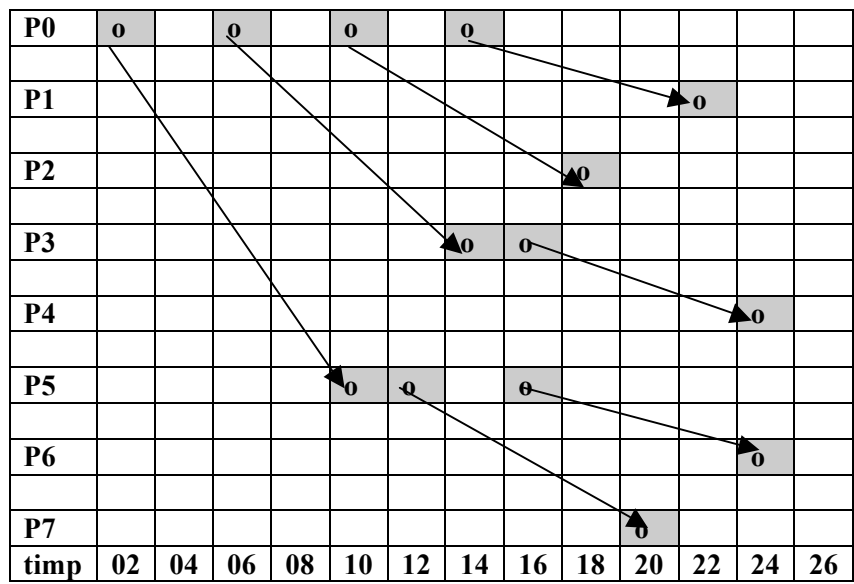


Figura 3.15. Schema de difuzare pentru varianta optimă

8. Ceasuri logice și ordonarea evenimentelor

8.1 Soluția lui Lamport

În vorbirea curentă se afirmă că un eveniment **a** s-a produs înaintea altui eveniment **b** dacă momentul de timp la care s-a produs **a** precede momentul producerii lui **b**. Pentru algoritmi, un eveniment poate fi execuția unei acțiuni (operații). În cazul execuției unui algoritm secvențial, ordinea evenimentelor poate fi stabilită ușor, operațiile algoritmului fiind executate secvențial. La execuția unui algoritm distribuit, ordinea evenimentelor este mai greu de stabilit: deși fiecare proces execută acțiunile secvențial, execuția paralelă a proceselor face ca evenimentele corespunzătoare diverselor procese să fie întretesute în timp. Cu toate acestea, ordinea evenimentelor este uneori importantă. De exemplu, o politică echitabilă pentru intrarea proceselor în secțiunea critică ar fi ordinea în care s-au făcut cererile. Ordonarea evenimentelor la sistemele distribuite întâmpină dificultăți, deoarece reclamă ca sistemele de calcul să conțină ceasuri de timp real, iar ceasurile să măsoare cu precizie timpul. O soluție mai realistă este aceea a introducerii unor **ceasuri logice**, care să marcheze ordinea evenimentelor fără a da cu exactitate momentele producerii lor. În acest scop, se definește relația **petrecut înainte** pe mulțimea evenimentelor dintr-un sistem, notată cu $\rightarrow$, ca fiind cea mai slabă relație care satisface următoarele trei condiții:

- în situația că **a** și **b** sunt evenimente din același proces și **a** îl precede pe **b**, atunci $a \rightarrow b$;
- când **a** reprezintă transmiterea unui mesaj de către un proces, iar **b** recepția aceluiași mesaj de către un altul, atunci $a \rightarrow b$;
- dacă $a \rightarrow b$ și $b \rightarrow c$ atunci $a \rightarrow c$.

Să considerăm în continuare o modalitate simplă de a introduce ceasuri logice într-un program paralel. Pentru aceasta, în loc să se atașeze unui eveniment timpul real de producere, i se asociază un număr de ordine, care să precizeze poziția sa în raport cu celelalte evenimente. Mai precis, pentru fiecare proces P_i , se definește ceasul logic ca o funcție C_i care asociază fiecărui eveniment **a** din P_i un număr unic $C_i(a)$. Întregul sistem de ceasuri poate fi privit ca o funcție C definită pe mulțimea tuturor evenimentelor cu valori în mulțimea numerelor naturale, care asociază oricărui eveniment **a** un număr $C(a)=C_i(a)$, dacă **a** este un eveniment al procesului P_i . Un astfel de sistem de ceasuri este corect dacă, pentru orice evenimente **a** și **b**, $a \rightarrow b$ implică $C(a) < C(b)$.

Realizarea practică a unui sistem de ceasuri logice presupune folosirea pentru fiecare proces A a unei variabile **cl**, a cărei valoare se mărește la producerea evenimentelor și, pentru fiecare mesaj, a unui câmp care să indice momentul transmiterii sale. Ceasurile logice sunt incrementate conform următoarelor reguli:

- (1) când A transmite un mesaj, el incrementează **cl** cu 1 și apoi actualizează timpul **tt** asociat mesajului la valoarea lui **cl**;
- (2) când A primește un mesaj având asociat timpul **tt**, el actualizează **cl** la valoarea maximă dintre **cl** și **tt + 1** și apoi incrementează **cl** cu 1.

Folosind ceasurile logice, putem asocia un moment de timp fiecărui eveniment. Pentru **send** acesta este timpul înregistrat în mesaj. Pentru **receive** acesta este valoarea lui **cl** din procesul receptor după actualizarea la maximum dintre **cl** și **tt+1** și incrementarea ei. Aceasta asigură că, dacă un eveniment **a** se produce înaintea unui eveniment **b**, timpul asociat lui **a** să fie mai mic decât timpul asociat lui **b**. Se induce astfel o ordonare parțială pe mulțimea evenimentelor unui program.

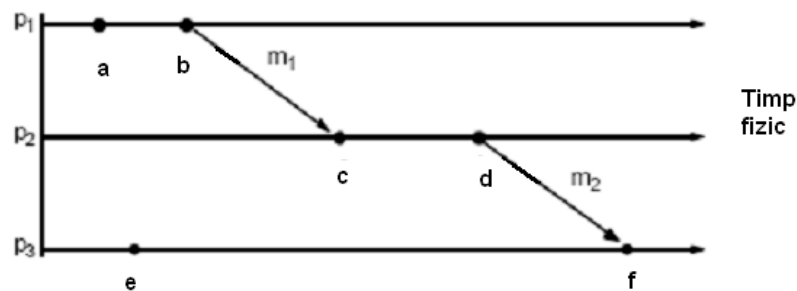


Figura 8.1 Trei procese și evenimentele corespunzătoare

În figura 8.1 sunt reprezentate evenimentele interne și de comunicare între trei procese cu respectarea momentelor de timp fizic la care s-au produs. Considerând că ceasurile logice asociate proceselor au inițial valoarea 0, prin actualizarea lor după regulile prezentate mai înainte se obțin valorile de timp logic prezentate în Figura 8.2.

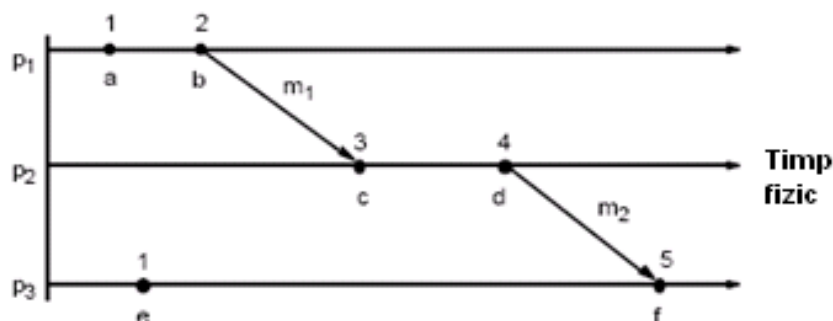


Figura 8.2 Valorile de timp logic asociate evenimentelor din Figura 8.1

Deoarece două evenimente din procese diferite pot avea același timp logic de producere (de exemplu, evenimentele *a* și *e* din Figura 8.2), pentru a obține o **ordonare totală** se poate recurge la ordonarea proceselor și la dispunerea evenimentelor cu același timp logic în ordinea proceselor corespunzătoare.

Ca aplicație, prezentăm utilizarea ordonării totale a evenimentelor în **realizarea excluderii mutuale** într-un sistem cu transmitere asincronă de mesaje, prin semafoare distribuite. Soluția are la baza următorul mecanism: când un proces execută o operație *P* (sau *V*) el difuzează mesaje celorlalte procese, analizând răspunsurile acestora pentru a determina continuarea execuției. Folosim forma:

broadcast *ch*[*m*]

pentru a desemna transmiterea mesajului *m* pe fiecare din canalele unui tablou *ch*[1:*n*]. Toate cele *n* replici ale lui *m* transmise prin **broadcast** poartă aceeași amprentă de timp, calculată la fel ca pentru o singură operație **send**.

Implementarea semafoarelor se face prin două colecții de procese. Procesele **Utiliz**[*i*] inițiază operațiile *P* sau *V*, prin difuzare de mesaje pe canalele **opsem**. Procesele **Ajutor**[*i*] implementează operațiile *P* și *V*.

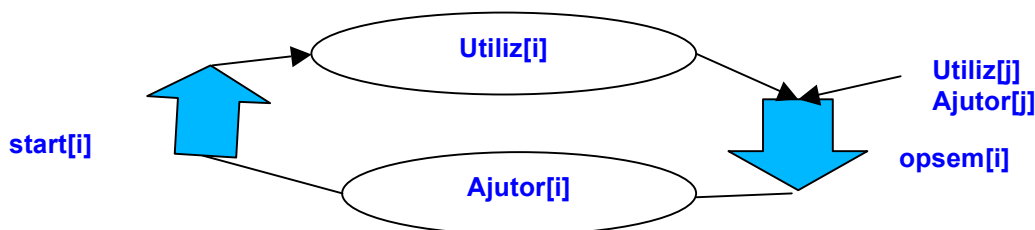


Figura 8.3

La o operație P sau V, un proces **Utiliz** difuzează un mesaj tuturor proceselor (inclusiv propriului) **Ajutor**. Mesajul conține identitatea propriului ajutor, un indicator P sau V și un moment de timp, același în toate copiile mesajului.

Fiecare proces **Ajutor** are o coadă locală de mesaje **qm** și un ceas logic **cl**. Când un proces **Ajutor** primește un mesaj P sau V, el îl memorează în **qm**, care este ordonată după momentele de timp conținute de mesaje (cu amendamentul suplimentar asupra ordonării proceselor). Mesajele unor procese diferite nu sunt recepționate în aceeași ordine de toate procesele **Ajutor**. Mai mult, un mesaj cu un timp mai redus poate fi recepționat după un altul cu un timp mai mare. Dar, mesajele trimise de un același proces vor fi recepționate de celelalte procese în ordinea generării și vor avea momente de timp crescătoare. Ca urmare, dacă **qm** conține un mesaj **m** cu timpul **ts** și procesul recepționează mesaje cu un timp mai mare de la toate celelalte procese, este sigur că nu va mai apare ulterior un mesaj cu un timp mai mic. În acest moment **m** este **complet confirmat** și, odată cu el, toate celelalte mesaje aflate în fața lui în **qm**. Acestea constituie un **prefix stabil**, nici un alt mesaj neputând fi inserat între cele din prefix.

Totodată, când un proces **Ajutor** primește un mesaj P sau V, acesta **difuzează** o confirmare **ack**. Mesajele **ack** au asociați timpi de producere, dar ele nu sunt memorate la recepție în **qm** și nu sunt confirmate la rândul lor. Ele folosesc doar la a determina când un mesaj P sau V devine complet confirmat.

Fiecare proces **Ajutor** simulează operațiile cu semafoare. El folosește o variabilă locală **sem**, pentru a păstra valoarea semaforului. Când procesul primește un **ack**, el actualizează **prefixul stabil** din **qm**. Pentru fiecare mesaj V din prefix, procesul incrementează **sem** și șterge mesajul. Procesul examinează apoi mesajele P. Dacă $sem > 0$, procesul decrementează **sem** și șterge mesajul P. Mesajele P sunt tratate de toate procesele în aceeași ordine.

Algoritmul este următorul:

```
enum fel {V, P, ack};
chan opsem[1:n](int transm, fel op, int timp);
chan start[1:n](int timp);

process Utiliz [i = 1 to n]{
  int cl = 0;                /*ceas logic*/
  int ts;                    /*timestamp mesaj*/
```

```

...
cl = cl+1;
broadcast opsem(i, P, cl);      /*operatia P*/
receive start[i](ts);          /*asteapta permisiune*/
cl = max(cl, ts) + 1;
    sectiune critica;
cl = cl+1;
broadcast opsem(i, V, cl);      /*operatia V*/
    sectiune ne-critica;
...
}

process Ajutor [i = 1 to n]{
typedef struct {int transm, fel k, int ts} qelem;
queue qm [lmax];      /*coada ordonata dupa timestamp tm*/
int cl = 0;
int sem = valoare_initiala;
int transm;
fel k;
int ts;

while (true) {
    receive opsem[i](transm, k, ts);
    cl = max(cl, ts) + 1;
    if (k==P or k==V) {
        insereaza (transm,k,ts) in loc corespunzator in qm;
        cl = cl+1; broadcast opsem(i, ack, cl);}
    else if (k==ack) {
        inregistreaza primirea unui ack;
        for [mesajele V complet confirmate ] {
            scoate mesaj din qm;
            sem = sem+1;}
        for [mesajele P complet confirmate st sem>0] {
            scoate mesaj (transm, k, ts) din qm;
            sem = sem-1;
            if (transm=i) {cl := cl+1; send start[i](cl); }
        }
    }
}      /*end process*/

```

Tema. Alcătuiți algoritmul distribuit de excludere mutuală care funcționează în modul descris în continuare.

Un proces care dorește să intre în secțiunea critică trimite mesaje de cerere **request** tuturor celorlalte procese. Pentru a putea intra efectiv în secțiunea critică, este necesar să primească de la fiecare câte un mesaj de răspuns **reply**. La recepția unui mesaj **request**, un proces poate determina dacă el sau procesul care a făcut cererea ar trebui să intre în secțiunea critică. Când el are prioritate, mesajul **reply** este întârziat; altfel, el este transmis imediat procesului ce a generat cererea.

8.2 Ceasuri logice vectoriale

Cu soluția ceasurilor logice dată de Lamport, avem următoarea relație:

$e \text{ precede } f \Rightarrow \text{amprenta_logică}(e) < \text{amprenta_logică}(f)$

deci, dacă **e** precede **f** atunci timpul logic asociat lui **e** este mai mic decât cel al lui **f**. Pe de altă parte,

$\text{amprenta_logică}(e) < \text{amprenta_logică}(f) \nRightarrow e \text{ precede } f$

adică, faptul că timpul logic al lui **e** este mai mic decât cel al lui **f** nu înseamnă neapărat că **e** precede **f**. De exemplu, în Figura 8.2, evenimentul **e** are timpul logic 1 iar **c** are timpul logic 3, dar nu se poate spune ca **e** precede **c**.

Prin utilizarea **ceasurilor logice vectoriale**, o astfel de implicație este garantată. Ceasurile logice vectoriale sunt definite în modul descris în continuare. Fiecare proces P_i are asociat un vector $V_i[1..n]$, reprezentând ceasul său logic vectorial, în care:

$V_i[i]$ este numărul de evenimente produse în procesul $P[i]$;

$V_i[j]$ este numărul de evenimente despre care $P[i]$ știe (a aflat) că au avut loc la $P[j]$.

Obs. V nu este o matrice, ci o colecție de vectori, fiecare V_i din această colecție fiind păstrat de procesul $P[i]$ în memoria sa locală.

Procesul $P[i]$ actualizează V_i la fiecare eveniment din $P[i]$ (trimitere / recepție de mesaj sau alt eveniment). Când $P[i]$ **transmite** mesajul m (eveniment "send m "):

$P[i]$ incrementează $V_i[i]$

$P[i]$ adaugă V_i la m ca vector de amprente de timp curent $vt(m)$

Când $P[j]$ **primește** mesajul m împreună cu $vt(m)$ (eveniment "receive m "):

$P[j]$ ajustează vectorul: $V_j[k] = \max\{V_j[k], vt(m)[k]\}$ pentru fiecare k

$P[j]$ incrementează $Vj[j]$ cu 1

Figura 8.4 arată rezultatul aplicării acestor reguli pe exemplul din Figura 8.2.

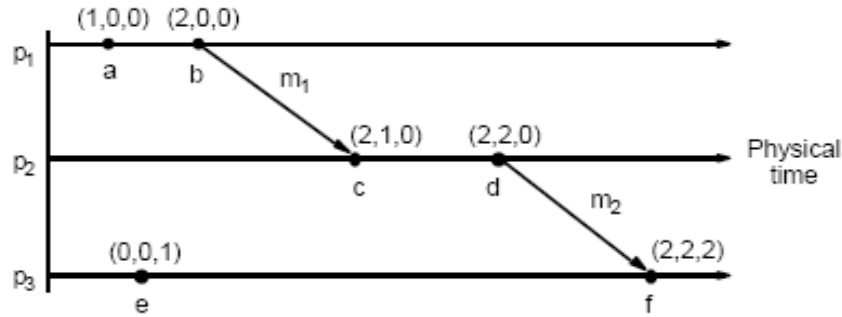


Figura 8.4 Aplicarea regulilor ceasurilor logice vectoriale

Pe multimea vectorilor de amprente de timp, VT pot fi stabilite câteva relații. Fie VT1 și VT2 doi vectori VT cu câte N elemente. Atunci,

- $VT1 = VT2 \Leftrightarrow VT1[i] = VT2[i]$, pentru $i = 1, \dots, N$
- $VT1 \leq VT2 \Leftrightarrow VT1[i] \leq VT2[i]$, pentru $i = 1, \dots, N$
- $VT1 < VT2 \Leftrightarrow VT1 \leq VT2$ și $VT1 \neq VT2$

Relațiile anterioare ne permit definirea ordonării cauzale a evenimentelor. Fie $vt(a)$ și $vt(b)$ vectorii de amprente de timp asociați evenimentelor **a** și **b**. Atunci:

- $vt(a) < vt(b) \Rightarrow$ evenimentul **a** precede cauzal evenimentul **b**
- $vt(a) \not< vt(b)$ and $vt(b) \not< vt(a)$ and $vt(a) \neq vt(b) \Leftrightarrow$ evenimentele **a** și **b** sunt concurente

Aplicație: Ordonare Cauzală Multicast

Presupunem că procesele unei colecții P comunică între ele doar prin mesaje cu difuzare (orice mesaj este trimis tuturor proceselor din colecție). Se cere ca oricare două mesaje **m** și **m'** transmise în cadrul colecției să respecte **dependența cauzală**, ceea ce înseamnă că dacă mesajul **m** a fost trimis înaintea mesajului **m'** atunci, la orice proces **p** din colecție, livrarea lui **m** să aibă loc înaintea livrării lui **m'**:

$$vt(m) < vt(m') \Rightarrow \text{livrare}_p(m) \rightarrow \text{livrare}_p(m').$$

Pentru asta, fiecare proces folosește un protocol care întârzie eventual livrarea mesajelor astfel încât aceasta să aibă loc în ordinea cauzală. Protocolul se bazează pe o variantă ușor modificată a ceasurilor logice vectoriale, pe care o numim **vectori de timp** pentru a face diferențierea față de varianta de bază. În noua variantă, ceasurile sunt avansate doar la operațiile de transmitere de mesaje.

Ca și în varianta inițială, fiecare proces $P[i = 1..n]$ are asociat un vector $V_i [1..n]$, cu toate elementele având inițial valoarea 0, reprezentând vectorul său de timp, în care:

$V_i [i]$ este numărul de evenimente de transmitere de mesaje produse în procesul $P[i]$;

$V_i [j]$ este numărul de evenimente de transmitere de mesaje despre care $P[i]$ știe (a aflat) că au avut loc la $P[j]$.

Procesul $P[i]$ actualizează V_i la fiecare eveniment de trimitere sau recepție de mesaj din $P[i]$.

Când un proces sursă $P[s]$ **transmite** mesajul m (eveniment "send m "):

$P[s]$ incrementează $V_s[s]$

$P[s]$ adaugă V_s la m ca vector de amprente de timp curent $vt(m)$.

Amprenta $vt(m)$ spune receptorului câte evenimente (din alte procese) au precedat m și ar putea influența cauzal pe m .

Când procesul de destinație $P[d]$ primește mesajul m împreună cu $vt(m)$ (eveniment "receive m "), mesajul este **livrat** (făcut disponibil procesului pentru prelucrare) doar dacă nu se violează constrangerile de cauzalitate. Mai precis, mesajul este păstrat într-o coadă de întârziere până când sunt îndeplinite următoarele două condiții:

$$vt(m)[s] = V_d[s] + 1$$

(m este următorul mesaj pe care d îl aștepta de la s);

$$vt(m)[k] \leq V_d[k] \text{ pentru } k \neq s$$

(toate mesajele primite deja de $P[s]$ când a trimis m au fost primite și de $P[d]$ când acesta a primit m).

Când mesajul este livrat, V_d este actualizat conform regulilor vectorilor de timp:

$$V_d[k] = \max\{V_d[k], vt(m)[k]\} \text{ pentru fiecare } k = 1..n.$$

Un exemplu de întârziere a mesajelor este prezentat în figura 8.5. Mesajul cu vectorul de timp $(1,1,0)$ este livrat după mesajul $(1,0,0)$ deși este primit înaintea acestuia de procesul P3.

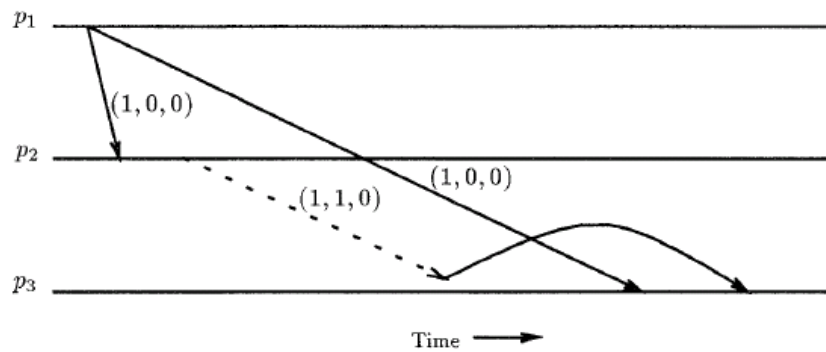


Figura 8.5 Întârzierea unui mesaj (Birman 1991)

Livrarea mesajului m poate determina livrarea altor mesaje, primite anterior, care acum îndeplinesc condiția de cauzalitate.

Acțiunile protocolului la livrarea mesajelor pot fi rezumate astfel:

- dacă $vt(m)[k] > Vd[k]$ pentru un oarecare k atunci se întârzie m
 Motivul este că $P[s]$ a primit mesaje de care mesajul curent poate fi cauzal dependent, dar pe care $P[d]$ încă nu le-a primit;
- dacă $vt(m)[s] > V(d)[s]+1$ atunci se întârzie m
 Motivul este că mai sunt mesaje de la $P[s]$ pe care $P[d]$ nu le-a primit; aceasta asigură ordinea FIFO pentru canale care sunt nonFIFO
- dacă $vt(m)[s] < Vd[s]$ atunci rejectează m
 Motivul: m este un duplicat al unui mesaj primit anterior.

9. Algoritmi undă

9.1 Noțiuni preliminare

9.1.1. Calcul

În capitolul 2 menționam că un **program concurent** constă dintr-o colecție de procese paralele comunicante, iar **execuția** sa poate fi privită ca o întrețesere a **acțiunilor atomice** ale proceselor. Efectul unei acțiuni este modificarea stării colecției de procese, numită convențional **configurație**. Ca urmare, o **execuție** a programului poate fi modelată printr-o succesiune de configurații.

Exemplul 1. Considerăm programul distribuit constând din (colecția a) două procese, cu următoarea descriere:

```
process P1 { int a =0;          process P2 { int b =0;
              a = a+1;}              b = b+1;}
```

Considerând configurația ca ansamblul valorilor celor două variabile pe care se aplică acțiunile proceselor, configurația inițială este $\langle 0,0 \rangle$. Dacă operația procesului P1 se execută mai întâi, configurația devine:

$\langle 1,0 \rangle$

Notăția folosită evidențiază valorile variabilelor **a** și **b** în această ordine. O execuție posibilă a programului poate fi modelată prin secvența de configurații:

$(\langle 0,0 \rangle, \langle 1,0 \rangle, \langle 1,1 \rangle)$

Programul concurent poate fi modelat ca un **sistem de tranziții**, care pune în evidență **configurațiile** colecției de procese, **tranzițiile** între configurații și configurația inițială:

$$S = (C, \rightarrow, I)$$

unde:

- C este o mulțime de configurații,
- $\rightarrow$ este relația de tranziție binară pe C,
- I este setul configurațiilor inițiale (o submulțime a lui C).

Un **sistem de tranziții** evidențiază mulțimea tuturor **configurațiilor** posibile ale sistemului (software), tranzițiile pe care sistemul le poate face între configurații și configurațiile din care sistemul poate porni.

Pentru exemplul dat, avem următoarele:

- mulțimea de configurații $C = (<0,0>, <0,1>, <1,0>, <1,1>)$
- relația de tranziție $\rightarrow$ este definită prin tranzițiile
 - $<0,0> \rightarrow <0,1>$
 - $<0,0> \rightarrow <1,0>$
 - $<0,1> \rightarrow <1,1>$
 - $<1,0> \rightarrow <1,1>$
- mulțimea configurațiilor inițiale este $I = \{<0,0>\}$.

O **execuție** a lui S este o **secvență maximală** de configurații $E = (\gamma_0, \gamma_1, \gamma_2, \dots)$, unde γ_0 aparține lui I și, pentru $i \geq 0$, există tranzițiile $\gamma_i \rightarrow \gamma_{i+1}$.

O secvență E este **maximală** dacă este infinită sau dacă sfârșește într-o configurație **terminală**. O configurație **terminală** γ nu are succesori (nu există δ astfel încât $\gamma \rightarrow \delta$).

O execuție, în exemplul dat, poate fi
 $E = (<0,0>, <0,1>, <1,1>)$
în care configurația $<1,1>$ este terminală.

O configurație δ este **tangibilă** din γ dacă există o secvență $\gamma_0, \gamma_1, \gamma_2, \dots, \gamma_k$ de configurații astfel încât

$$\begin{aligned}\gamma &= \gamma_0 \\ \delta &= \gamma_k \\ \gamma_i &\rightarrow \gamma_{i+1}, \text{ pentru } i = 1, k-1.\end{aligned}$$

Prin convenție, se utilizează termeni diferiți pentru elementele sistemului de tranziții și pentru procese. Astfel, o **configurație** corespunde reuniunii **stărilor** proceselor, iar o **tranziție** corespunde unui **eveniment** al unui **proces**. Evenimentele pot fi **interne**, de calcul, sau de comunicare, **send** sau **receive**.

Cu aceasta, unei **execuții** E îi corespunde o **secvență de evenimente** din diferite procese. Dacă două evenimente **depind** unul de altul (spunem că sunt într-o relație de **ordine cauzală** - unul din evenimente depinde de celălalt) ele vor apărea în aceeași ordine în toate execuțiile. Altfel, ordinea celor două evenimente poate diferi de la o execuție la alta.

Pentru o execuție E , **relația de ordine cauzală** $<$ între două evenimente **a** și **b** este cea mai slabă relație care satisface următoarele proprietăți:

- dacă **a** și **b** sunt două evenimente diferite ale aceluiași proces și **a** se produce înaintea lui **b** atunci **a < b**
- dacă **a** este un eveniment **send** iar **b** evenimentul **receive** corespunzător atunci **a < b**
- dacă **a < c** și **b < c** atunci **a < c** (< este tranzitivă).

Dacă nici **a < b** și nici **b < a** atunci **a** și **b** sunt **concurente**.

Exemplul 2. Considerăm două procese **P1** și **P2**, care au variabilele locale **a**, respectiv **b** și care comunică printr-un canal **Can**. Inițial **a=0** și **b=0**. Evenimentele constând din execuția acțiunilor proceselor sunt notate de la **ev1** la **ev4**.

chan Can (int);

process P1 { int a = 0;

 a = a+2; (ev1)

 send Can (a); } (ev2)

process P2 { int b = 0;

 b = b+1; (ev3)

 receive Can (b); } (ev4)

Evenimentul **ev4** depinde de **ev2** deoarece procesul **P2** nu poate executa acțiunea de recepție a unui mesaj decât după ce **P1** a trimis acel mesaj. Ca urmare, în orice execuție, ordinea evenimentelor este **ev2 < ev4**. În schimb, evenimentele **ev1** și **ev3** pot fi executate în orice ordine de cele două procese, fiind astfel concurente. Ordinea execuției acestor evenimente poate fi **ev1 < ev3** sau **ev3 < ev1**.

În exemplul 1, execuțiile următoare, notate **E** și **F** au același rezultat:

E: (<0,0> , <0,1> , <1,1>) operația b = b+1 prima

F: (<0,0> , <1,0> , <1,1>) operația a = a+1 prima

Evenimentele ce corespund celor două operații de adunare sunt concurente și se pot executa în orice ordine.

Evenimentele unei execuții **E** se pot produce în orice altă ordine consistentă cu relația de cauzalitate, fără a afecta rezultatul execuției. Noua ordine a evenimentelor produce o altă secvență de configurații, **F** despre care spunem că este **echivalentă** cu prima. Mai precis, o execuție **E** este **echivalentă** cu **F** (se notează **E ~ F**) dacă:

- au aceeași colecție de evenimente (ordinea diferă)
- evenimentele respectă aceeași relație de ordine cauzală
- ultima configurație a lui **E** coincide cu ultima configurație a lui **F**.

Obs. Două execuții echivalente pot să nu aibă aceleași configurații.

De exemplu, execuțiile E și F sunt echivalente: ultimele configurații ale celor două execuții coincid, dar execuțiile nu au aceleași configurații:

E: $\langle 0,0 \rangle, \langle 0,1 \rangle, \langle 1,1 \rangle$

F: $\langle 0,0 \rangle, \langle 1,0 \rangle, \langle 1,1 \rangle$

Folosind relația " $\sim$ " putem împărți execuțiile unui sistem de tranziții în clase de echivalență. Vom numi **calcul** (*computation*) al unui algoritm distribuit, o **clasă de echivalență** (sub relația $\sim$) a execuțiilor algoritmului.

Cu alte cuvinte, un calcul este o colecție de evenimente parțial ordonată prin relația de ordine cauzală. Numărul de evenimente dintr-un calcul C este notat $|C|$, iar submulțimea evenimentelor produse în procesul p este notată C_p .

9.2. Algoritmi undă (Wave algorithms)

Mulți algoritmi distribuiți se bazează pe transmiterea de mesaje în conformitate cu o anumită **schemă**, dependentă de topologie, care asigură participarea tuturor proceselor. Ca **exemple** menționăm algoritmi pentru: difuzarea informației, realizarea unei sincronizări globale între procese, declanșarea unui eveniment în fiecare proces, calculul unei funcții în care fiecare proces participă cu o parte a datelor de intrare. **Algoritmi undă** implementează astfel de scheme de transmitere de mesaje.

Un algoritm undă este un algoritm distribuit care are următoarele **proprietăți**:

- **terminare** – fiecare calcul este finit,
- **decizie** – fiecare calcul conține cel puțin un eveniment de decizie (*decide*),
- **dependență** – în fiecare calcul, fiecare *decide* este precedat cauzal de un eveniment din fiecare proces.

Pentru un astfel de algoritm, un **calcul** (*computation*) este denumit **undă** (*wave*). Se face distincție între două categorii de procese: cele care inițiază execuția, numite **inițiatoare** (*starters*) și celelalte procese, **ne-inițiatoare** (*followers*). Este ușor de înțeles că primul eveniment dintr-un proces inițiator este unul intern sau un *send*, în timp ce primul eveniment dintr-un proces ne-inițiator este un *receive*.

Algoritmi undă pot fi **clasificați** după mai multe criterii, prezentate succint în continuare.

Centralizarea. Într-un algoritm centralizat există un singur inițiator. Într-un algoritm descentralizat există un set arbitrar de inițiatori.

Topologia. Poate fi inel, arbore, clică, graf etc. Topologia este **fixă** atunci când nu se produc modificări topologice; ea este **conectată** dacă există o cale între oricare două procese. În general, topologia este nedirecționată (legăturile între procese sunt bi-direcționale).

Cunoștințele inițiale ale unui proces pot include identitatea proprie (nume), identitățile vecinilor, **sensul direcției** (sau ordinea vecinilor, prezentată într-un paragraf următor).

Numărul de decizii. Regula generală este "cel mult o decizie în fiecare proces." Putem avea una din următoarele situații: un singur proces decide; toate procesele decid; doar unele decid.

Complexitatea unui algoritm distribuit se măsoară în: număr de mesaje schimbate, număr de biți interschimbați; timpul necesar pentru un calcul. Pentru complexitatea în timp se aplică următoarele reguli:

- un proces poate executa orice număr finit de evenimente în timp nul;
- durata de la transmiterea la recepția unui mesaj este de cel mult o unitate de timp.

Sensul legăturilor

Complexitatea comunicării într-un algoritm distribuit depinde de topologie, dar și de cunoașterea ei la nivelul fiecărui nod. În notația folosită până acum, instrucțiunile de comunicare de mesaje făceau apel la identificatori globali ai canalelor utilizate, care nu includeau nici un fel de informație asupra topologiei globale a rețelei de procese. O alternativă utilă în cazul topologiilor regulate este utilizarea unui singur set de etichete pentru oricare proces, care ar corespunde **sensului legăturilor** cu procesele vecine sau **direcțiilor** către nodurile vecine. Dimensiunea setului depinde de topologie (de ex. 2 direcții pentru inel, 4 pentru tor, etc). Se adaugă o **condiție suplimentară de consistență** diferită de la o topologie la alta.

Topologia Inel. Pentru fiecare nod din inel (Figura 9.1a) sunt 2 direcții, denumite: **Prec** (precedent) și **Urm** (următor). Condiția de consistență este:

precedentul lui p este q $\Leftrightarrow$ următorul lui q este p

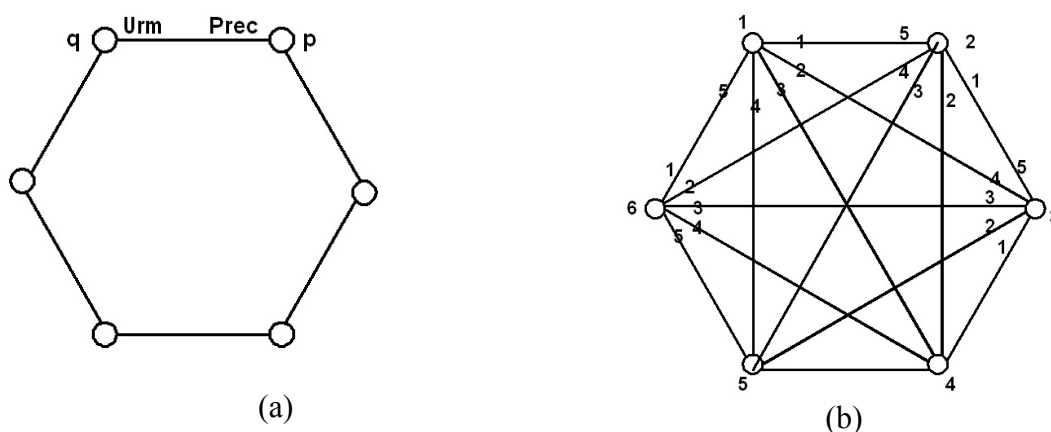


Figura 9.1 Un inel (a) și o clică (b)

Clica. Clica (Figura 9.1b) conține N noduri de grad N-1. Setul direcțiilor este $\{1, 2, \dots, N-1\}$, iar condiția de consistență este:

direcția de la nodul i la j are eticheta $(j-i) \bmod N$

Hipercub. Pentru un hipercub n dimensional (Figura 9.2), setul direcțiilor este $\{0, \dots, n-1\}$, iar condiția de consistență este:

doua noduri $(b_0, \dots, b_{n-1}), (c_0, \dots, c_{n-1})$ legate prin muchie cu eticheta i diferă doar în bitul i

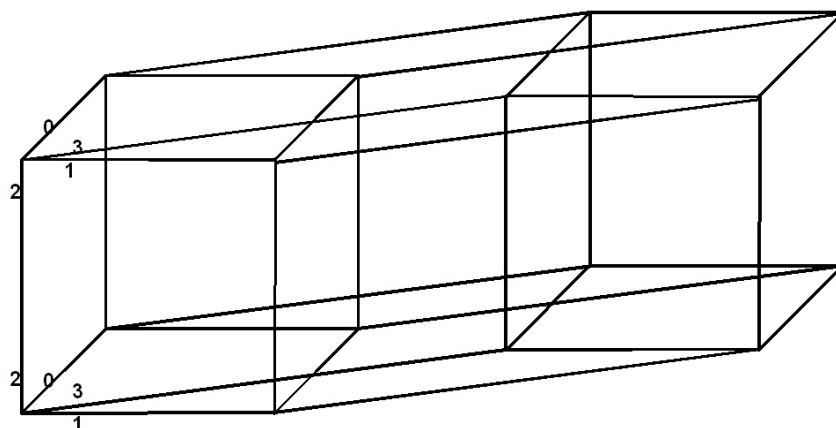


Figura 9.2 Hipercub

Cu acestea, notația pentru transmiterea unui **mesaj** poate avea două forme:

(1) transmitere prin adresare directă, folosită atunci când transmițătorul cunoaște identitatea unică, globală a canalului q către receptor

send ch[q] (mesaj)

(2) transmitere prin adresare indirectă, folosind etichete locale ale direcțiilor

send ch[direcție] (mesaj)

unde **direcție** identifică unul din canalele pe care procesul poate transmite.

9.3. Căția algoritmi undă

Prezentăm în continuare câțiva algoritmi undă elementari.

9.3.1. Algoritmul inel

Algoritmul are următoarele caracteristici:

- fiecare proces are un vecin dedicat, Urm
- transmiterea folosește adresarea prin direcție (Urm, Prec)
- toate canalele selectate prin Urm formează un ciclu Hamiltonian

- algoritmul este centralizat: inițiatorul trimite un token (jeton) care este pasat de fiecare proces de-a lungul ciclului până ajunge înapoi la inițiator; inițiatorul ia apoi decizia.

Descrierea este următoarea:

```
chan token[1:n] (tok_type tok);
```

```
/*initiator*/
```

```
process P[I]{
```

```
  tok_type tok;
```

```
  send token[Urm](tok);
```

```
  receive token[I](tok);
```

```
  decide
```

```
/*non-initiatori*/
```

```
process P[k=1 to n, k<>I]{
```

```
  tok_type tok;
```

```
  receive token[k](tok);
```

```
  send token[Urm](tok);
```

Număr de mesaje = n

Cele **n** procese pasează un token de la unul la altul; în total se transmit n mesaje.

Timp = n

Trimiterea (send) și recepția (receive) fiecărui mesaj durează o unitate de timp.

Obs. **Regula folosită pentru calculul complexității este următoarea:**

complexitatea în timp a unui algoritm distribuit este timpul maxim cerut de un calcul al algoritmului, în următoarele condiții:

- un proces poate executa în timp zero orice număr finit de evenimente;
- timpul între transmiterea unui mesaj și recepția lui este de (cel mult) o unitate de timp.

9.3.2. Algoritmul arbore

Algoritmul se aplică unei topologii arbore sau unei topologii arbitrare în care se cunoaște un arbore de acoperire.

Fiecare nod cunoaște identitatea proprie și identitățile vecinilor. **Inițiatorii** sunt toate nodurile frunză, fiecare având un singur vecin.

În **algoritm**:

- fiecare proces trimite exact un mesaj;
- când un proces a primit un mesaj pe fiecare canal incident mai puțin unul (condiție îndeplinită inițial de frunze) el trimite un mesaj pe canalul rămas;
- când un proces a primit câte un mesaj pe toate canalele sale atunci **decide**.

```
chan ch[1:n] (int id, tok_type tok);
    /* fiecare proces are un canal propriu */

process Proc[p = 1 to n]{
    int r = numar_vecini_p;
    bool Vecini[1:n] = vecinii_lui_p;
        /* Vecini[i]=true daca i este vecin lui p */
    bool rec[1:n] = ([n] false);
        /* rec[i]=true daca p a primit mesaj de la i */
    tok_type tok; int id, q0;

    while (r>1){receive ch[p](id,tok);
        rec[id] = true; r = r-1;}}
/* de la un singur vecin, q0, nu s-a primit mesaj */
afla q0 pentru care: Vecini[q0] and NOT rec[q0];
send ch[q0](p, tok);
receive ch[p](q0, tok); rec[q0] := true;
decide;
/* informeaza celelalte procese despre decizie */
/* for [q = 1 to n st Vecini[q] and q<>q0]
    send ch[q](p,tok); */
}
```

Algoritmul de bază se termină când unul din procese decide. În descriere sunt adăugate (in formă de comentariu) și instrucțiunile de transmitere a deciziei către vecini.

Număr de mesaje = n

Fiecare proces trimite un mesaj și sunt n procese.

Timp = $O(D)$

Mesajul trimis de o frunză atinge orice alt nod într-un număr de pași mai mic decât diametrul D al arborelui, fiecare pas fiind făcut într-o unitate de timp.

Un exemplu de execuție este prezentat în figura 9.3.

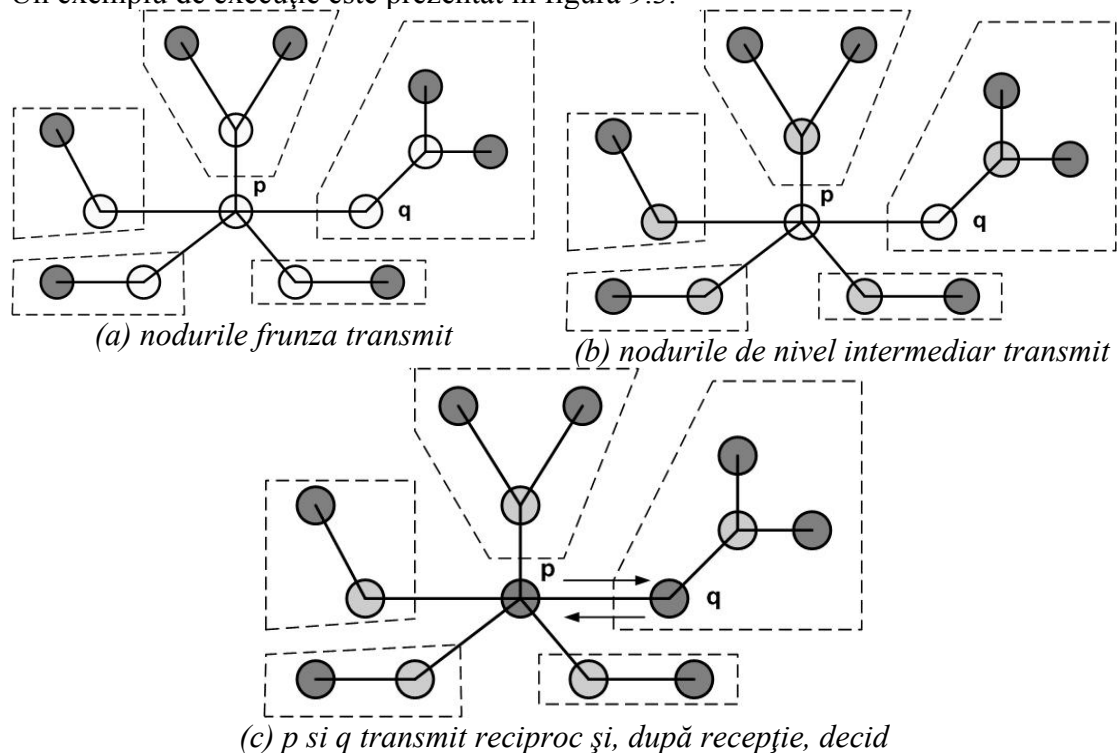


Figura 9.3 Exemplu de execuție al algoritmului arbore (Tel 1994)

În continuare, justificăm următoarea

Teoremă. Algoritmul "arbore" este un algoritm undă. (justificarea ignoră instrucțiunea inclusă în ultimul comentariu al descrierii algoritmului).

Demonstrația constă în analiza celor trei proprietăți fundamentale (terminare, decizie, dependență) ale algoritmilor undă.

(1) **calcul finit**

Deoarece fiecare proces trimite cel mult un mesaj, algoritmul folosește cel mult N mesaje. Ca urmare, algoritmul atinge o configurație terminală γ după un număr finit de pași.

(2) **în γ , cel puțin un proces a executat un eveniment "decide"**

Demonstrația se face prin reducere la absurd. Deoarece γ este terminală, în γ nu există nici un mesaj în tranzit.

Pentru n noduri avem $n-1$ legături bidirecționale între vecini, deci sunt $2n-2$ posibile recepții de mesaje (pentru care procesele așteaptă în bucla while după "r" din algoritm). Atunci când K procese au trimis mesaje (când execuția s-a terminat, deci s-a atins configurația γ), un număr de $F = 2n-2-K$ recepții rămân neconsumate.

Presupunem prin absurd că nici un proces nu a executat "decide" în γ . Asta înseamnă că fiecare proces este "blocat" la una din cele două operații **receive** din algoritm. Deoarece $n-K$ procese nu au trimis mesaj, ele așteaptă la primul receive, fiecare mai având de primit cel puțin 2 mesaje. K procese au trimis un mesaj, dar nici unul nu a decis (ele așteaptă la al doilea **receive**). Ca urmare, fiecare mai are de primit un mesaj. Rezultă de aici că $F \geq 2(n-K)+K$. Înlocuind F cu expresia sa din formula anterioară obținem:

$$\begin{aligned} 2n-2-K &\geq 2(n-K)+K \\ \text{sau} \quad -2 &\geq 0, \end{aligned}$$

ceea ce este absurd. Deci, cel puțin un proces a executat "decide" în γ .

(3) **"decide" este precedat de un eveniment in fiecare proces**

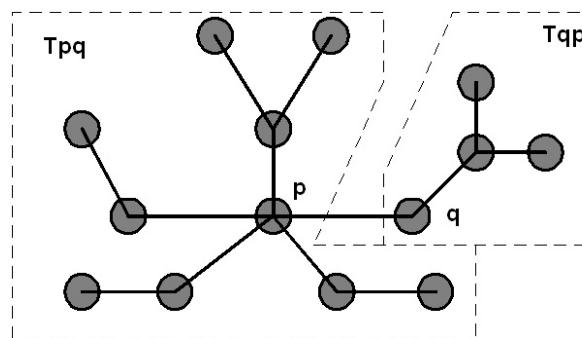


Figura 9.4 Subseturile T_{pq} și T_{qp} (Tel 1994)

Fie:

T_{pq} – subsetul proceselor tangibile din p fără a parcurge legătura pq

T_{qp} – subsetul proceselor tangibile din q fără a parcurge legătura qp

și evenimentele:

f_{pq} p trimite un mesaj lui q ,

g_{pq} q primește un mesaj de la p.

Se demonstrează prin inducție peste evenimentele de recepție că:

pentru orice s din T_{pq} există $e \in C_s$: $e \leq g_{pq}$

unde C_s este submulțimea evenimentelor produse în procesul s iar $\leq$ denotă relația de precedență. Presupunem că aceasta este adevărată pentru toate evenimentele *receive* care preced g_{pq} și demonstrăm că este adevărată și pentru g_{pq} .

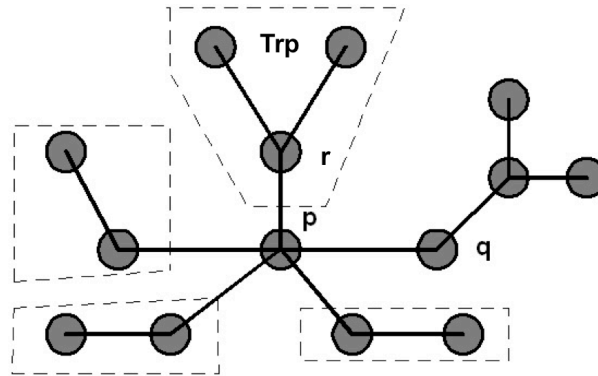


Figura 9.5 Descompunerea lui T_{pq} (Tel 1994)

Conform algoritmului, p trimite mesaj lui q după ce primește mesaje de la toți ceilalți vecini r. În figura 9.5 sunt arătați toți sub-arborii care au ca rădăcină acești vecini. Trp este unul dintre ei și raționamentele prezentate aici pentru Trp se pot aplica și pentru restul sub-arborilor din T_{pq} .

Conform inducției, pentru orice s din Trp , există un eveniment $e \leq g_{rp}$; apoi:

$g_{rp} \leq f_{pq}$ – p trimite mesaj lui q după ce primește mesaj de la r;

$f_{pq} \leq g_{pq}$ – mesajul este primit de p după ce este transmis de p;

rezultă $e \leq g_{pq}$

și prin extindere la ceilalți sub-arbori din T_{pq} rezulta că relația $e \leq g_{pq}$ este valabilă pentru orice proces din T_{pq} .

Demonstrația este similară pentru procesele din T_{qp} , unde:

$e \leq g_{qp}$ - pentru orice proces există un eveniment care precede recepția de către p a mesajului de la q;

$g_{qp} \leq d_p$ primirea mesajului de la q precede luarea deciziei de către p.

Rezultă că pentru orice proces s există e în C_s astfel că $e \leq d_p$.

9.3.3. Algoritm ecou

Algoritmul prezentat în continuare se aplică unor **topologii arbitrare de graf** și este **centralizat** (există un singur inițiator notat I în algoritm). Versiunea discutată aici a fost propusă de A. Segall în lucrarea “Distributed network protocols, IEEE Transactions on Information Theory, 29:23-35, 1983 (o variantă anterioară, mai puțin eficientă aparține lui Chang).

Ideea algoritmului este folosirea inundării rețelei cu mesaje **tok**. Inițiatorul trimite câte un mesaj tuturor vecinilor. La rândul lor, vecinii trimit câte un mesaj vecinilor lor și așa mai departe. Inițiatorul așteaptă să primească un răspuns de la fiecare vecin după care ia decizia. Pentru ca decizia inițiatorului să fie precedată de un eveniment în fiecare proces, se stabilește un **arbore de acoperire** peste topologia arbitrară de graf. Astfel, vecinul de la care un proces primește primul mesaj este marcat drept **părintele** său în arborele de acoperire. După primirea mesajului, procesul îl difuzează tuturor vecinilor mai puțin părintelui. La rândul lui, primește mesaje de la toți vecinii (mai puțin de la părinte, de la care a primit deja un mesaj). Când primește mesaje de la toți vecinii, trimite mesaj către părinte. Este ușor de văzut că trimerile mesajelor către părinți sunt ordonate, începând cu cele ale nodurilor frunză în arborele de acoperire și terminând cu mesajele trimise procesului rădăcină. Se asigură astfel că luarea deciziei de către inițiator este precedată de un eveniment în fiecare din celelalte procese.

```
chan ch[1:n] (int id, tok_type tok);
const int I = id_initiator;
process Proc(I){
    int r = numar_vecini_I;
    bool Vecini[1:n] = vecinii_lui_I;
    tok_type tok;
    int id;
    for [i=1 to n st Vecin[i]]
        send ch[i](I, tok);
    while (r>0){receive ch[I](id, tok);
                r = r-1; }
    decide;
}

process Proc[p=1 to n, p<>I]{
    int r = numar_vecini_p;
    bool Vecini[1:n] = vecinii_lui_p;
    tok_type tok;
```

```

int id, parinte;
receive ch[p](parinte, tok); r = r-1;
for [q=1 to n st Vecini[q] and q<>parinte]
    send ch[q](p, tok);
while (r>0){receive ch[p](id, tok);
            r = r-1};
send ch[parinte](p, tok);
}

```

Număr mesaje = $2|E|$

se trimite două mesaje pe fiecare legătură bi-direcțională.

Timp = $O(N)$

în fiecare etapă a algoritmului, cel puțin încă un nod (care până atunci nu a primit nimic) primește un prim mesaj.

9.3.4. Algoritmul fazelor

Algoritmul fazelor este **descentralizat**. El se aplică unor **topologii arbitrare** în care sunt evidențiate **canale unidirectionale** (digraf). Ca urmare, vecinii unui proces sunt grupați în două categorii: **in-vecini**, de la care procesul primește mesaje și **out-vecini**, cărora procesul le trimite mesaje. În algoritm se presupune că procesele cunosc diametrul grafului D (sau o valoare $D' > D$).

Ideea algoritmului este următoarea: fiecare proces trimite exact D mesaje fiecărui out-vecin. Funcționarea proceselor este împărțită în faze; în fiecare fază un proces p trimite un mesaj fiecărui out-vecin și primește un mesaj de la fiecare in-vecin. Astfel, după două faze, un proces primește informații propagate (prin intermediul in-vecinilor săi) de la procesele situate la distanța 2 față de el. După D faze, un proces primește mesaje de la toate procesele situate la distanță mai mică sau egală cu D (altfel spus, de la procesele de la care există o cale de distanță cel mult D până la procesul curent). După aceea procesul decide.

În descrierea următoare, există două categorii de procese: Inițiatori – care pornesc cu trimiterea unui mesaj fiecărui out-vecin și Ne-inițiatori – care așteaptă primirea unui prim mesaj și apoi trimit fiecărui out-vecin mesajele din prima fază a algoritmului.

```

chan ch[1:n] (int id, tok_type tok);
const int D = diametrul_retelei;

```

```

process Proc [p = 1 to n]{
  bool in[1:n] = in-vecinii_lui_p;
  /* in[i]==true daca i este invecin */
  bool out[1:n] = out-vecinii_lui_p;
  /* out[i]==true daca i este outvecin */
  int rec[1:n] = ([n] 0);
  /* rec[q] = numar mesaje primite de la q */
  int sent = 0;
  /* nr. mesaje transmise out-vecinilor */
  tok_type tok;
  int id;

  if (p este initiator){
    for [q = 1 to n st out[q]]{
      send ch[q](p, tok); sent = sent+1;}
  /*min(rec) = min(rec[q]),q=1..n and in[q])*/
  while (min(rec) < D){
    receive ch[p](id, tok);
    rec[id] = rec[id]+1;
    if (min (rec) >= sent and sent < D){
      for [q=1 to n st out[q]]
        send ch[q](p,tok);
        sent = sent+1;
      }
    }
  }
  decide;
}

```

Număr mesaje = $2D|E|$, unde E este multimea canalelor bidirecționale;

sunt D faze; în fiecare fază se trimite un mesaj pe fiecare direcție a canalelor.

Timp = $2D$

în fiecare fază, se trimit mesaje către out-vecini după ce au fost recepționate mesaje de la toți in-vecinii.

Comentariu. Dacă p este inițiator, el transmite mesaje tuturor out-vecinilor și face sent=1. În bucla **while**, el așteaptă apoi primirea unui set de mesaje de la in-vecini, până când primește cel puțin un mesaj de la fiecare in-vecin [$\min(\text{rec}) \geq 1$]. Când condiția este îndeplinită, el transmite mesajele din următoarea fază. De fiecare dată, trecerea la transmiterea mesajelor din faza k+1 se face după primirea de la toți in-vecinii a mesajelor din faza k.

Dacă p nu este inițiator, el intră direct în bucla **while**, unde așteaptă primirea unui prim mesaj. Deoarece, pentru el, $\min(\text{rec})=0$ și $\text{sent}=0$, procesul trece la transmiterea mesajelor din prima fază, după care așteaptă în buclă primirea mesajelor de la in-vecini. Comportamentul este apoi similar cu cel al proceselor inițitoare.

În continuare, justificăm următoarea

Teoremă. Algoritmul "fazelor" este un algoritm undă.

(1) calcul finit

Fiecare proces trimite cel mult D mesaje prin fiecare canal. Ca urmare, algoritmul atinge o configurație terminală γ după un număr finit de pași.

(2) în γ , fiecare proces a executat un eveniment "decide"

Considerăm că γ este o configurație terminală a unui calcul C și presupunem că există cel puțin un inițiator în C (pot fi mai mulți).

(2.1) fiecare proces a trimis cel puțin un mesaj

Există cel puțin un inițiator care transmite mesaje out-vecinilor săi. La rândul său, fiecare out-vecin trimite mesaje cel târziu când primește un mesaj.

(2.2) în γ fiecare proces a decis

Observație. Un proces decide atunci când iese din bucla **while**, deci atunci când condiția $\min(\text{rec}) < D$ nu mai este îndeplinită. Asta se întâmplă când $\min(\text{rec})$ devine egal cu D , adică procesul a primit mesaje de la toți in-vecinii în faza D . Cu alte cuvinte, nu există vreun proces care să fi trimis mai puțin de D mesaje. Pentru demonstrație, luăm în considerație procesul cu cel mai mic număr de mesaje trimise.

Fie p procesul cu cel mai mic sent_p în γ . Pentru orice proces q este îndeplinită relația $\text{sent}_q \geq \text{sent}_p$ în γ . În particular, ține dacă q este in-vecin al lui p adică $\min_q(\text{sent}_q) \geq \text{sent}_p$ pentru $q \in \text{in}$. Dar $\text{rec}_p[q] = \text{sent}_q$ (în γ , toate mesajele transmise au fost recepționate), deci $\min_q(\text{rec}_p[q]) \geq \text{sent}_p$. Rezultă $\text{sent}_p = D$; altfel p ar fi trimis mesaje suplimentare la ultima recepție (vezi decizia din algoritm). Rezultă $\text{sent}_p = D$ pentru toți p , deci $\text{rec}_p[q] = D$ pentru orice canal qp și fiecare proces a decis.

(3) "decide" este precedat de un eveniment în fiecare proces

Folosim notațiile:

$f_{pq}^{(i)}$ al i -lea eveniment "p transmite lui q"

$g_{pq}^{(i)}$ al i -lea eveniment "q primește de la p"

$\leq$ relația "precede pe"

Ne referim la decizia din procesul p . Deoarece D este diametrul, pentru orice q există o cale la p cu lungime $\leq D$, pe care o notăm $q = p(0), p(1), \dots, p(k) = p$, cu lungimea $k \leq D$. Într-o fază, transmisia precede recepția aceluiasi mesaj, deci

$$f_{p(i)p(i+1)}^{(i+1)} \leq g_{p(i)p(i+1)}^{(i+1)} \text{ pentru } 0 \leq i < k$$

Din algoritm, recepția unui mesaj precede transmiterea lui în faza următoare, adică

$$g_{p(i)p(i+1)}^{(i+1)} \leq f_{p(i+1)p(i+2)}^{(i+2)} \text{ pentru } 0 \leq i < k-1$$

Aplicând succesiv relațiile, rezultă $f_{p(0)p(1)}^{(1)} \leq g_{p(k-1)p(k)}^{(k)}$

În plus, din algoritm, $g_{p(k-1)p(k)}^{(k)} \leq d_p$ deci decizia lui p este precedată de un eveniment în orice q : $f_{qp(1)}^{(1)} \leq d_p$.

9.3.5. Algoritmul fazelor pentru clici

Într-o topologie **clică** (vezi Fig. 9.1b) cu n noduri, fiecare proces are $n-1$ vecini (la fiecare proces sunt conectate direct toate celelalte procese). Ca urmare, diametrul este 1, fiecare proces primind (direct) mesaje de la fiecare din celelalte procese. În total se transmit $n(n-1)$ mesaje, comunicarea lor făcându-se în paralel pe canale unidirectionale. Pentru evidența mesajelor primite, fiecare proces folosește o singură variabilă simplă.

Algoritmul este relativ simplu: fiecare proces trimite un mesaj tuturor vecinilor săi; inițiatorul începe execuția cu această operație, un alt proces făcând transmiterea imediat ce primește primul mesaj, de la un alt proces. În continuare, fiecare proces așteaptă primirea unui mesaj de la fiecare vecin și apoi decide.

```
chan ch[1:n] (int id, tok_type tok);
//canale unidirectionale
```

```
process Proc [p = 1 to n]{
    bool Vecini[1:n] = in-vecinii_lui_p;
    int n_vecini = numar_vecini_p;
    int rec = 0;
    //un mesaj primit per vecin
    int sent = 0;
    tok_type tok;
    int id;

    if (p este initiator){
        for [q = 1 to n st Vecini[q]]
            send ch[q](p, tok);
```

```

        sent = sent+1;
    }
    while (rec < n_vecini){
        receive ch[p](id, tok); rec = rec+1;
        if (sent == 0){
            for [q =1 to n st Vecini[q]]
                send ch[q](p, tok);
                sent = sent+1;}
        }
    decide;
}

```

Număr mesaje = $n(n-1)$

Timp = 2

transmiterea mesajelor se face in paralel.

9.3.6. Algoritmul lui Finn

Algoritmul nu cere cunoașterea diametrului. El se bazează pe cunoașterea identificatorilor proceselor, folosiți în mesajele transmise între procese.

Ideea: fiecare proces **p** păstrează două mulțimi de identificatori cu următoarele proprietăți:

- **Inc** – mulțimea proceselor **q** pentru care un eveniment în **q** precede cel mai recent eveniment din **p**;
- **NInc** – mulțimea proceselor **q** pentru care fiecare vecin **r** are un eveniment care precede cel mai recent eveniment din **p**.

Proprietățile mulțimilor **Inc** și **NInc** se mențin astfel:

- Initial

$Inc = \{p\}$

$NInc = \emptyset$

- In algoritm, procesul **p** trimite mesaje care conțin **Inc** și **NInc** de fiecare dată când unul din **Inc** sau **NInc** se modifică (crește) prin adăugarea unui nou identificator. Când **p** primește mesaje cu **Inc** și **NInc**, el adaugă noile procese la propriile **Inc** și **NInc** (actualizează **Inc** și **NInc** prin operații de reuniune). Când **p** a primit un mesaj de la toți in-vecinii, **p** este adăugat la **Ninc**.

Inc și **NInc** includ treptat identificatorii proceselor. Un proces **p** decide atunci când mulțimea **Ninc** pe care el o gestionează conține identificatorii tuturor proceselor.

Algoritmul este descentralizat. În algoritm se folosesc canale bidirecționale, similar algoritmului fazelor, iar graful care modelează topologia colecției de procese este tare conex. Fiecare proces își cunoaște in-vecinii și out-vecinii. În descriere introducem un nou tip, **SOP** care desemnează o mulțime de identificatori de procese.

```

typedef SOP {multime identificatori};
chan ch[1:n] (int id, SOP Inc, SOP NInc);
process Proc [p = 1 to n]{
    SOP Inc = {p};
    SOP NInc = ∅;
    bool rec[1:n] = ([n] false);
    /* de la cine a receptionat */
    bool out[1:n] = out-vecinii_lui_p;
    bool in[1:n] = in-vecinii_lui_p;
    SOP rInc, rNInc;
    int id;
    if (p este initiator)
        for [q = 1 to n st out[q]]
            send ch[q](p, Inc, NInc);

    while (Inc <> NInc){
        receive ch[p](id, rInc, rNInc);
        Inc = Inc U rInc; NInc = NInc U rNInc;
        rec[id] = true;
        if (rec[id], ∄id=1..n and in[id])
            NInc = NInc U {p};
        if (Inc sau NInc modificat)
            for[q=1 to n st out[q]]
                send ch[q](p, Inc, NInc);
    }
    decide;
}

```

Număr mesaje $\leq 4N|E|$

sunt $2|E|$ canale uni-direcționale; în cel mai rău caz, se modifică o singură mulțime dintre Inc și NInc prin adaugarea unei singure identități; rezultă că pe fiecare canal se transmit cel mult $2N$ mesaje

Timp = $O(D)$

timpul de propagare a unei schimbări petrecute în mulțimile Inc sau NInc este proporțional cu diametrul grafului care modelează topologia.

Teorema. Algoritmul lui Finn este un algoritm undă

(1) calcul finit

În algoritm, se trimite un mesaj la fiecare modificare a Inc sau $Ninc$. Numărul de elemente din cele două mulțimi luate împreună este inițial 1 și ajunge la $2N$. Ca urmare, numărul de mesaje transmise este limitat superior la $4N|E|$.

(2) în γ , fiecare proces a executat un eveniment "decide"

2.1. în γ fiecare proces a trimis cel puțin un mesaj pe fiecare canal

Când p trimite primul mesaj vecinilor săi, fiecare vecin q adaugă p la Inc_q și trimite un mesaj tuturor vecinilor săi. Fiecare vecin actualizează Inc și trimite un mesaj mai departe. Din faptul că graful este tare conex rezultă proprietatea menționată.

2.2. în γ fiecare proces a decis

2.2.1. pentru oricare proces p , Inc_p conține toate procesele în configurația γ .

Dacă există canalul unidirecțional pq , atunci orice modificare a lui Inc_p este trimisă de p lui q , care o include în Inc_q ; rezultă că Inc_q include Inc_p . Deoarece graful este tare conex, avem și Inc_p include Inc_q ; rezultă $Inc_p = Inc_q$ pentru orice p și q și că, pentru orice p , Inc_p conține identitățile tuturor proceselor.

2.2.2. pentru orice p și q , $NInc_p = NInc_q$

Fiecare proces p trimite un mesaj pe fiecare canal în γ . Ca urmare, fiecare proces p este recepționat de la toți vecinii (pentru orice $id \in in: rec[id]$). Conform algoritmului, p este adăugat la $NInc_p$ care este apoi difuzată celorlalte procese. Deoarece graful este tare conex și orice $NInc_p$ conține p , $NInc_p$ va conține toate procesele în γ . Deci, pentru orice p , $Inc_p = NInc_p$ și fiecare proces p decide în γ .

(3) "decide" este precedat de un eveniment în fiecare proces

Fie q un in-vecin al lui p și fie evenimentul intern lui q , a_q : $NInc = NInc \cup \{q\}$ care modifică $NInc_q$. Conform algoritmului, evenimentul a_p (intern procesului p) se produce dacă pentru fiecare in-vecin r al lui q un eveniment din r precede a_q .

Procesul p decide doar când $Inc_p = NInc_p =$ setul tuturor proceselor. Când procesul p ia decizia sunt îndeplinite următoarele:

- $p \in Inc$

- in-vecinul $q \in Inc$ și $q \in NInc$

Deci, decizia lui p este precedată de un eveniment din q și de un eveniment din fiecare din in-vecinii lui q .

Relația este valabilă pentru oricare două procese din care unul este in-vecinul celuilalt. Rețeaua fiind conexă, decizia lui p este precedată de un eveniment din fiecare din celelalte procese.

Capitolul 10. Alegerea unui lider

Alegerea unui lider este necesară ca fază premergătoare execuției unui algoritm centralizat, în care doar unul din procesele unei colecții urmează să execute operații de inițializare sau să joace rolul de coordonator în derularea algoritmului. Evident, se presupune că desemnarea statică a liderului nu este aplicabilă, de regulă pentru că nu se cunoaște compoziția exactă a grupului de procese. (Este cazul algoritmilor folosiți la refacerea unor sisteme după defectare - crash). În schimb, fiecare proces își cunoaște propria identitate și vecinii. Identitățile proceselor aparțin unei mulțimi total ordonate, astfel că alegerea liderului înseamnă determinarea procesului care are identitatea cea mai mică (sau cea mai mare).

Algoritmii de alegere a unui lider sunt descentralizați (mai multe procese pot declanșa alegerea) și presupun luarea unei decizii cu participarea tuturor proceselor. De aceea, algoritmii undă reprezintă soluții naturale ale problemei.

10.1. Alegerea unui lider cu algoritmi undă

Soluția are următoarele caracteristici:

- toate procesele au același algoritm local
- algoritmul este **descentralizat** (poate fi inițiat de oricare subset de procese)
- în fiecare calcul algoritmul atinge o **configurație finală** în care un proces este lider, toate celelalte au pierdut.

O variantă mai "slabă" este cea în care un proces este lider și este "conștient" de acest lucru, în timp ce celelalte procese nu "știu" ca au pierdut. Evident, liderul poate înștiința celelalte procese că el a câștigat, aducând astfel soluția la cazul precedent.

Fiecare proces are una din următoarele stări:

- sleep** – procesul nu a executat nici o acțiune
- leader** – procesul a câștigat alegerile
- lost** – procesul a pierdut.

În soluțiile următoare, sunt acceptate următoarele presupuneri:

- sistemul este asincron (comunicație asincronă, fără un ceas global)
- fiecare proces este identificat printr-un nume unic
- identitățile sunt extrase dintr-un **set total ordonat**
- aflarea liderului se concretizează în **aflarea procesului cu cel mai mic identificador**.

Soluția prezentată aici se aplică unei topologii arbore sau pe care s-a definit ad hoc un arbore de acoperire. Topologia arbore **impune ca** cel puțin toate frunzele să fie inițiatori. În realitate, doar unul sau unele procese, care nu sunt neapărat frunze în topologia ansamblului de procese, observă căderea liderului curent și ar putea declanșa alegerea unui nou lider. Pentru a putea folosi algoritmul arbore, se adaugă o fază de *wakeup*, în care procesul (procesele) care au observat căderea liderului curent trimite (trimit) mesaje *wakeup* tuturor celorlalte procese, inclusiv frunzelor. Odată trezite, procesele frunză inițiază alegerea noului lider.

Fiecare proces folosește variabilele:

ws – boolean, asigură că fiecare proces transmite mesaje *wakeup* cel mult o dată;

wr – int, contorizează mesajele de *wakeup* recepționate.

Când un proces a primit *wakeup* prin fiecare canal, el începe algoritmul de alegere pentru a determina procesul cu **cea mai mica** identitate. Când un proces decide, el află identitatea liderului și, în funcție de ea, procesul trece în starea **leader** sau **lost**.

```

chan ch[1:n](int id_transm, int id_mic);
chan wakeup[1:n]();
process Proc(p = 1 to n){
    bool ws = false;
    int wr = 0;
    int r = numar_vecini_p;
    bool Vecini[1:n] = vecinii_lui_p;
    bool rec [1:n] = ([n]*false);
    int v = p;
    enum state {sleep, leader, lost}
    state stare = sleep;
    int id;

    /* aici incepe faza wake-up */
    if (p este initiator){
        ws = true;
        for [q = 1 to r st Vecini[q]]
            send wakeup[q]();
    }
    while (wr < numar_vecini_p){
        receive wakeup[p](); wr = wr+1;
    }
}

```

```

        if (NOT ws) ->
        ws = true;
        for [q = 1 to n st Vecini[q]]
            send wakeup[q]();
    }

/* aici incepe algoritmul tree */
do (r>1){
    receive ch[p](q,id);
    rec[q] = true; r = r-1;
    V = min(V, id);
}

/* de la un singur vecin, q0 nu s-a primit mesaj */
afla q0: Vecini[q0] and NOT rec[q0];
send ch[q0](p, V);
receive ch[p](q0, id);
V = min (V, id);
if (V=p) stare = leader;
else stare = lost;
/*transmite vecinilor decizia*/
for [q = 1 to n st Vecini[q] and q<>q0]
    send ch[q](p,V);
}

```

Numar mesaje = $O(N)$

Pe fiecare canal se trimit două mesaje *wakeup* și două *token*-uri ceeace înseamnă un total de $4N-4$ mesaje.

Timp = $O(D)$

În D pași după ce primul proces pornește algoritmul, fiecare proces a trimis mesajele *wakeup*. Ca urmare, în $D+1$ pași fiecare proces a pornit unda. Prima decizie se ia la D pași după pornire, iar ultima după alți D pași. Rezultă totalul de $3D+1$ pași.

10.2 Algoritmi pentru topologia inel

Una din situațiile cele mai comune este cea în care procesele, identificate prin numere distincte între ele, sunt organizate într-o topologie inel.

Problema cere, ca și mai înainte, desemnarea prin consens a unui singur proces drept lider al grupului de procese. De data aceasta convenim ca el să fie procesul cu identificatorul **maxim**. Nu se cunoaște apriori numărul de procese din grup. De asemenea, dispunerea proceselor în inel este oarecare (nu neapărat în ordinea numerelor de identificare). În fine, nu există un control centralizat al alegerii liderului (soluția este distribuită).

Exemplu de utilizare: un inel de controlare în care jetonul de control (care asigură protocolul de coordonare a operațiilor din inel) s-a pierdut. Situația este detectată printr-un timeout, la care fiecare controler declanșează procedura de selecție a liderului, a cărei sarcină este de a genera un nou jeton.

10.2.1 Algoritmul LeLann

Algoritmul realizează următoarele acțiuni:

1. Fiecare proces transmite în inel un mesaj care conține identificatorul său
2. Fiecare proces colectează identificatorii celorlalte procese
3. Fiecare proces calculează maximum.
4. Procesul al cărui număr este egal cu maximum devine lider.

Complexitate $O(n^2)$ mesaje comunicate - fiecare proces transmite un mesaj care trece pe la toate celelalte.

Descrierea detaliată a algoritmului este următoarea.

```

chan ch[Ids](token tok, int id_mare);
process Proc(p = 1 to n){
typedef SOP {multime identificatori};
SOP List = {p};
enum state (candidate, leader, lost);
state stare = candidate;
send ch[Urm](tok, p);
receive ch[p](tok, q);
while (q <> p) List = List U {q};
    send ch[Urm](tok, q);
    receive ch[p](tok, q);
}
if (p == max(List)) stare = leader;
else stare = lost;

```

Algoritmul se bazează pe proprietatea FIFO a canalelor, conform căreia ordinea mesajelor transmise pe înel se păstrează. Astfel, oricare proces **p** va trimite mesajul propriu **(tok, p)** urmat în ordine de mesajele **(tok, q)** primite de la alte procese. Având în vedere topologia inel folosită, mesajul propriu va ajunge înapoi la procesul **p** după ce acesta a primit și retrimis mesaje de la toate celelalte procese. Deci, când **(tok, p)** ajunge înapoi la procesul **p**, variabila sa **List** va conține identificatorii tuturor proceselor. Acest lucru este valabil pentru toate procesele, urmarea fiind că toate vor identifica un același maxim, deci un același nou lider al colecției de procese.

10.2.2 Algoritmul Lelann-Chang-Robert

Algoritmul LeLann nu este prea eficient, procesele având de găsit, în paralel, maximum dintr-o mulțime de identificatori. Varianta **Lelann-Chang-Robert** ocolește acest neajuns prin eliminarea pe parcurs a valorilor mai mici, în paralel cu pasarea mesajelor mai mari de la un proces la altul. Mesajele sunt transmise în inel **în sensul acelor de ceasornic**. Astfel:

1. Fiecare proces **p** transmite procesului aflat în **dreapta** un mesaj cu identificatorul propriu, **p**.
2. Un proces **p** care primește un mesaj **q** îl compară cu identificatorul propriu **p**
 - a. **if** $q > p$ -> transmite **q** în dreapta
 - b. **if** $q < p$ -> elimină **m** (nu transmite nimic în dreapta)
 - c. **if** $q = p$ -> procesul curent devine lider.

Propozitie: Algoritmul detectează un singur cel mai mare număr.

Argument: Un mesaj întâlnește toate celelalte procese înainte de a reveni, eventual, la transmițător. Numai mesajul cu valoarea cea mai mare nu este eliminat de procesele parcurse.

Descrierea detaliată a algoritmului este următoarea.

```

chan ch[1:n](token tok, intid_mare);
process Proc(p = 1 to n){
enum state (candidate, leader, lost);
state stare = candidate;
send ch[Urm](tok, p);
while (state <> leader){
    receive ch[p](tok, q);
    if (q==p) stare = leader;

```

```

    else if (q>p){
        if (stare == candidate){stare = lost;
            send ch[Urm](tok, q);
        }
    }
}

```

Observații

Condiția de ieșire din buclă este "state = leader". Aceasta are două consecințe:

- un proces care devine **lost** rămâne în buclă pentru a pasa alte mesaje pe care le primește
- doar procesul cu identificatorul maxim termină; celelalte vor rămâne blocate în așteptarea unui token; pentru deblocare, liderul ales poate trimite un mesaj special prin care anunță identificatorul lui și faptul că alegerea s-a terminat.

Algoritm modificat

În cazul în care nu toate procesele sunt inițiatori, se pot introduce următoarele modificări:

- Cel puțin un proces inițiază alegerea și se auto-marchează.
- Când un mesaj **m** ajunge la un proces nemarcat, acesta va genera un mesaj cu **id** propriu, se va marca și va continua cu prelucrarea mesajului **m**.
- Liderul ales asigură că fiecare proces se va demarca (pentru execuția corectă a unei viitoare alegeri de lider).

Performanța

Măsurată în timp și număr de mesaje

Timp

Dacă toate procesele pornesc simultan, este necesar un singur ciclu prin inel. Deci, timpul este $O(n)$, unde **n** este numărul de procese.

Similar, dacă procesul cu numărul maxim pornește primul timpul este $O(n)$. Altfel, în cel mult $n-1$ pași mesajul de marcare ajunge la procesul cu **id** maxim și în n pași se face alegerea acestuia ca lider. Deci timpul este $O(n)$.

Număr mesaje

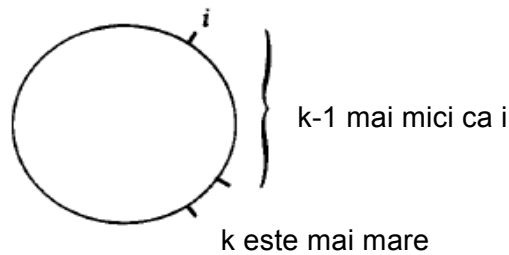
a) *Cazul cel mai bun*. Procesele sunt ordonate crescător în sensul acelor de ceasornic. Fiecare mesaj (exceptând mesajul **n**) este transmis o dată. Mesajul **n** este transmis de **n** ori. Rezulta $2n-1$ mesaje.

b) *Cazul cel mai rău.* Procesele sunt ordonate descrescător în sensul acelor de ceasornic. Toate mesajele sunt eliminate de procesul n.

Mesajul i este transmis de i ori. Rezultă

$$\sum_{i=1,n} i = n(n+1)/2.$$

c) *Cazul mediu.*



Sunt **i-1** procese mai mici ca **i** și **n-i** mai mari ca **i**. Fie $C(a,b)$ numărul de moduri în care putem alege b obiecte din a (combinări de a luate câte b).

Probabilitatea $P(i, k)$ ca mesajul **i** sa fie pasat de **k** ori este egală cu probabilitatea ca cei k-1 vecini ai lui i în sensul acelor de ceasornic să fie mai mici ca i și vecinul k să fie mai mare, adică:

$$P(i,k) = [C(i-1,k-1)/C(n-1,k-1)] * [(n-i)/(n-k)]$$

Cunoscând că mesajul n se transmite întotdeauna de n ori și că există doar un singur asemenea mesaj, considerăm doar n-1 mesaje, fiecare fiind transmis de cel mult n-1 ori. Numărul probabil de transmiteri pentru mesaje altele decât n este:

$$E_i(k) = \sum_{k=1,n-1} k.P(i,k)$$

Numărul total de transmiteri pentru toate mesajele este:

$$E(k) = n + \sum_{i=1,n-1} \sum_{k=1,n-1} k.P(i,k)$$

sau

$$E(k) = n + \sum_{k=1,n-1} n/(k+1) = n(1 + 1/2 + 1/3 + \dots + 1/n) \approx n(C + \ln n)$$

Rezultă că numărul mediu de mesaje este **O(n ln n)**.

10.2.3 Algoritmul Hirschberg-Sinclair

Spre deosebire de algoritmul Chang si Roberts care are o complexitate medie $O(n \ln n)$, algoritmul Hirschberg-Sinclair are complexitatea $O(n \ln n)$ în cel mai defavorabil caz.

El lucrează pe un inel bidirecțional și presupune că procesele pot detecta din ce direcție vine un mesaj astfel că pot trimite răspunsuri către transmițător. Ca urmare, operațiile de comunicare folosite de procese au o semantică particulară, diferită de cea definită de noi anterior și anume:

- un proces poate iniția mesaje în ambele direcții prin *sendboth*.
- un proces poate pasa un mesaj (eventual modificat) prin *sendpass*.
- un proces poate trimite un răspuns în direcția din care a primit un mesaj prin *sendecho*.

Algoritmul operează în faze. În **faza k**, un proces inițiază mesaje care sunt transmise în ambele direcții pe căi de lungime 2^k (1, 2, 4, 8 ...). Procesele de pe o cale analizează mesajul. Dacă un proces decide că nu poate câștiga alegerea el va pasa mesajul și nu va mai iniția mesaje proprii. Dacă procesul decide că sursa nu poate câștiga alegerea, transmite un ecou negativ sursei. Ultimul proces din calea 2^k transmite un ecou pozitiv sursei (dacă nu poate câștiga alegerea).

Un proces care primește propriul mesaj câștigă; în acest caz, calea “acoperă” toate procesele. El va trimite apoi un mesaj pentru a anunța celelalte procese că a câștigat.

Algoritmul are descrierea dată în continuare.

Execuția pentru alegere:

```
stare = candidate
maxnum = 1          /* lungimea caii curente */
while (stare == candidate){
    sendboth ("from", myvalue, 0, maxnum);
    /* 0 = pasii facuti pe calea curenta */
    await both replies (but react to other messages);
    if (either reply is "no")
        stare = lost;
    maxnum = 2*maxnum
}
```

La recepție mesaj ("from", value, num, maxnum):

```
if (value < myvalue) sendecho ("no", value);
else if (value > myvalue){
    status = lost;          /* nu va mai trimite mesajele sale */
    num = num + 1;          /* a mai facut un pas */
    if (num < maxnum)
        sendpass ("from",value, num, maxnum);
    else sendecho ("ok", value);
        /* ultimul proces din cale trimite OK */
else status = won;
```

La recepție mesaj ("no", value) sau ("ok", value):

```
if (value <> myvalue) sendpass the message;
else this is a reply the processor was awaiting;
```

Netratat în algoritm - acțiunile unui proces neinițiator (care nu "știe" de alegerea în curs): la primirea unui mesaj devine conștient de alegerea în curs și poate deveni *candidate* dacă identitatea are o valoare mai mare decât sursa mesajului.

Analiza complexității

Un proces x inițiază mesaje pe căi de lungime 2^i numai dacă el nu este "învins" de un proces aflat la distanța 2^{i-1} (în oricare direcție) de x . În orice grup de $2^{i-1} + 1$ procese consecutive, cel mult unul poate iniția mesaje pe căi de lungime 2^i . Deși este posibil ca toate cele n procese să inițieze căi de lungime 1, cel mult $\lceil n/2 \rceil$ (adică *superior* de $n/2$) procese vor iniția căi de lungime 2, cel mult $\lceil n/3 \rceil$ de lungime 4, cel mult $\lceil n/5 \rceil$ de lungime 8 etc.

Un proces care inițiază mesaje pe căi de lungime 2^i provoacă trimiterea de mesaje în ambele sensuri, dus-întors. Ca urmare, vor fi transmise cel mult $4 \cdot 2^i$ mesaje. Numărul total de mesaje este cel mult

$$4 \cdot (1 \cdot n + 2 \cdot \lceil n/2 \rceil + 4 \cdot \lceil n/3 \rceil + 8 \cdot \lceil n/5 \rceil + \dots + 2^i \cdot \lceil n/(2^{i-1} + 1) \rceil + \dots).$$

Fiecare din termenii din paranteze este mai mic decât $2n$ și există nu mai mulți de $1 + \lceil \log n \rceil$ termeni. Motivul este ca nici un proces nu va pasa mesaje pe căi de lungime $2n$ sau mai mari deoarece, odată ce un proces a inițiat căi de lungime cel puțin n și mesajul este acceptabil pe toată lungime inelului, procesul devine lider și nu mai

inițiază mesaje. Ca urmare, numărul total de mesaje transmise este mai mic decât $8n + 8[n \log n] = O(n \log n)$.

Pentru **complexitatea în timp**, considerăm că transmiterea mesajelor se suprapune în timp (atât cât permite algoritmul) și că fiecare mesaj este transmis în cel mult o unitate de timp. Ca urmare, complexitatea este lineară cu numărul de procese. Mai precis:

$$\text{Timp} = 2*(1 + 2 + 4 + \dots + 2^i + \dots + n).$$

Când n este o putere alui 2 (cazul cel mai favorabil):

$$\text{Timp} = 4n - 2.$$

Când n este cu 1 mai mare decât o putere a lui 2 (cazul cel mai defavorabil):

$$\text{Timp} = 6n - 6.$$

Bibliografie

Ernest Chang and Rosemary Roberts *An Improved Algorithm for Decentralized Extrema-Finding in Circular Configurations of Processes*, Communications of **the** ACM Number 5 Volume 22 May 1979

D.S. Hirschberg and J.B. Sinclair *Decentralized Extrema-Finding in Circular Configurations of Processors*, Communications of **the** ACM Number 11 Volume 23 November 1980

Gary L. Peterson *An $O(n \log n)$ Unidirectional Algorithm for the Circular Extrema Problem*, ACM Transactions on Programming Languages and Systems, Vol. 4, No. 4, October 1982, Pages 758-762.